

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Hanoi, August 2019 –

Table of Contents

I. Record of Changes..... 3

II. Software Design Document..... 4

1. System Design..... 4

1.1 System Architecture..... 4

1.2 Package Diagram..... 4

2. Database Design..... 4

3. Detailed Design..... 5

3.1 <Feature/Function Name1>..... 5

3.2 <Feature/Function Name2>..... 6

I. Record of Changes

Date	A* M, D	In charge	Change Description

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

SMGS follows a client–server architecture with separate visitor and staff clients connected to a central backend. The Flutter mobile application supports visitors during on-site museum visits. A shared React web portal provides role-based functions for Administrators, System Staff, Museum Staff, and Curators. Both clients communicate with the ASP.NET Core Web API over HTTPS.

The backend handles authentication and authorization, museum and artifact data, content review and publication, visit and tour records, quizzes, feedback, support, and commerce. It stores system data in PostgreSQL. AI and other external services provide image recognition, narration, conversational answers, and payment processing. Google Maps supports directions to museums. Indoor maps and routes are managed by SMGS.

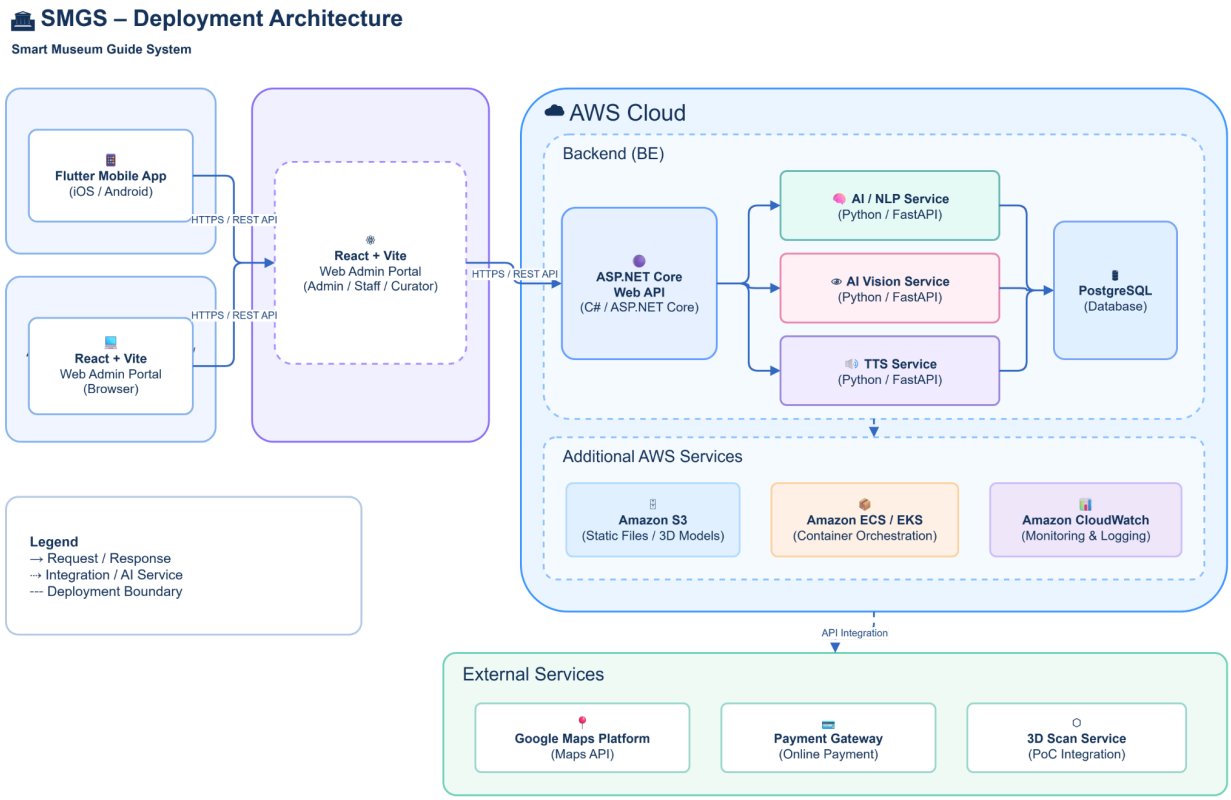


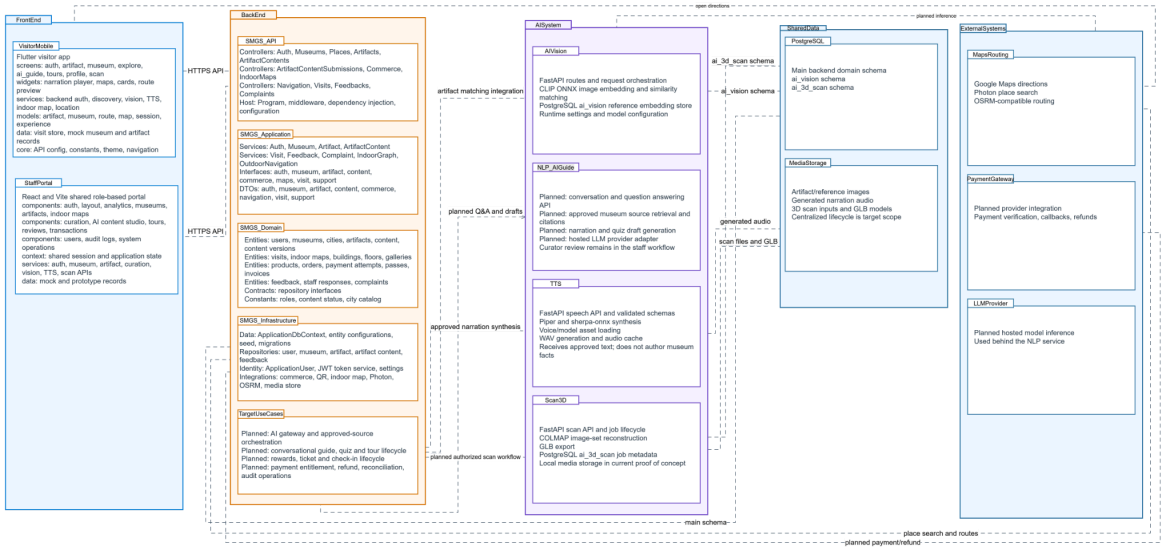
Diagram components:

- Flutter Mobile App: Lets visitors discover museums and artifacts, identify artifacts by QR code or image, view published content, interact with the guide, take quizzes, follow tours, purchase products, and submit feedback or support requests.

- React Web Portal: Provides role-based tools. Museum Staff manage museum-scoped information and create content drafts; Curators review and publish content; System Staff handle support, refunds, reconciliation, and monitoring; Administrators manage users, roles, museums, and system settings.
- ASP.NET Core Web API: Applies authentication, role-based permissions, and business rules. It coordinates client requests with PostgreSQL and the connected services.
- PostgreSQL: Stores user, museum, artifact, content-version, visit, tour, quiz, feedback, and transaction data. AI Vision and 3D Scan use separate schemas in the shared database.
- AI Vision Service: Compares visitor photos with indexed artifact images and returns a matching artifact candidate.
- AI / NLP Service: Supports conversational answers and draft narration or quiz generation using approved museum source information. Live chatbot responses are generated at request time; narration and quiz content intended for publication go through curator review.
- TTS Service: Converts approved narration text into audio for playback in the mobile application.
- Online Payment Gateway: Processes payment requests and returns transaction results. The backend records attempts and grants access after successful verification.
- Google Maps: Provides outdoor directions from a visitor's location to supported museums.
- 3D Scan Service: Reconstructs artifact models from uploaded images. It is a proof of concept and is not yet connected end to end to the main application.

1.2 Package Diagram

[Provide the package diagram for each sub-system. The content of this section includes overall package diagram(s) and the explanation for each package (or namespace). Please see the sample and description table format below]



Package Descriptions

No	Package	Description
01	`FrontEnd`	Contains the Flutter visitor application and the React/Vite staff portal.
02	`FrontEnd.VisitorMobile`	Flutter app for visitor authentication, museum and artifact discovery, QR/image recognition, approved artifact content, narration, 3D viewing, AI guide, tours, quizzes, maps, tickets, feedback, and support. Its packages include screens, widgets, services, models, data, and core configuration. Some flows are partial or mock-backed.
03	`FrontEnd.StaffPortal`	Shared React/Vite portal for Museum Staff, Curators, Platform Administrators, and System Staff. Its packages include app/routing, components, context, services, and mock data. Some workflows are prototypes or only partially integrated.
04	`BackEnd`	ASP.NET Core backend subsystem organized into API, Application, Domain, and Infrastructure projects, with additional report-scope use cases identified as planned or incomplete.
05	`BackEnd.SMGS.API`	Hosts the HTTP API, middleware, configuration, and controllers for authentication, museums, artifacts, content review, commerce, maps, navigation, visits, feedback, and complaints.
06	`BackEnd.SMGS.Application`	Contains application services, interfaces, and DTOs for authentication, museum and artifact catalog, content, commerce, navigation, visits, feedback, and support.
07	`BackEnd.SMGS.Domain`	Defines domain entities, repository contracts, roles, and content-status constants for museum, artifact, visit, indoor-map, commerce, feedback, and complaint data.
08	`BackEnd.SMGS.Infrastructure`	Implements database access, repositories, identity, QR generation, commerce and indoor-map services, media handling, place search, and routing integrations.
09	`BackEnd.TargetUseCases`	Report-scope packages for AI orchestration, conversational guidance, quizzes, tours, rewards, ticket/check-in, access rights, payment, refunds, reconciliation, and audit. Several are planned or incomplete in the current source.
10	`AI-System`	Contains the separate AI microservices for artifact image recognition, NLP/AI guidance, speech synthesis, and 3D reconstruction.
11	`AI-System.AIVision`	FastAPI service that embeds and matches artifact images with CLIP ONNX and stores reference embeddings in the `ai_vision` schema. Implemented; it does not train a custom model.

12	`AI-System.NLP_AIGuide`	Planned service for conversational question answering and narration/quiz draft generation grounded in approved museum sources. The service is not implemented in the repository; curator review remains a human workflow.
13	`AI-System.TTS`	FastAPI speech service that synthesizes supplied narration text using Piper/sherpa-onnx, loads voice assets, and generates/caches audio. Implemented as a standalone service; it does not author or approve museum content.
14	`AI-System.Scan3D`	FastAPI proof of concept for scan jobs, COLMAP image-set reconstruction, and GLB export. Job metadata uses `ai_3d_scan`; media handling and publication are not integrated end to end
15	`SharedData`	Groups the shared PostgreSQL database and media/model storage used by the product and AI services.
16	`SharedData.PostgreSQL`	Shared database instance containing the backend domain schema and separate `ai_vision` and `ai_3d_scan` schemas. TTS does not connect to the database.
17	`SharedData.MediaStorage`	Stores or serves artifact/reference images, narration audio, 3D scan inputs, and GLB models. The current 3D proof of concept uses local files; a centralized lifecycle is target scope.
18	`ExternalSystems`	Groups external map, routing, payment, and hosted language-model dependencies.
19	`ExternalSystems.MapsRouting`	Represents Google Maps directions, Photon place search, and OSRM-compatible route calculation.
20	`ExternalSystems.PaymentGateway`	Planned payment-provider integration for payment verification, callbacks, and refunds. No provider adapter was found in the current backend source.
21	`ExternalSystems.LLMProvider`	Planned hosted inference provider called by the NLP/AI Guide service; approved museum content remains the source for factual answers.

2. Database Design

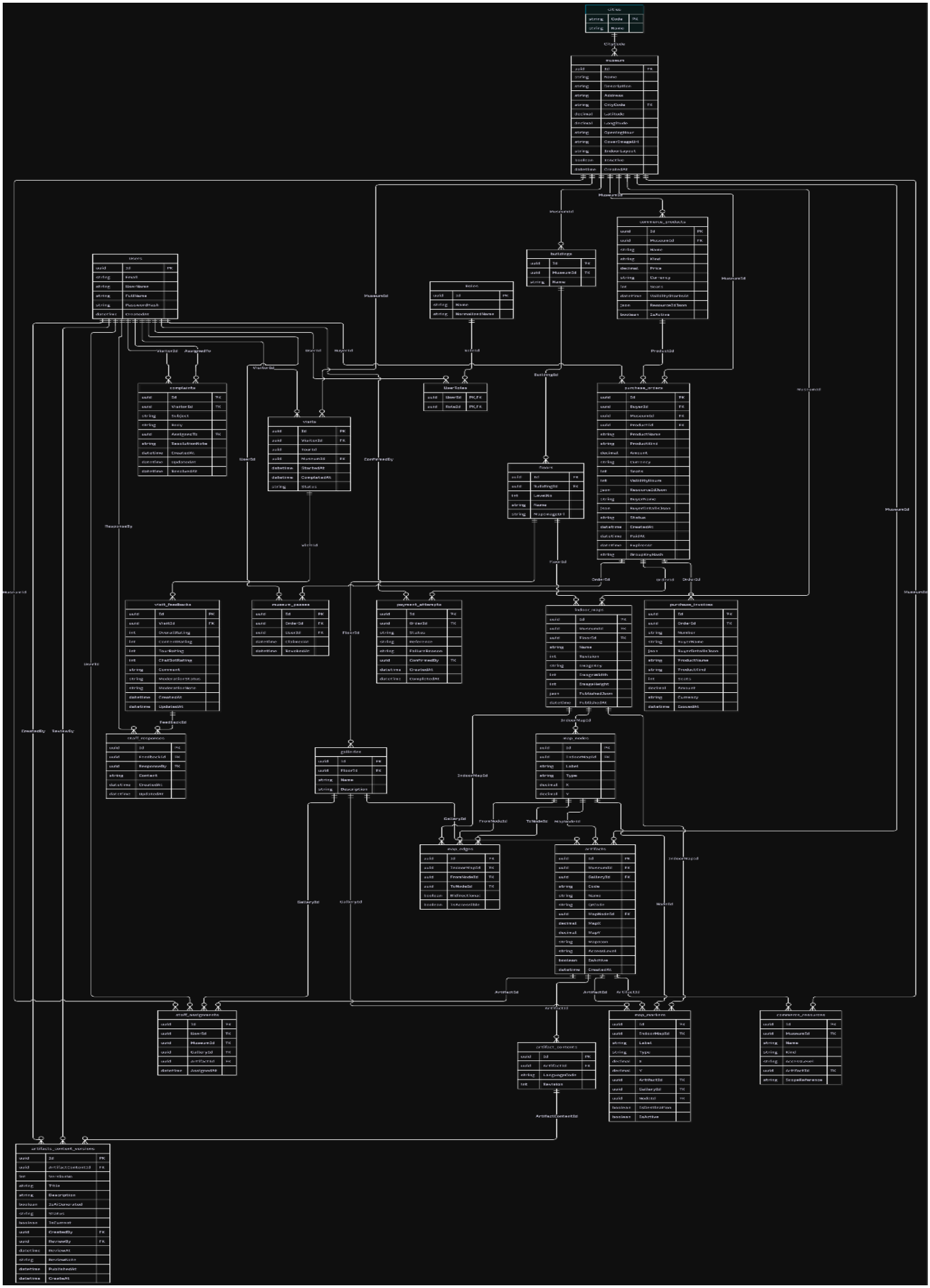


Table Descriptions

No	Table	Description
01	Users	Stores system user accounts including administrators, staff, curators, and visitors. - **Primary keys:** Id - **Foreign keys:** None
02	Roles	Stores user roles for authorization purposes. - **Primary keys:** Id - **Foreign keys:** None
03	UserRoles	Mapping table for the many-to-many relationship between Users and Roles. - **Primary keys:** UserId, RoleId - **Foreign keys:** UserId (References Users), RoleId (References Roles)
04	cities	Stores city master data for geographic categorization. - **Primary keys:** Code - **Foreign keys:** None
05	museum	Stores main information about museums including location, operations, and cover images. - **Primary keys:** Id - **Foreign keys:** CityCode (References cities)
06	buildings	Stores buildings belonging to a specific museum. - **Primary keys:** Id

		- **Foreign keys:** MuseumId (References museum)
07	floors	Stores floor levels within a museum building along with map imagery. - **Primary keys:** Id - **Foreign keys:** BuildingId (References buildings)
08	galleries	Stores specific galleries, rooms, or exhibition sections on a given floor. - **Primary keys:** Id - **Foreign keys:** FloorId (References floors)
09	artifacts	Stores physical artifacts displayed within galleries, including QR codes and map locations. - **Primary keys:** Id - **Foreign keys:** MuseumId, GalleryId, MapNodeId
10	artifact_contents	Stores multi-language content headers for specific artifacts. - **Primary keys:** Id - **Foreign keys:** ArtifactId (References artifacts)
11	artifacts_content_versions	Tracks version history, AI generation status, and curator reviews for artifact text/audio content. - **Primary keys:** Id - **Foreign keys:** ArtifactContentId, CreatedBy (Users), ReviewBy (Users)
12	staff_assignments	Tracks the assignment of museum staff to specific museums, galleries, or artifacts. - **Primary keys:** Id - **Foreign keys:** UserId, MuseumId, GalleryId, ArtifactId
13	visits	Records physical visits and tour sessions undertaken by visitors. - **Primary keys:** Id - **Foreign keys:** VisitorId (Users), MuseumId

14	visit_feedbacks	Stores visitor feedback and ratings (overall, content, tour, chatbot) after a visit. - **Primary keys:** Id - **Foreign keys:** VisitId (References visits)
15	staff_responses	Stores replies from museum staff to visitor feedback. - **Primary keys:** Id - **Foreign keys:** FeedbackId, ResponseBy (Users)
16	complaints	Stores issues/complaints raised by visitors and tracks resolution status by assigned staff. - **Primary keys:** Id - **Foreign keys:** VisitorId (Users), AssignedTo (Users)
17	commerce_resources	Defines specific sellable entities or access scopes within the museum. - **Primary keys:** Id - **Foreign keys:** MuseumId, ArtifactId
18	commerce_products	Defines commercial products, ticket packages, or services available for visitors to purchase. - **Primary keys:** Id - **Foreign keys:** MuseumId
19	purchase_orders	Stores transaction records for visitor purchases, amounts, and validity. - **Primary keys:** Id - **Foreign keys:** BuyerId (Users), MuseumId, ProductId
20	payment_attempts	Logs payment gateway interaction results for specific orders. - **Primary keys:** Id - **Foreign keys:** OrderId, ConfirmedBy (Users)
21	museum_passes	Represents the actual tickets/passes claimed by users post-purchase. - **Primary keys:** Id

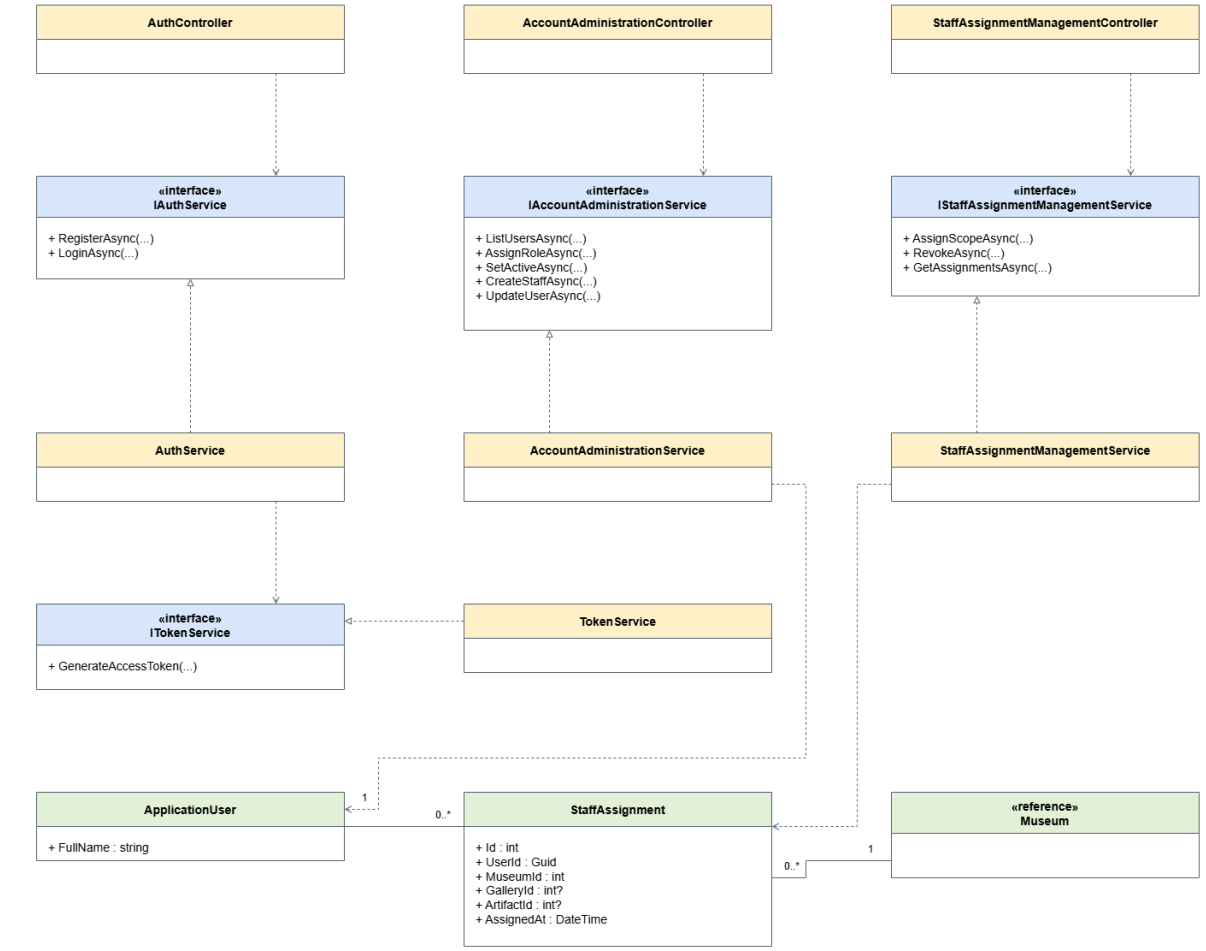
		- **Foreign keys:** OrderId, UserId
22	purchase_invoices	Stores generated billing invoices and buyer details for completed transactions. - **Primary keys:** Id - **Foreign keys:** OrderId
23	indoor_maps	Manages indoor map configuration, coordinates, and rendering parameters for museum floors. - **Primary keys:** Id - **Foreign keys:** MuseumId, FloorId
24	map_nodes	Stores positional nodes (X, Y) on the indoor map used for navigation/routing. - **Primary keys:** Id - **Foreign keys:** IndoorMapId
25	map_edges	Defines paths/connections between map nodes for route calculation. - **Primary keys:** Id - **Foreign keys:** IndoorMapId, FromNodeId, ToNodeId
26	map_markers	Stores points of interest (destinations, artifacts, galleries) linked to map nodes. - **Primary keys:** Id - **Foreign keys:** IndoorMapId, ArtifactId, GalleryId, NodeId

3. Detailed Design

3.1 Authentication & Account Management

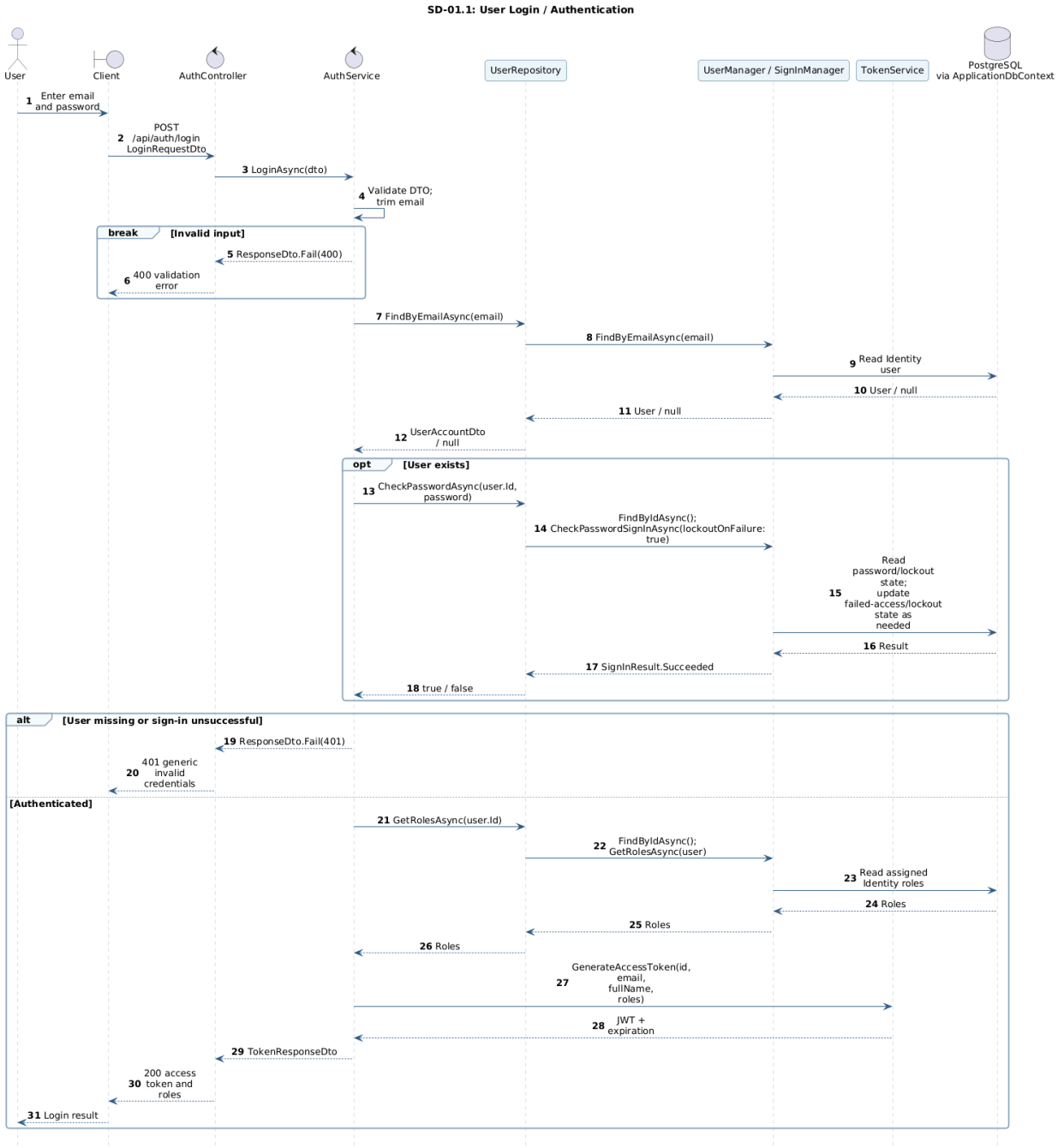
3.1.1 Class Diagram

CD-01 Class Diagram — Authentication & Account Management



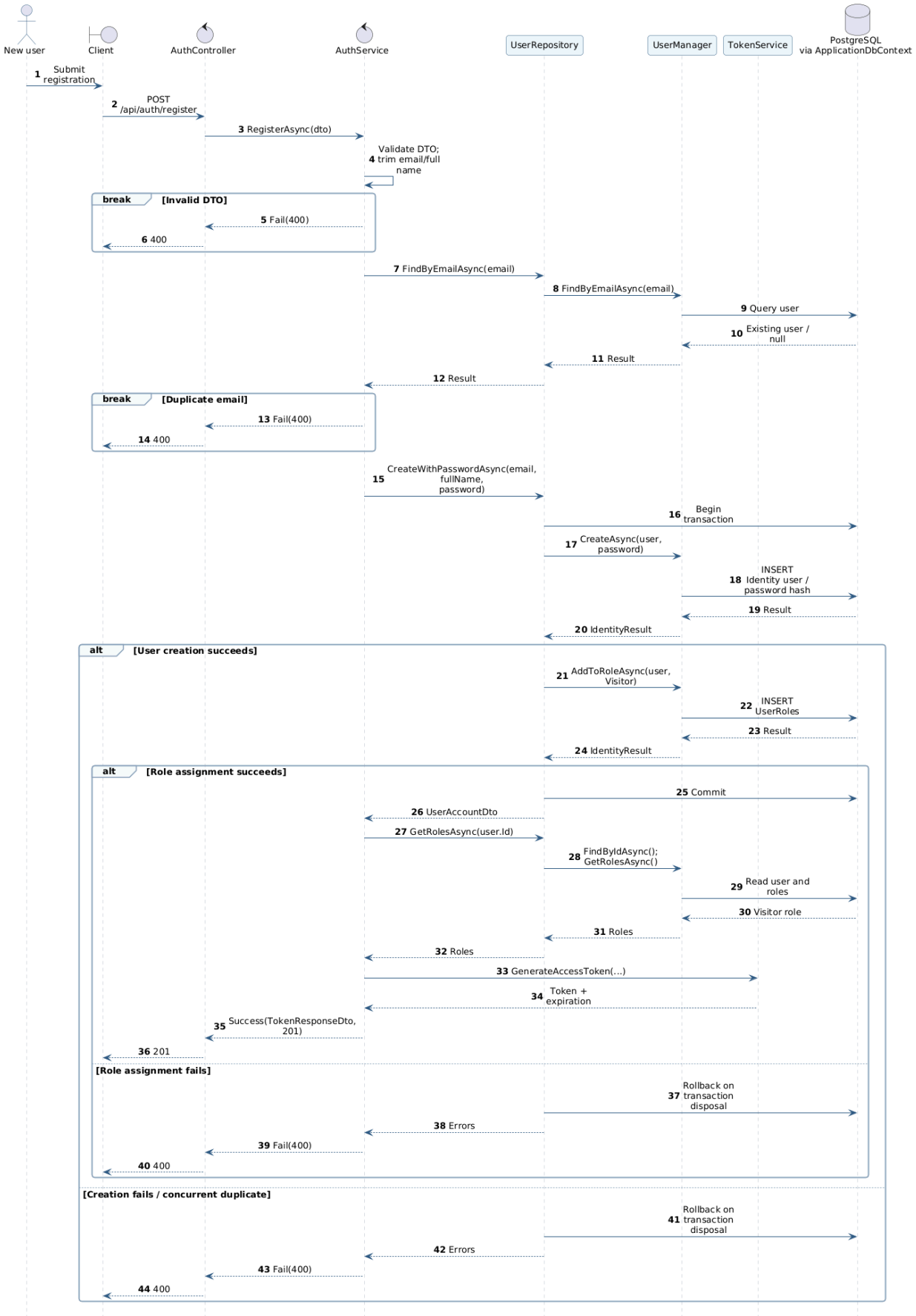
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.1.2 Sequence Diagrams-User Login / Authentication

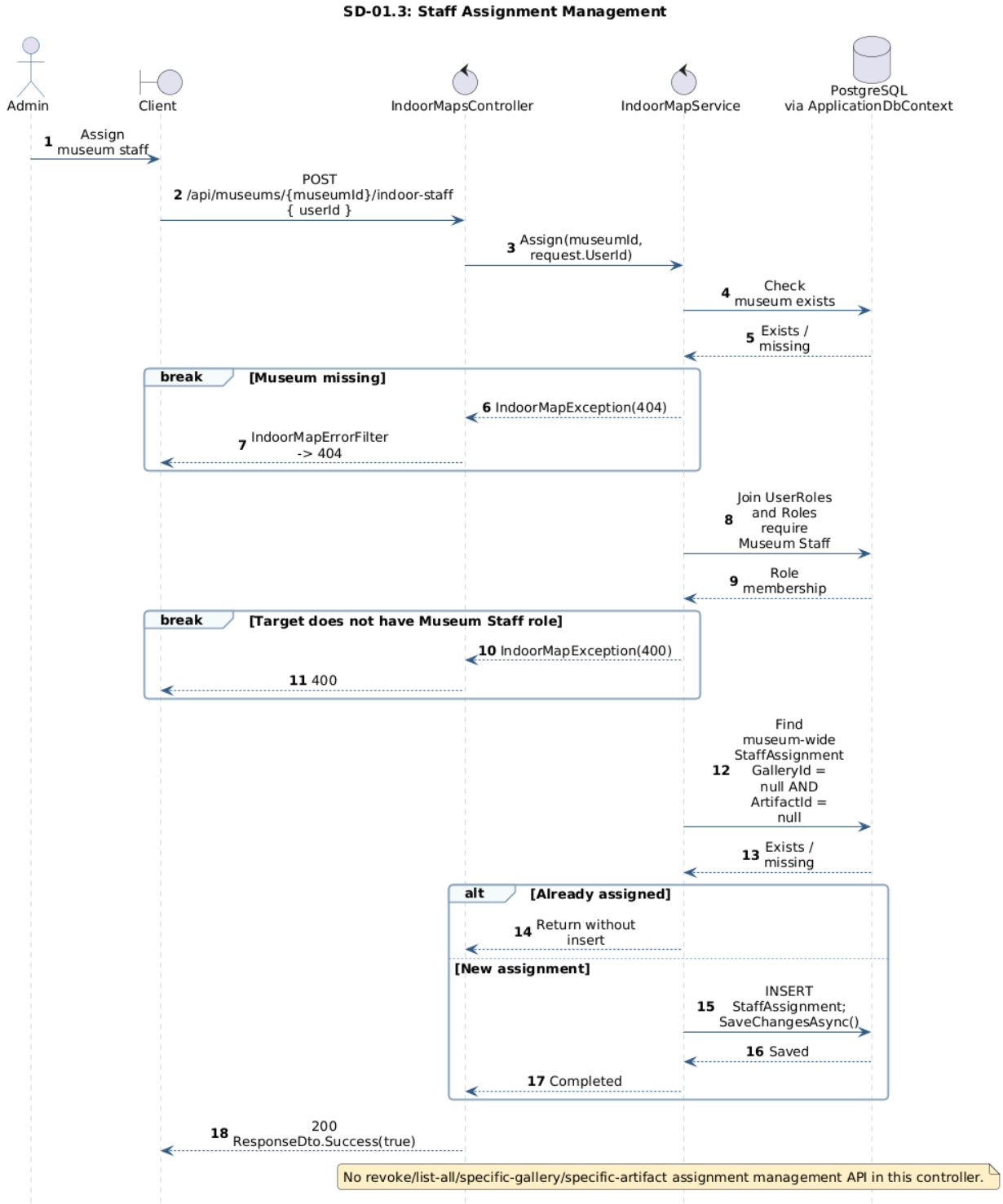


3.1.3 Sequence Diagrams-User & Role Management

SD-01.2: User & Role Management



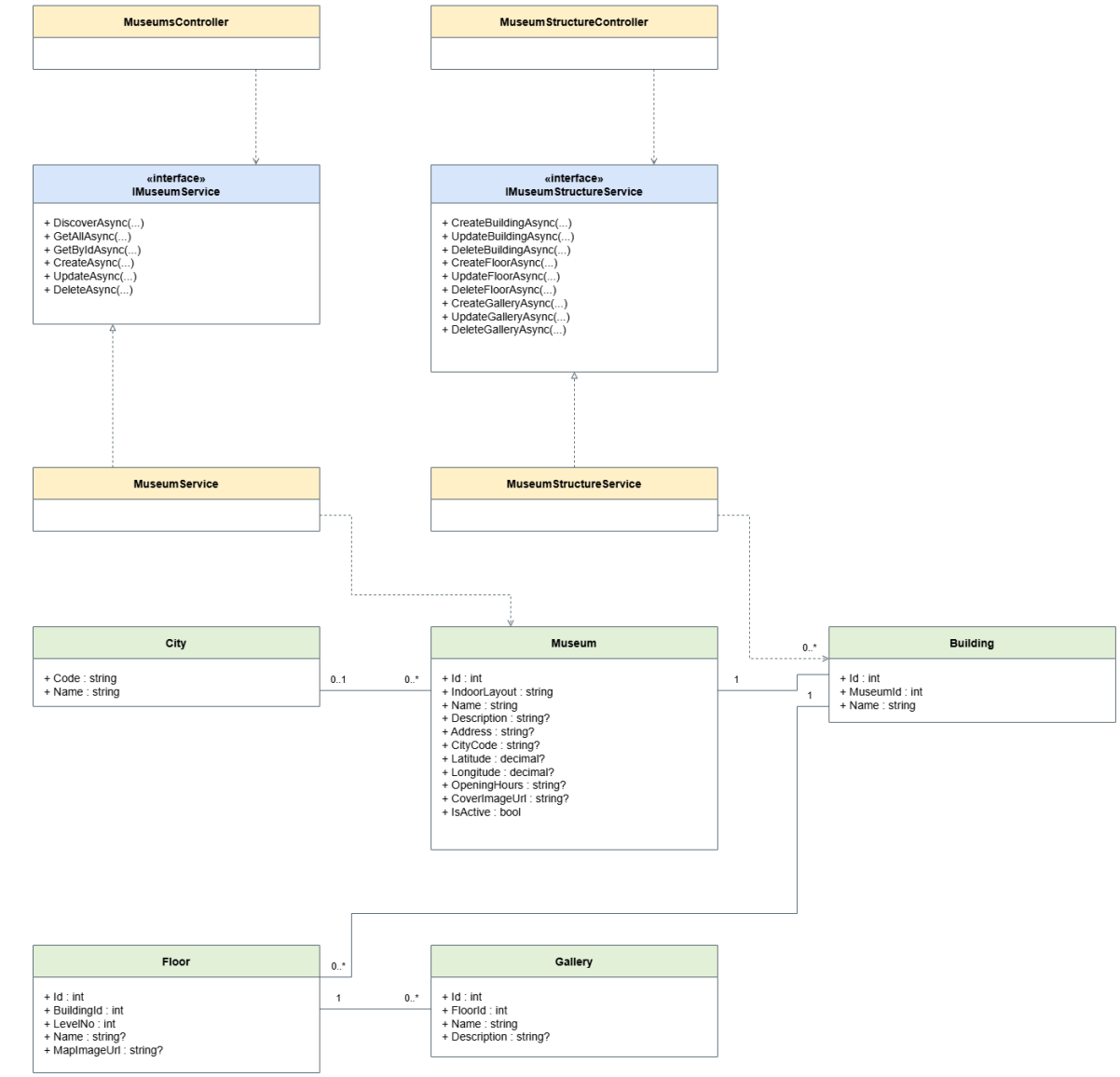
3.1.4 Sequence Diagrams-Staff Assignment Management



3.2 Museum Management & Discovery

3.2.1 Class Diagram

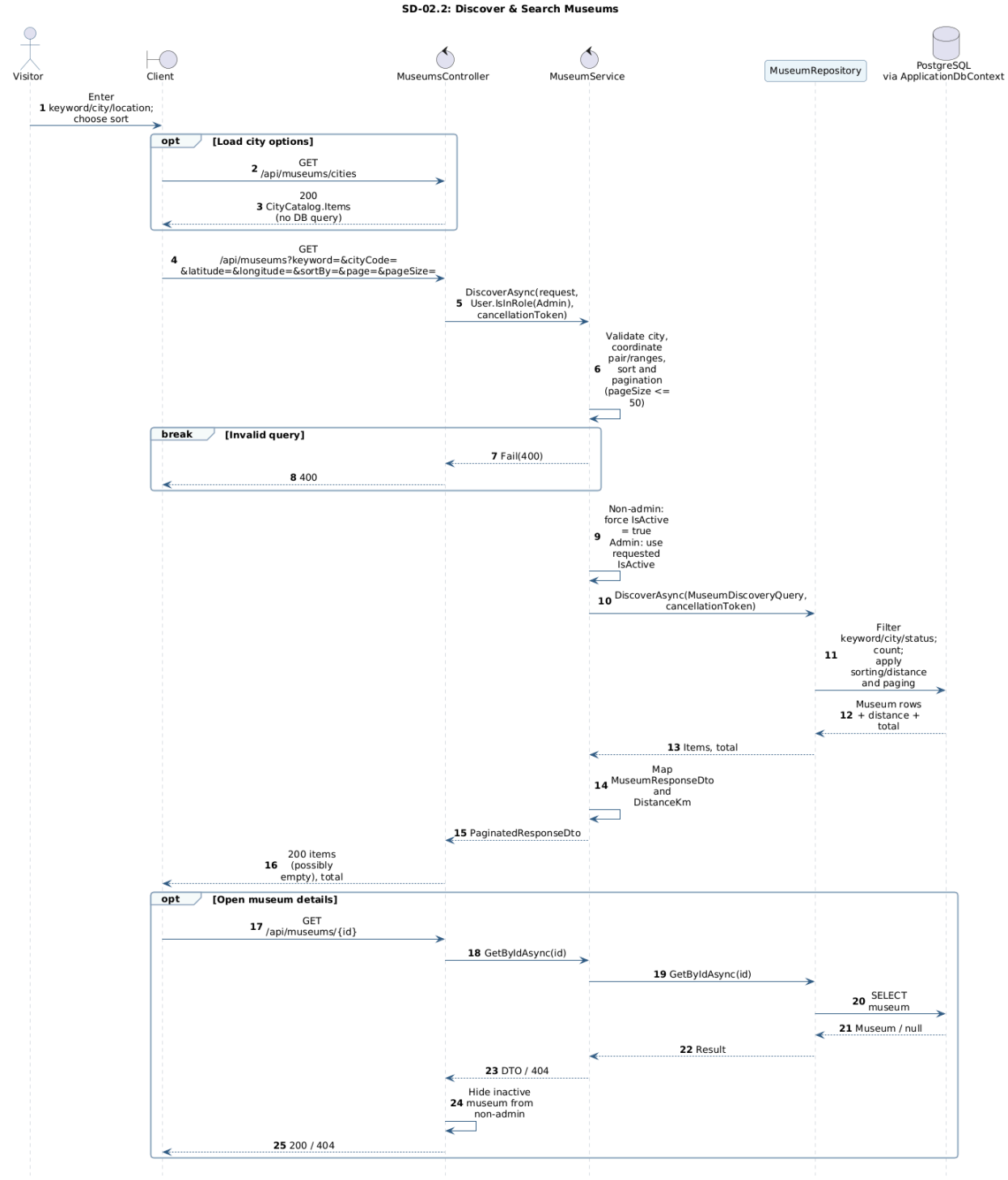
CD-02 Class Diagram — Museum Management & Discovery



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

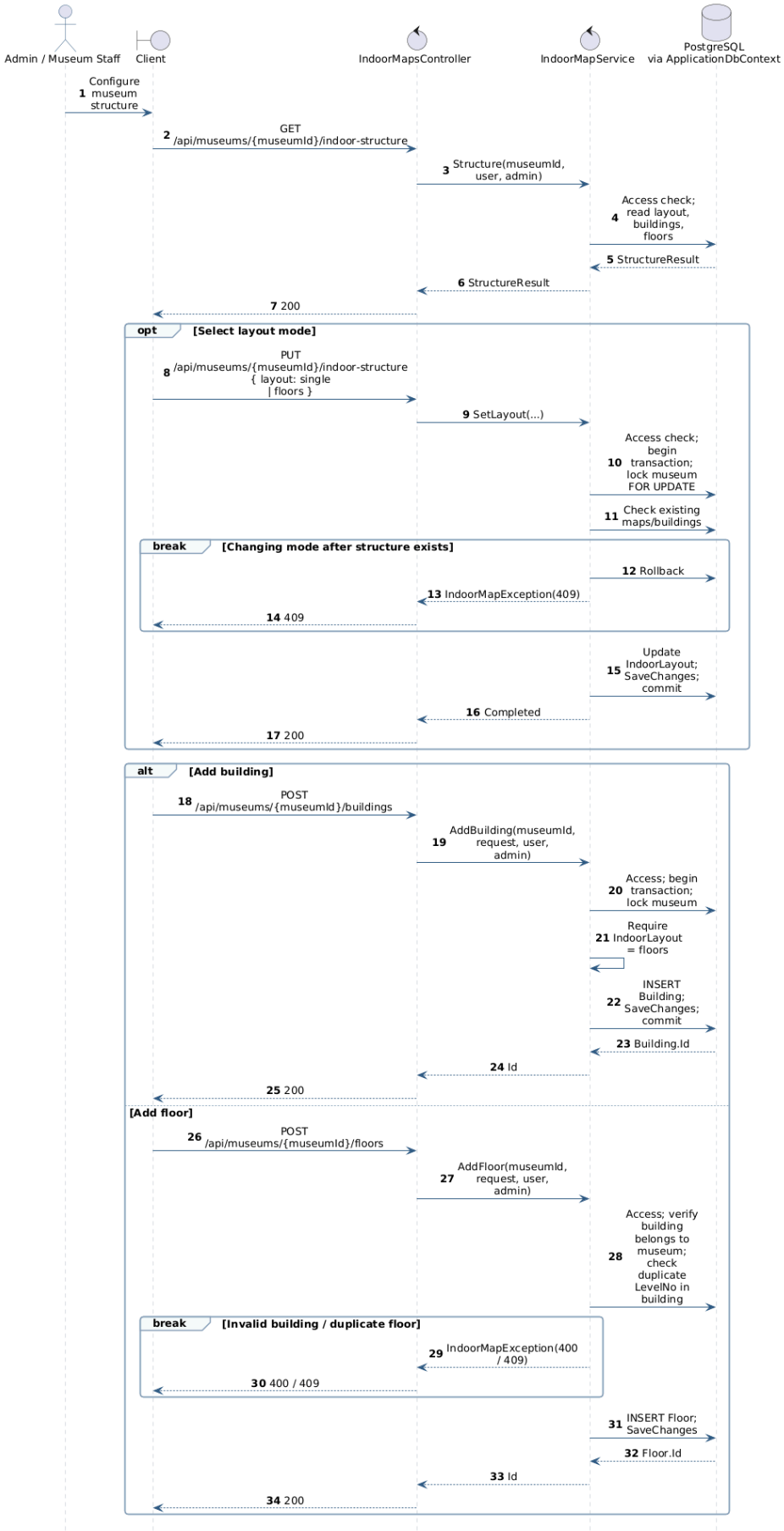
3.2.2 Sequence Diagrams-Create / Update / Delete Museum

3.2.3 Sequence Diagrams-Discover & Search Museums



3.2.4 Sequence Diagrams-Manage Building / Floor / Gallery

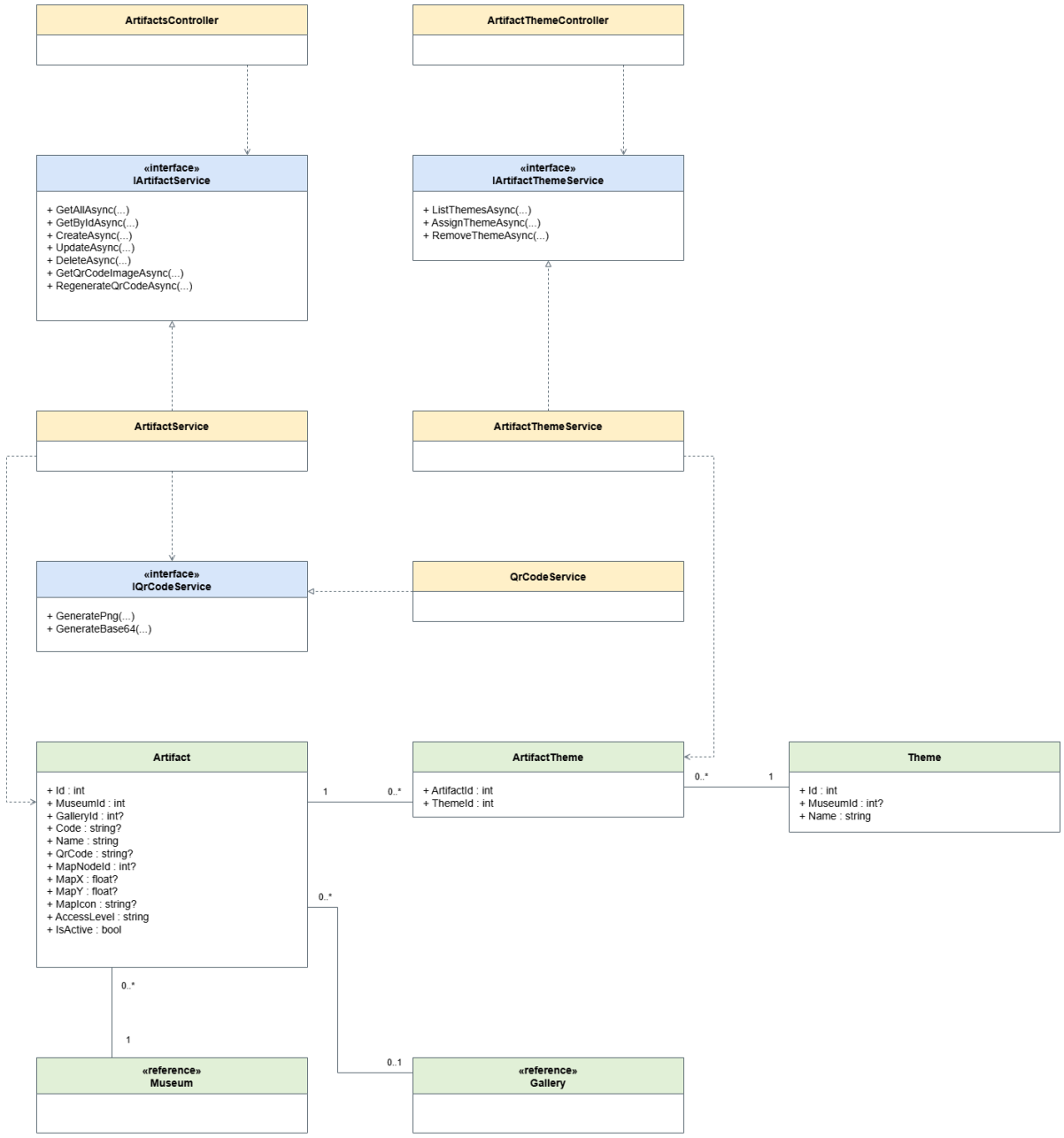
SD-02.3: Manage Building / Floor / Gallery



3.3 Artifact Management & Discovery

3.3.1 Class Diagram

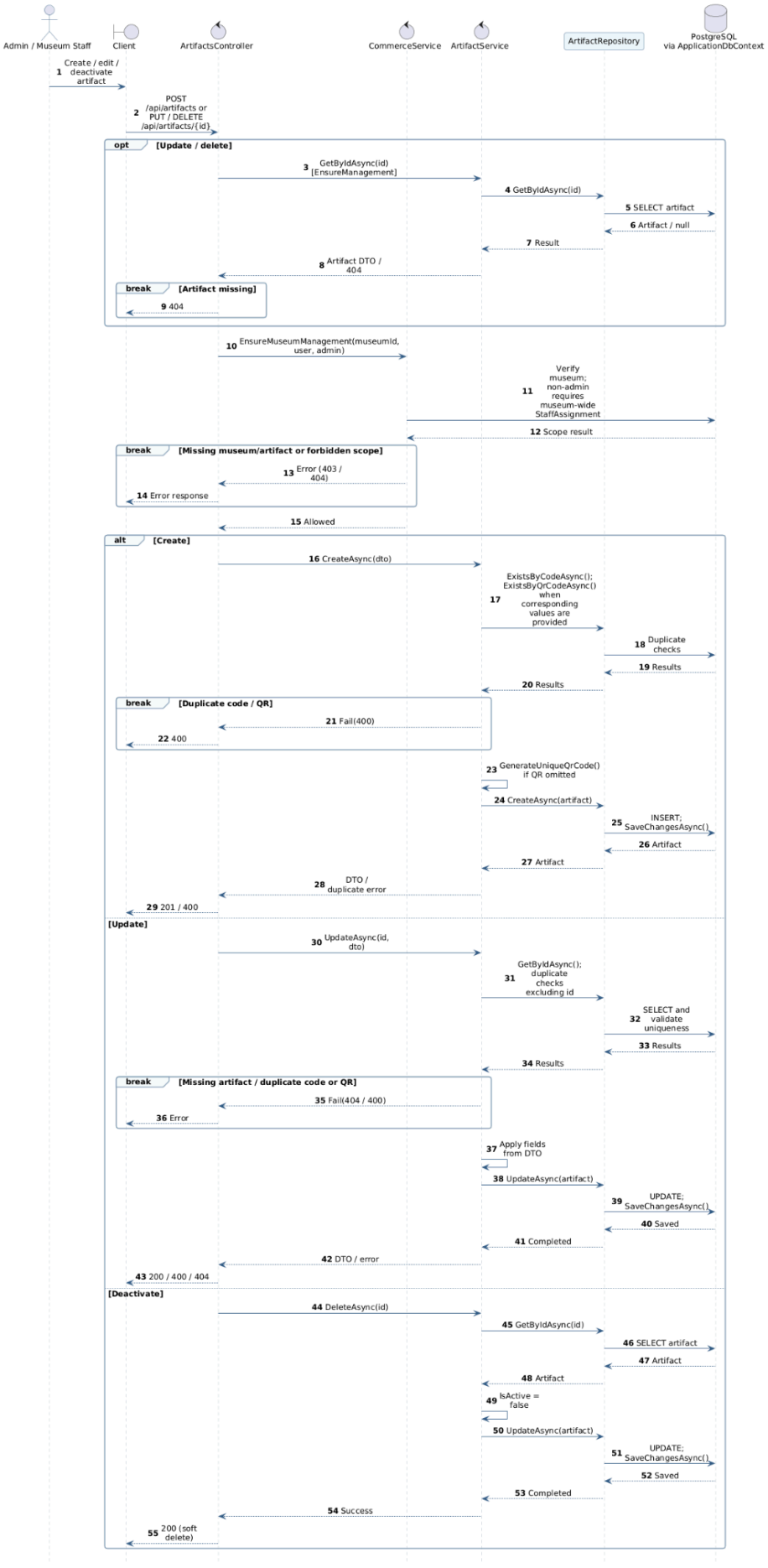
CD-03 Class Diagram — Artifact Management & Discovery



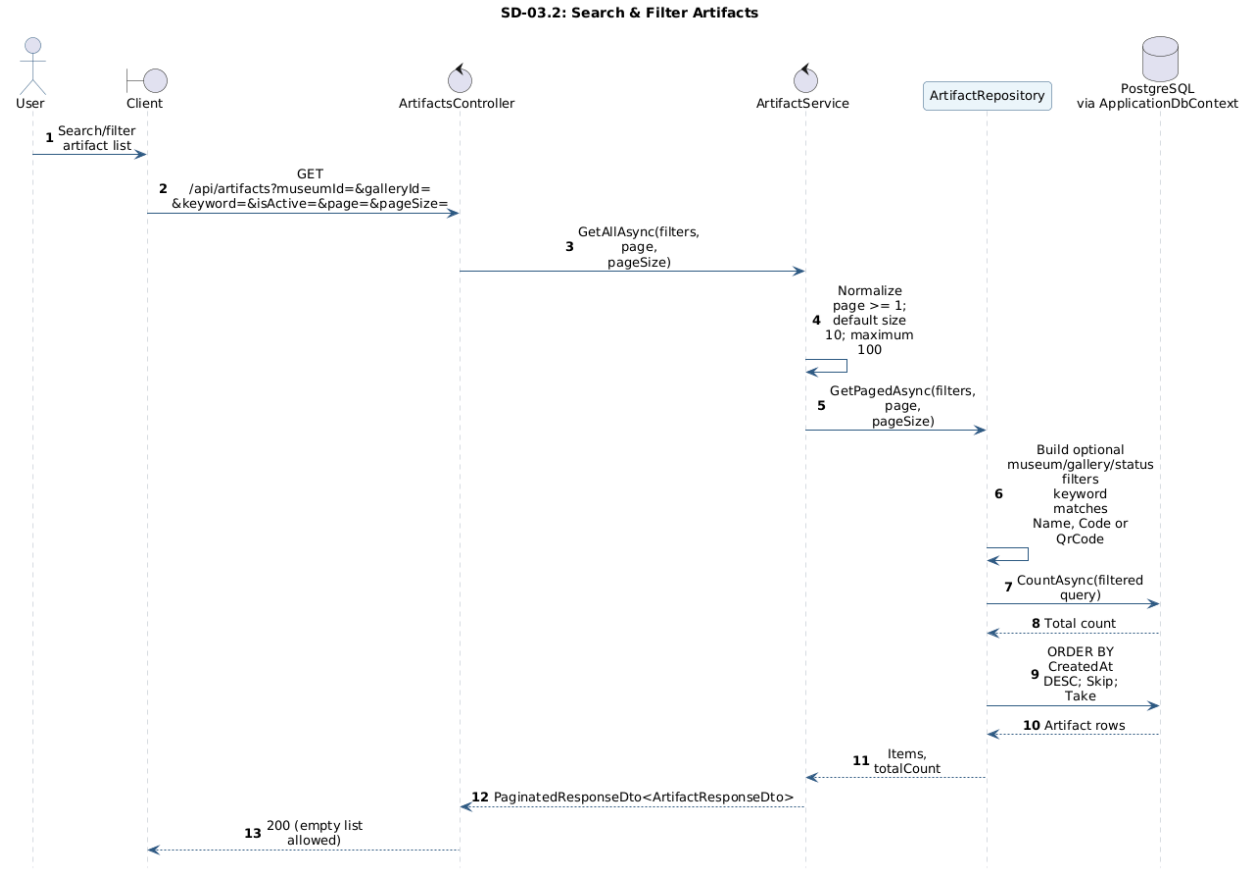
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.3.2 Sequence Diagrams-Manage Artifact Information

SD-03.1: Manage Artifact Information

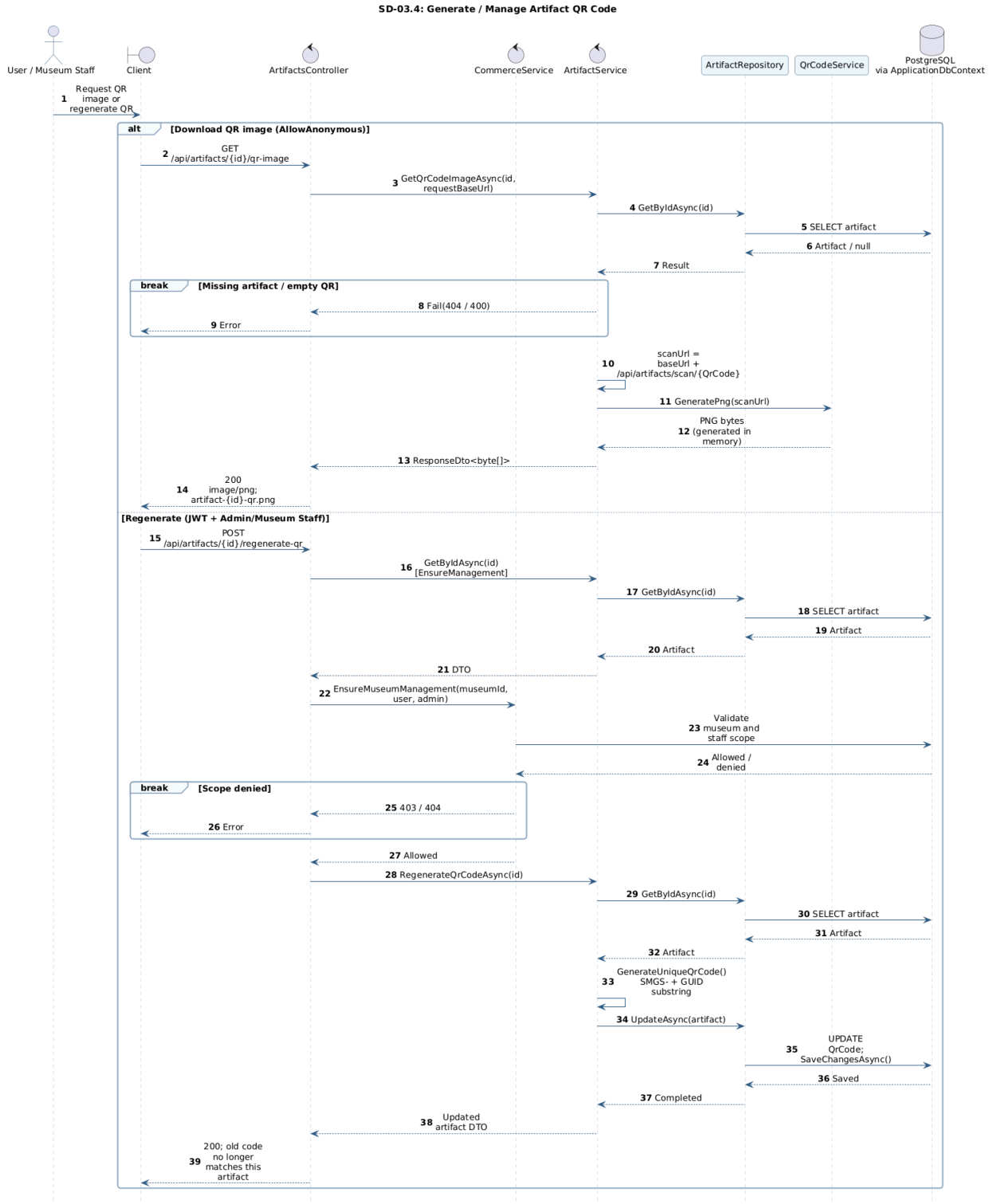


3.3.3 Sequence Diagrams-Search & Filter Artifacts



3.3.4 Sequence Diagrams-Manage Artifact Themes

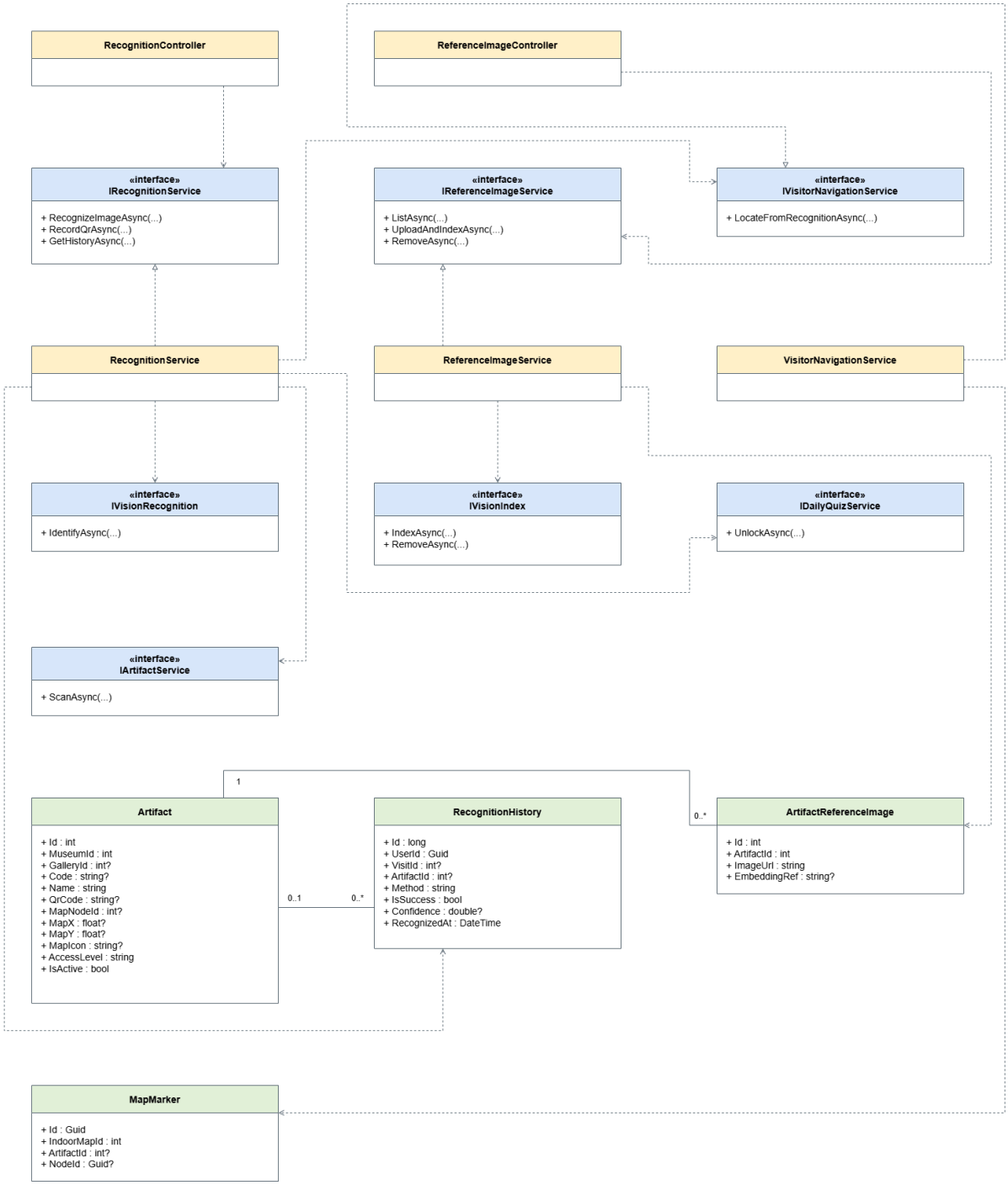
3.3.5 Sequence Diagrams-Generate / Manage Artifact QR Code



3.4 Artifact Recognition & Visitor Location

3.4.1 Class Diagram

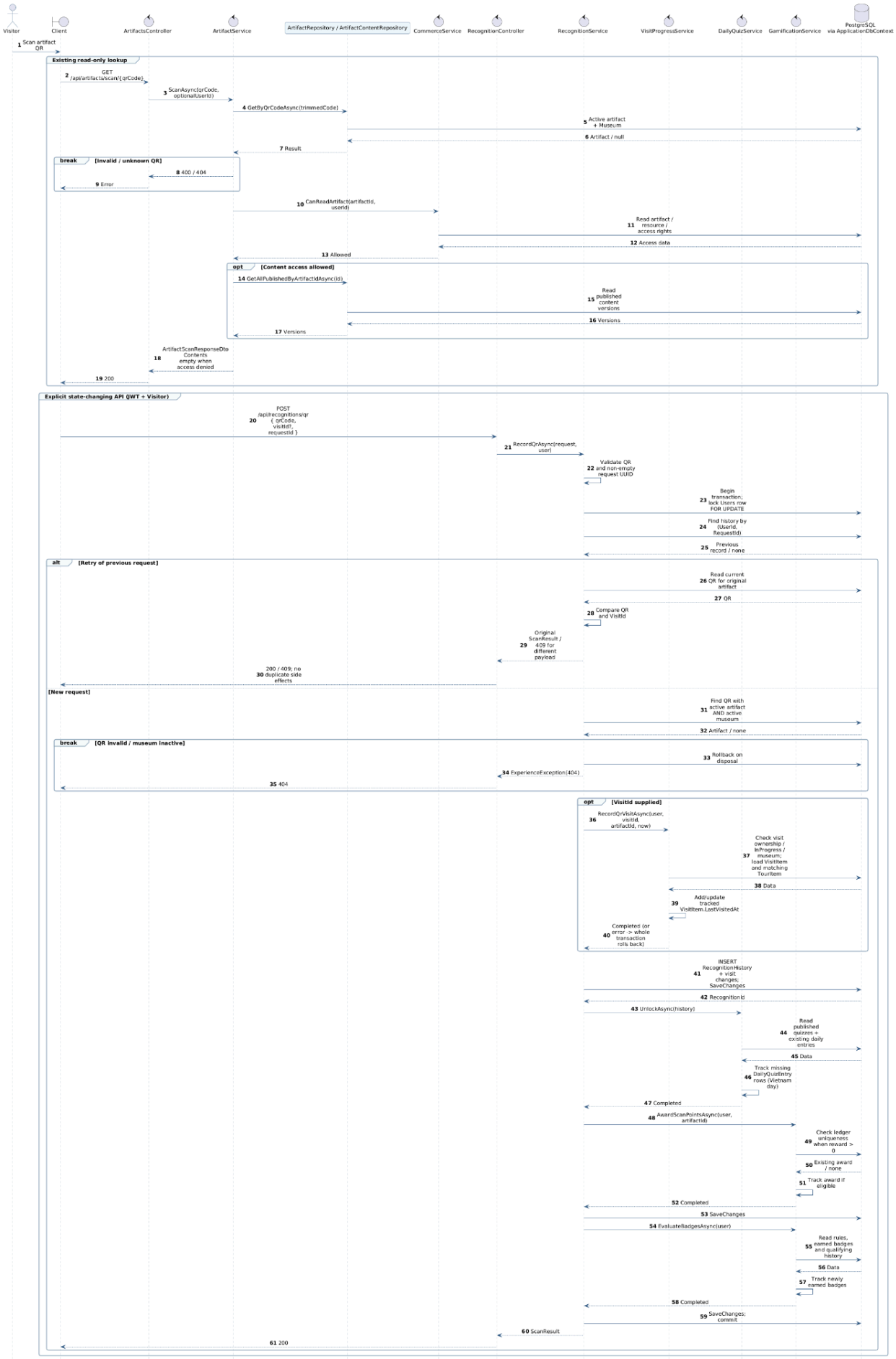
CD-04 Class Diagram — Artifact Recognition & Visitor Location



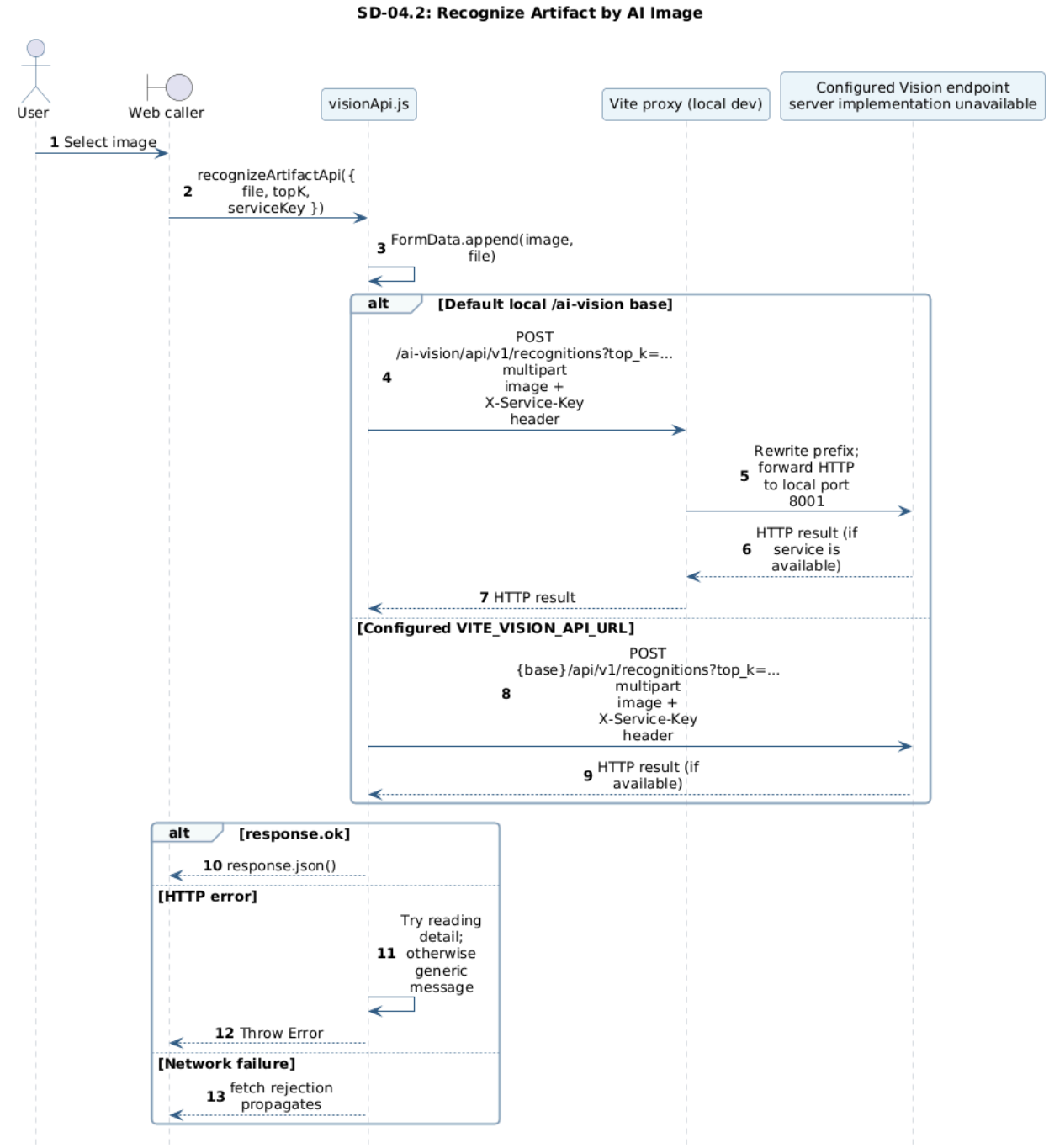
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.4.2 Sequence Diagrams-Recognize Artifact by QR

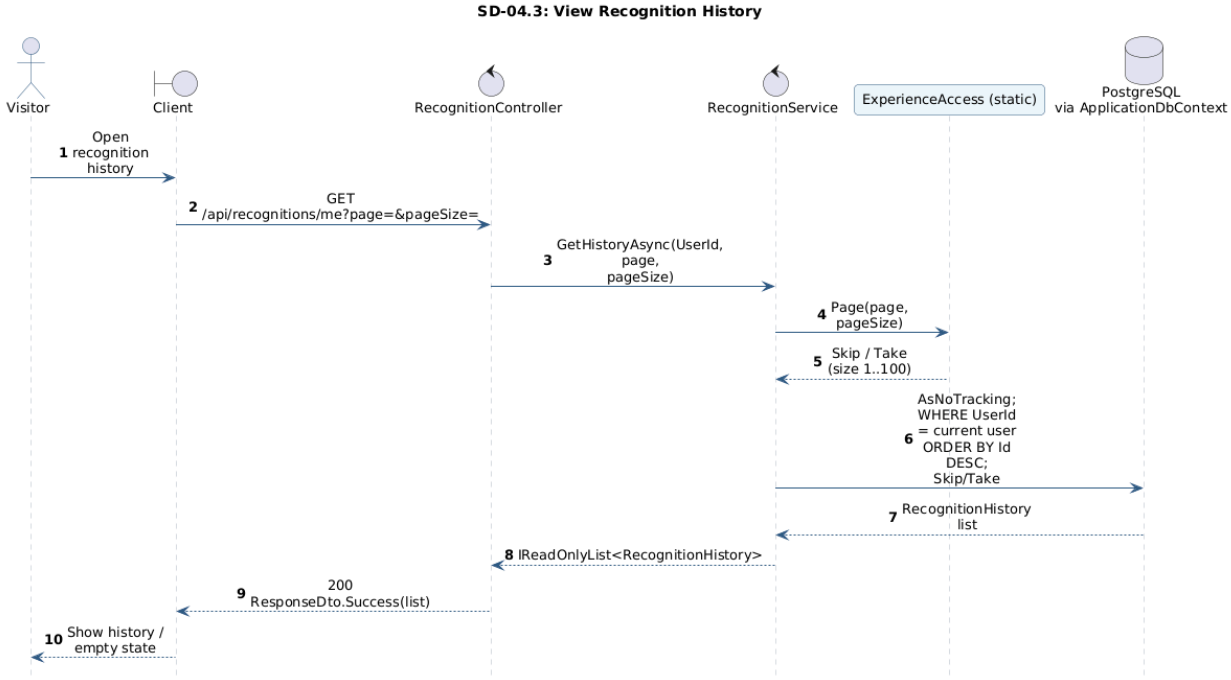
SD-04.1: Recognize Artifact by QR



3.4.3 Sequence Diagrams-Recognize Artifact by AI Image

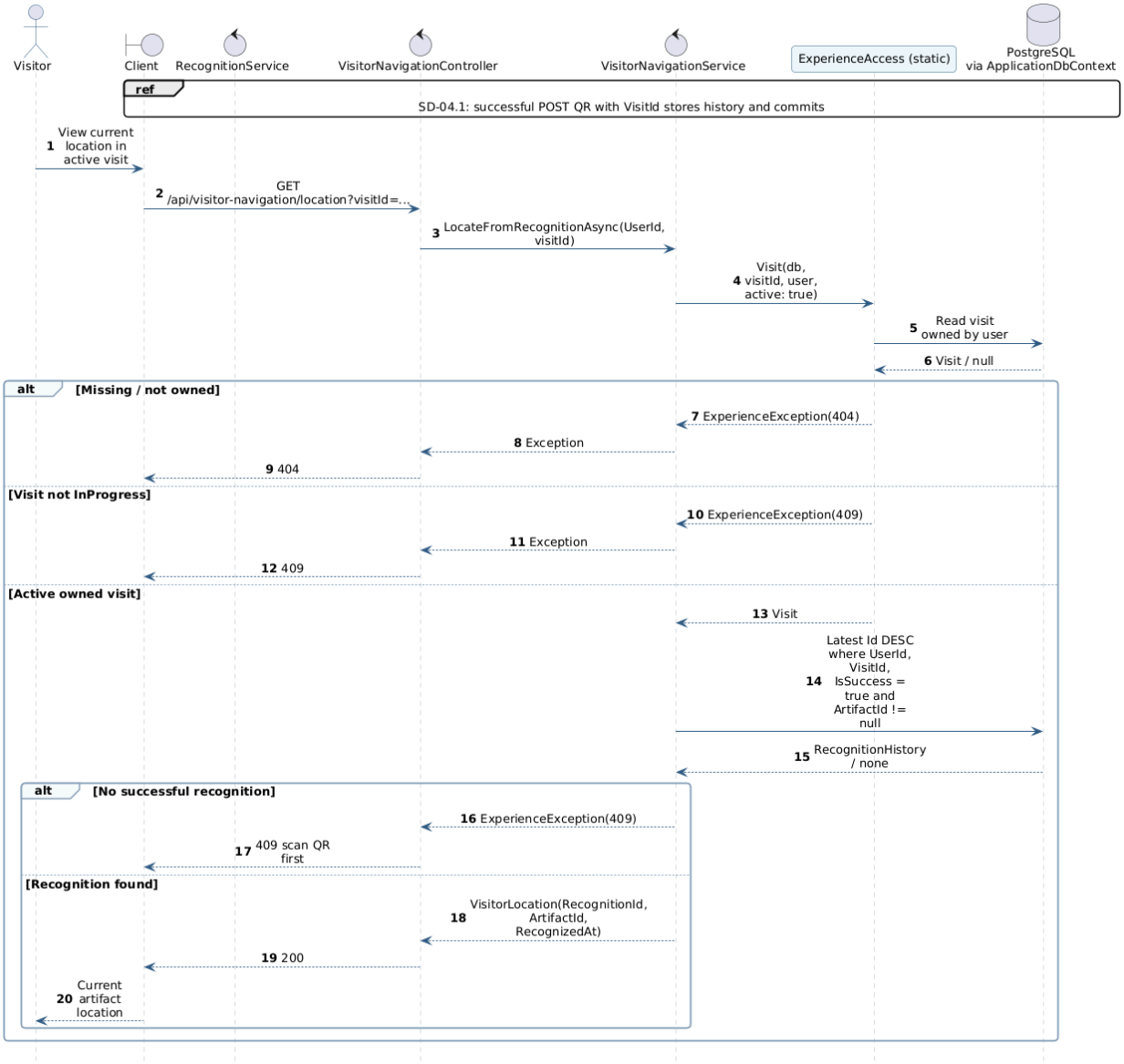


3.4.4 Sequence Diagrams-View Recognition History



3.4.5 Sequence Diagrams-Update Visitor Location After Recognition

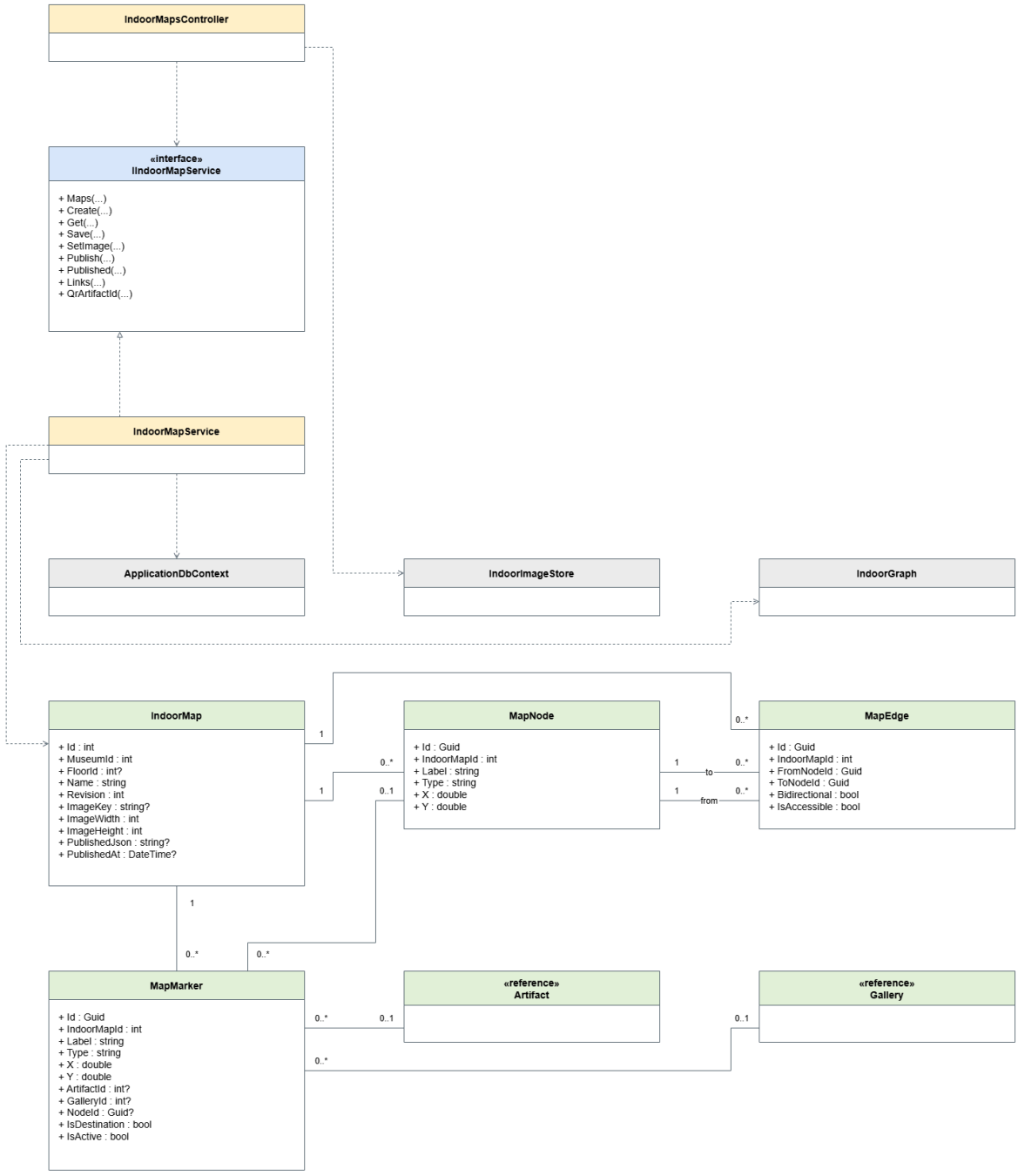
SD-04.4: Update Visitor Location After Recognition



3.5 Indoor & Outdoor Navigation

3.5.1 Class Diagram

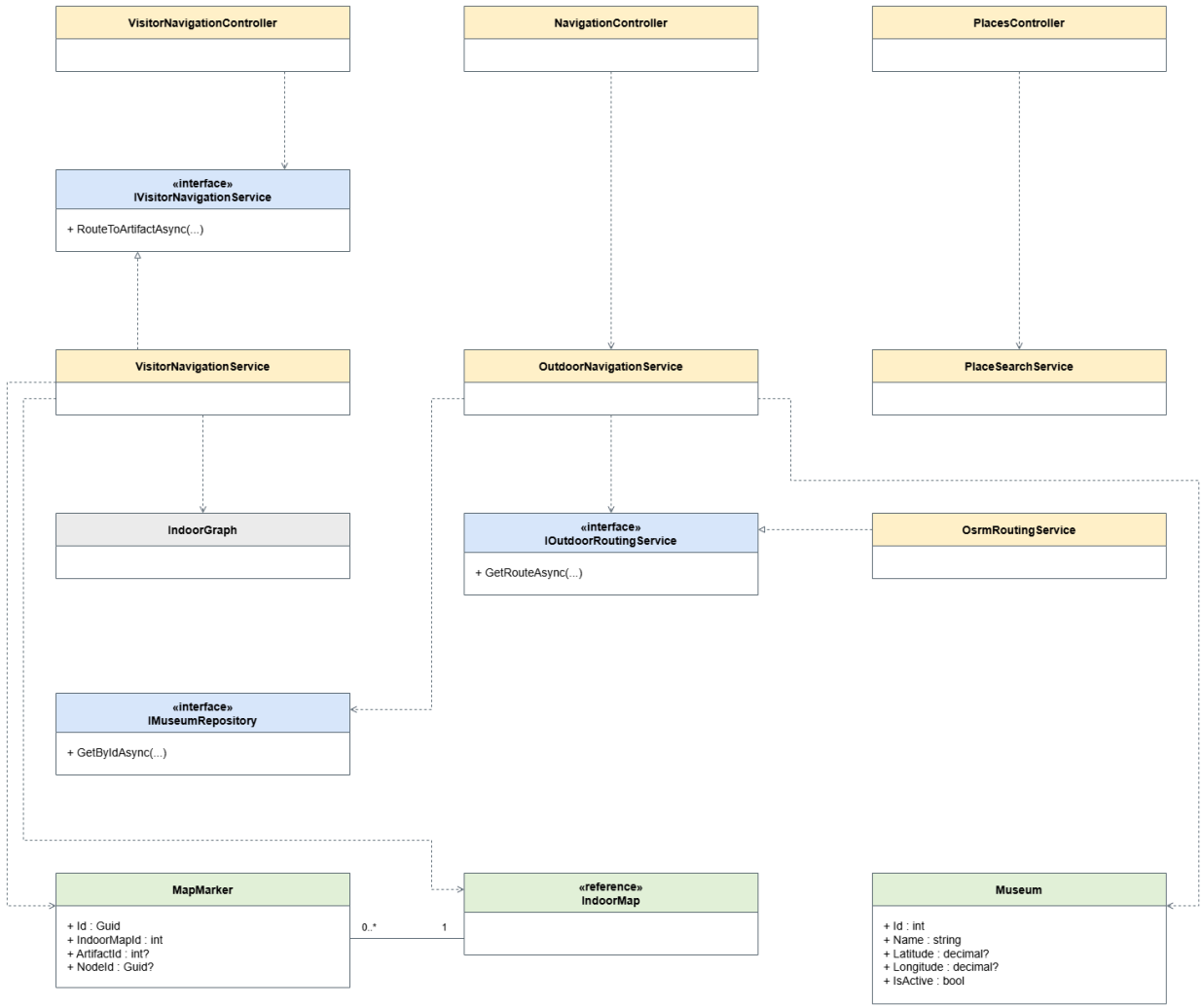
CD-05A Class Diagram — Indoor Map Management



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

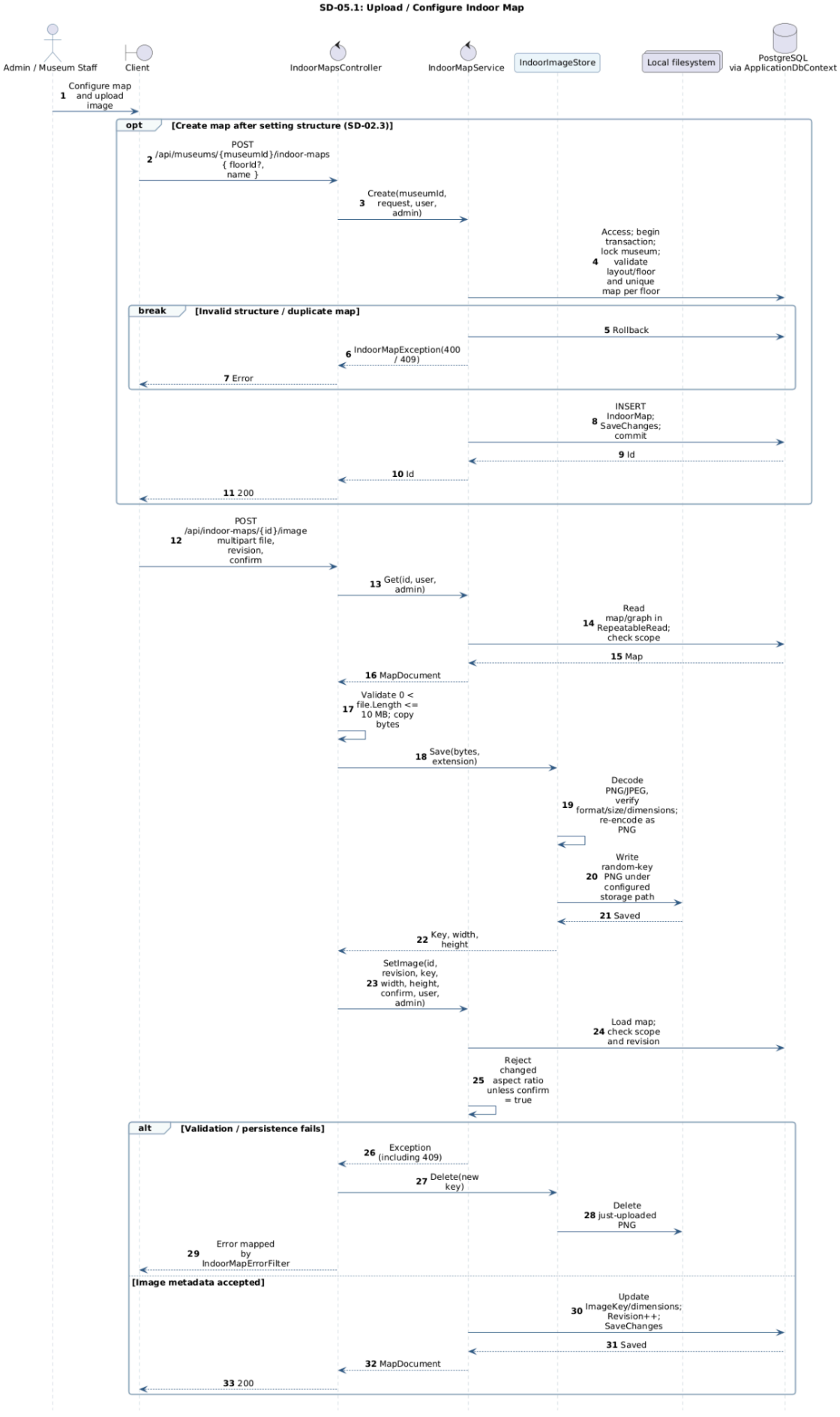
3.5.2 Class Diagram

CD-05B Class Diagram — Indoor & Outdoor Navigation



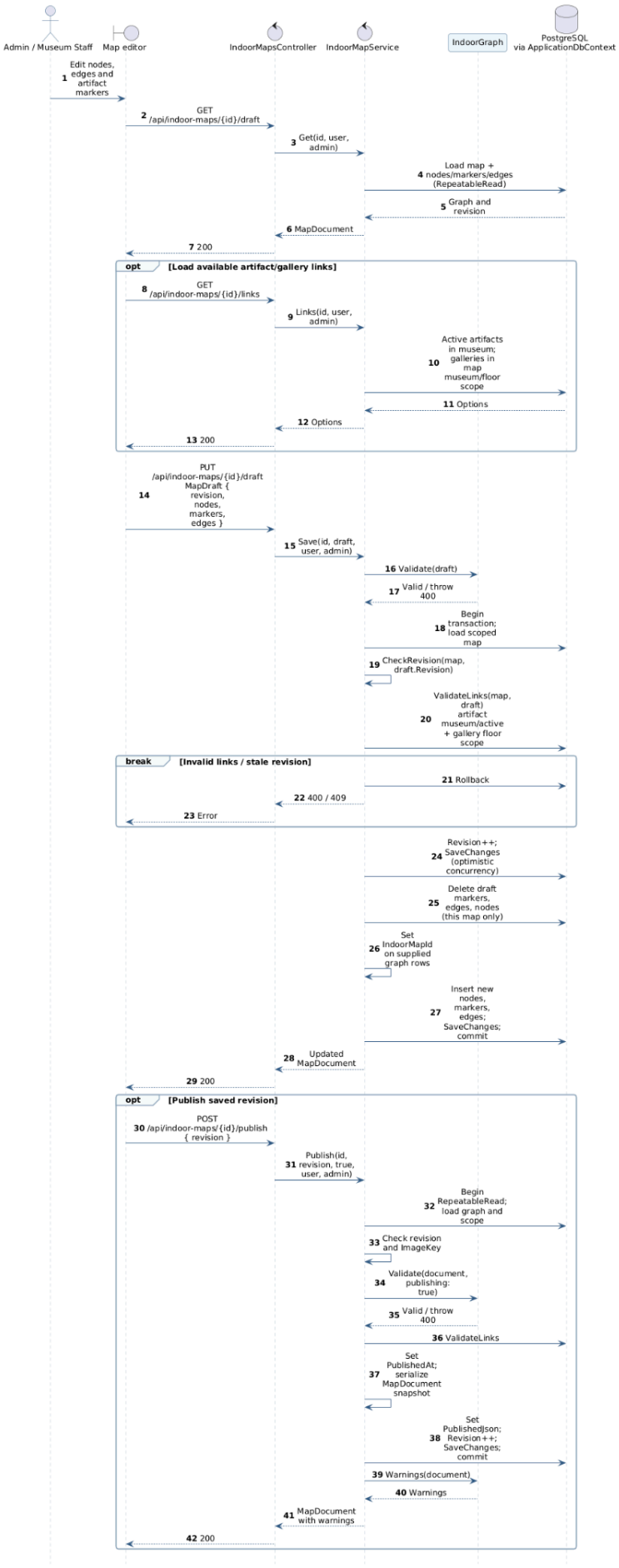
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.5.3 Sequence Diagrams-Upload / Configure Indoor Map


SD-05.1: Upload / Configure Indoor Map
Admin / Museum Staff
Client
IndoorMapsController
IndoorMapService
IndoorImageStore
Local filesystem
PostgreSQL via ApplicationDbContext
1 Configure map and upload image
opt [Create map after setting structure (SD-02.3)]
2 POST /api/museums/{museumId}/indoor-maps { floorId?, name }
3 Create(museumId, request, user, admin)
4 Access; begin transaction; lock museum; validate layout/floor and unique map per floor
break [Invalid structure / duplicate map]
5 Rollback
6 IndoorMapException(400 / 409)
7 Error
8 INSERT IndoorMap; SaveChanges; commit
9 Id
10 Id
11 200
12 POST /api/indoor-maps/{id}/image multipart file, revision, confirm
13 Get(id, user, admin)
14 Read map/graph in RepeatableRead; check scope
15 Map
16 MapDocument
17 Validate 0 < file.Length <= 10 MB; copy bytes
18 Save(bytes, extension)
19 Decode PNG/JPEG, verify format/size/dimensions; re-encode as PNG
20 Write random-key PNG under configured storage path
21 Saved
22 Key, width, height
23 SetImage(id, revision, key, width, height, confirm, user, admin)
24 Load map; check scope and revision
25 Reject changed aspect ratio unless confirm = true
alt [Validation / persistence fails]
26 Exception (including 409)
27 Delete(new key)
28 Delete just-uploaded PNG
29 Error mapped by IndoorMapErrorFilter
[Image metadata accepted]
30 Update ImageKey/dimensions; Revision++; SaveChanges
31 Saved
32 MapDocument
33 200

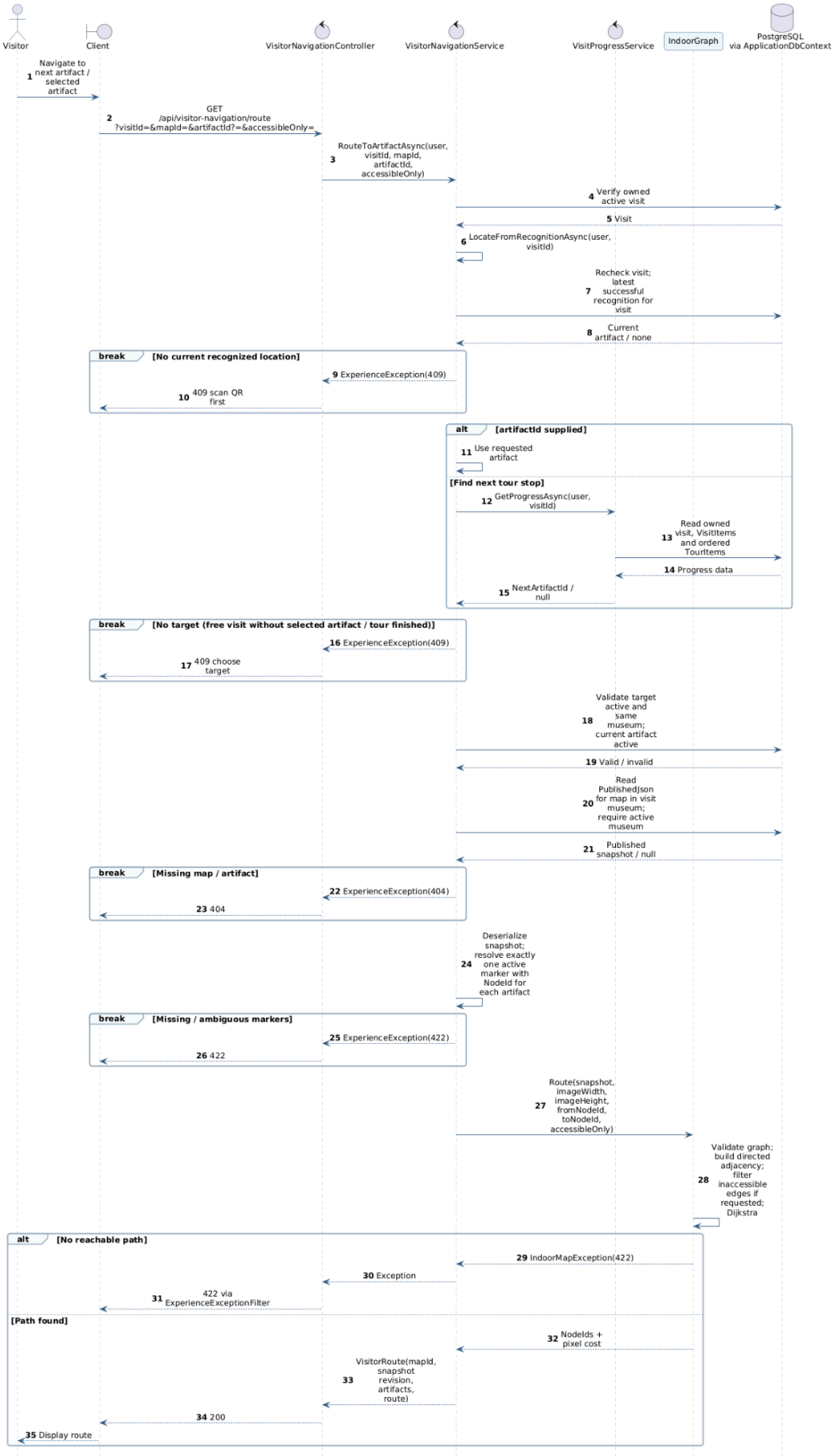
3.5.4 Sequence Diagrams-Manage Map Nodes, Edges & Artifact Positions

SD-05.2: Manage Map Nodes, Edges & Artifact Positions

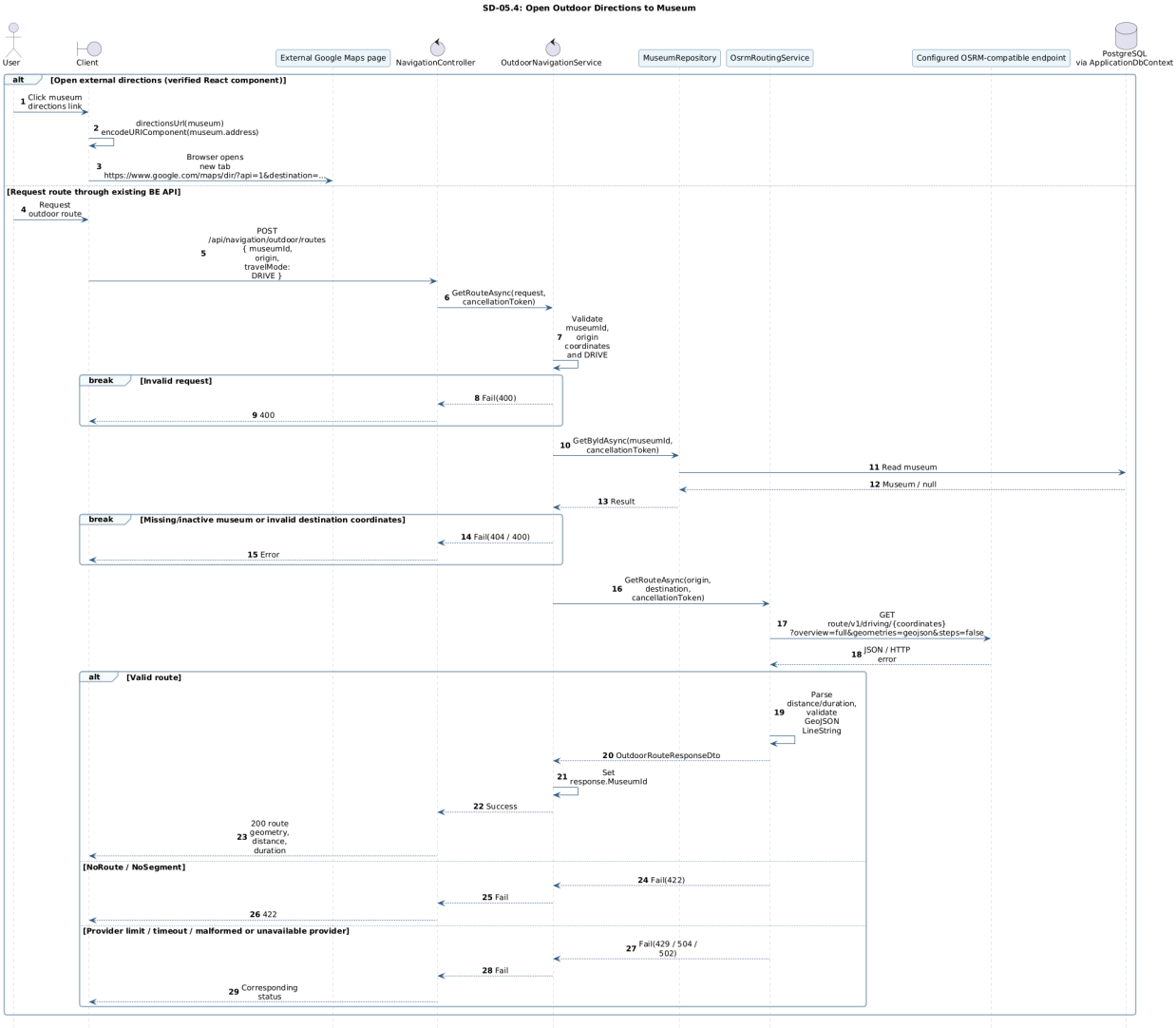


3.5.5 Sequence Diagrams-Find Indoor Route to Next Artifact

SD-05.3: Find Indoor Route to Next Artifact



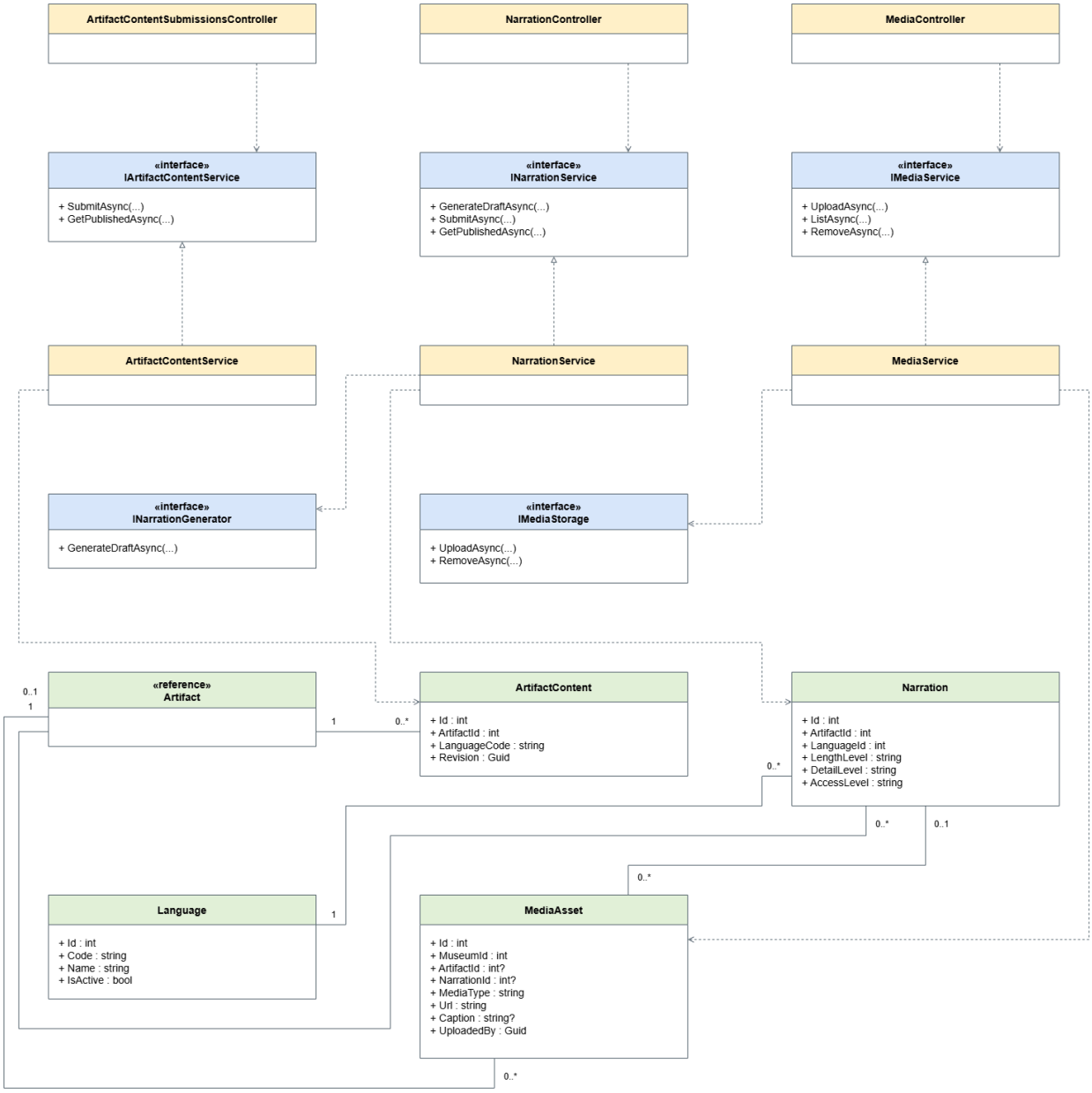
3.5.6 Sequence Diagrams-Open Outdoor Directions to Museum



3.6 Artifact Content & Media Management

3.6.1 Class Diagram

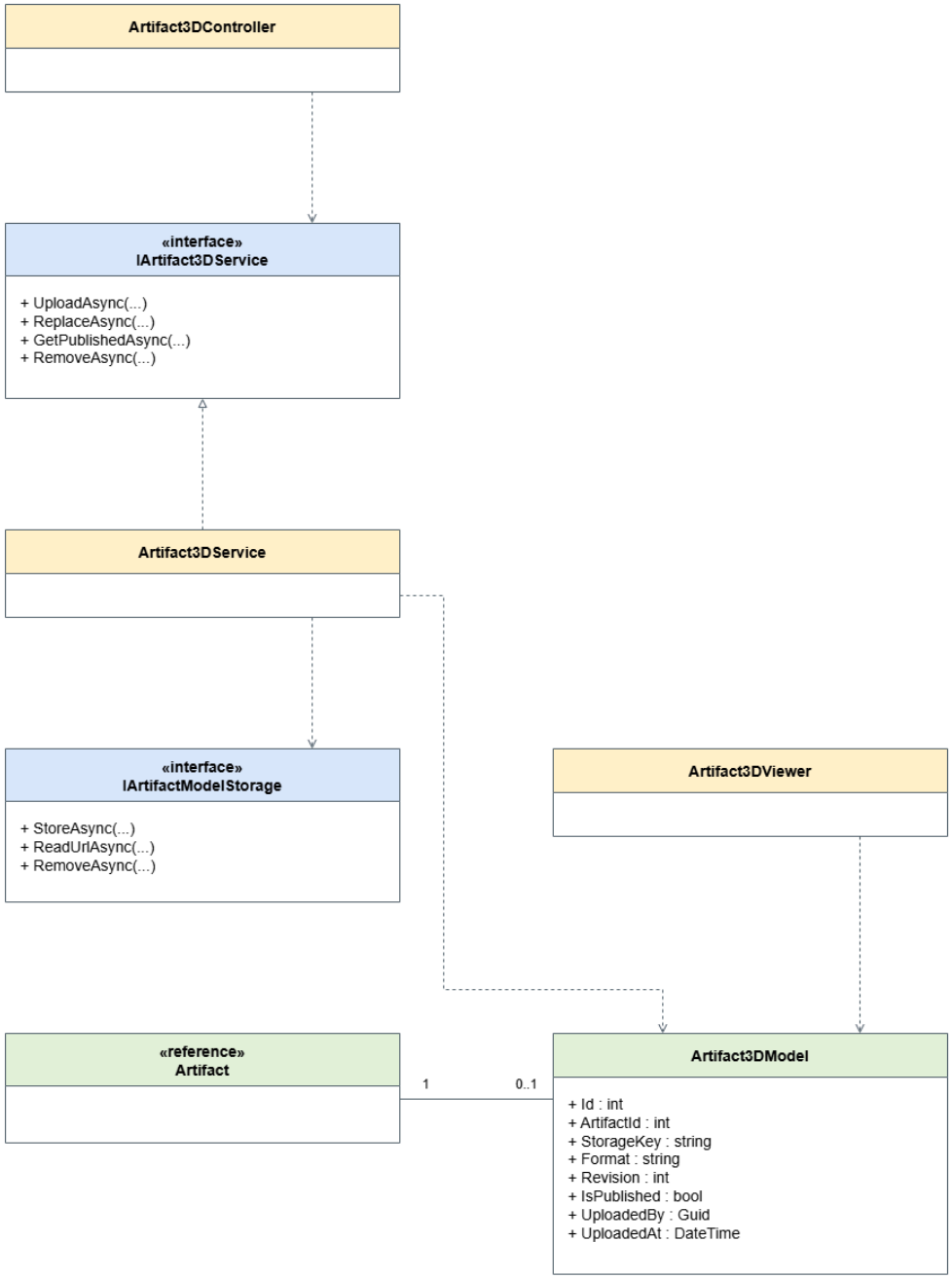
CD-06A Class Diagram — Artifact Content & Media Management



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

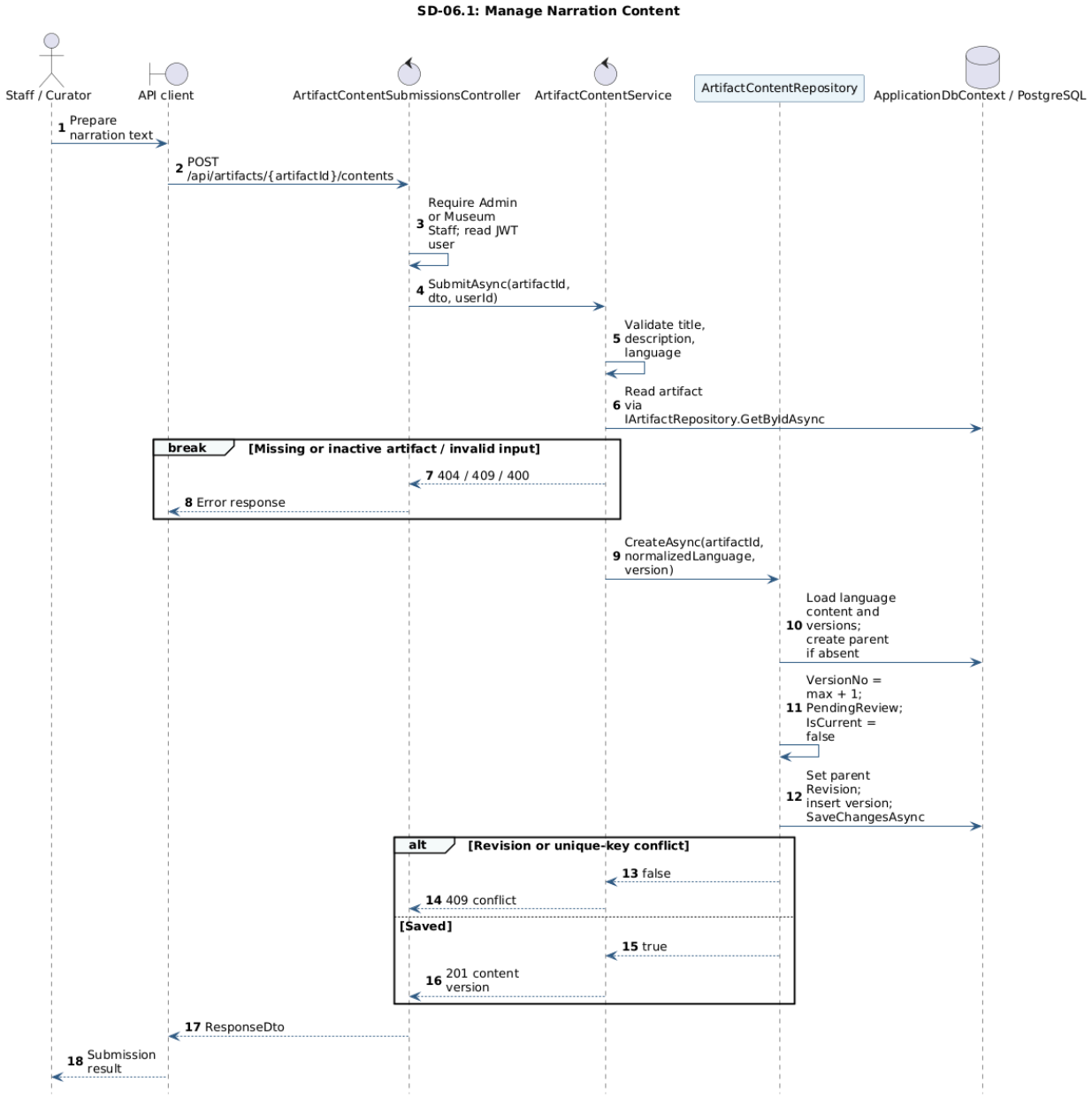
3.6.2 Class Diagram

CD-06B Class Diagram — Artifact 3D Model Management & Viewing

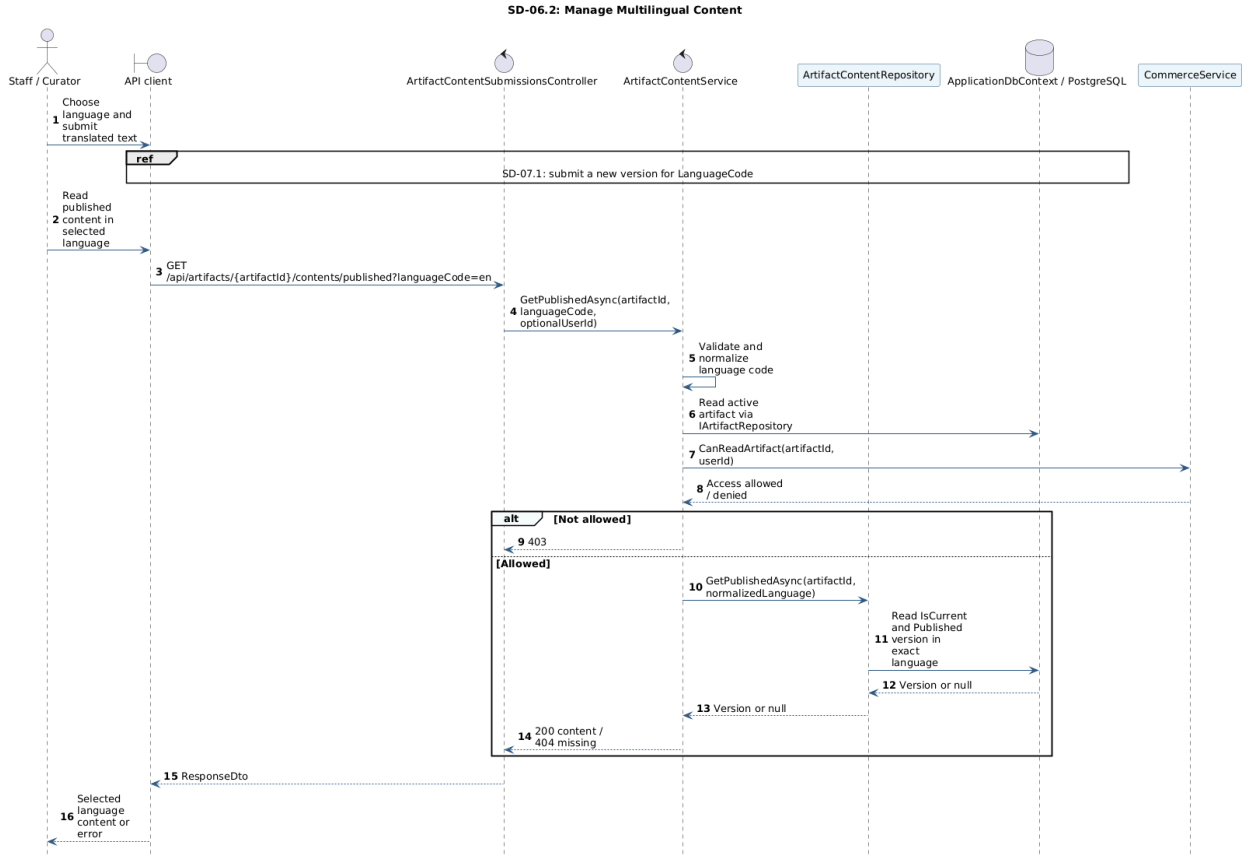


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

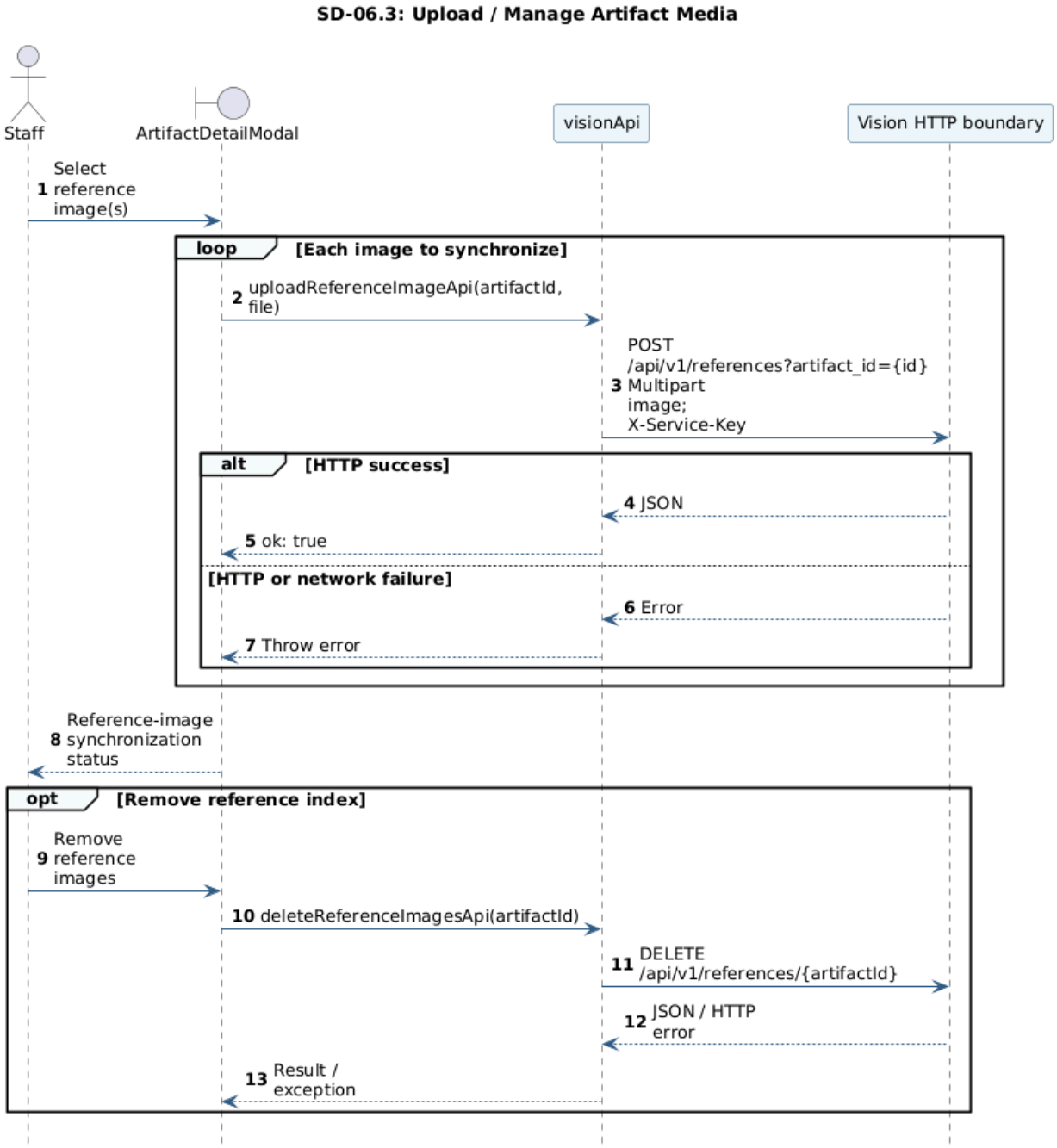
3.6.3 Sequence Diagrams-Manage Narration Content



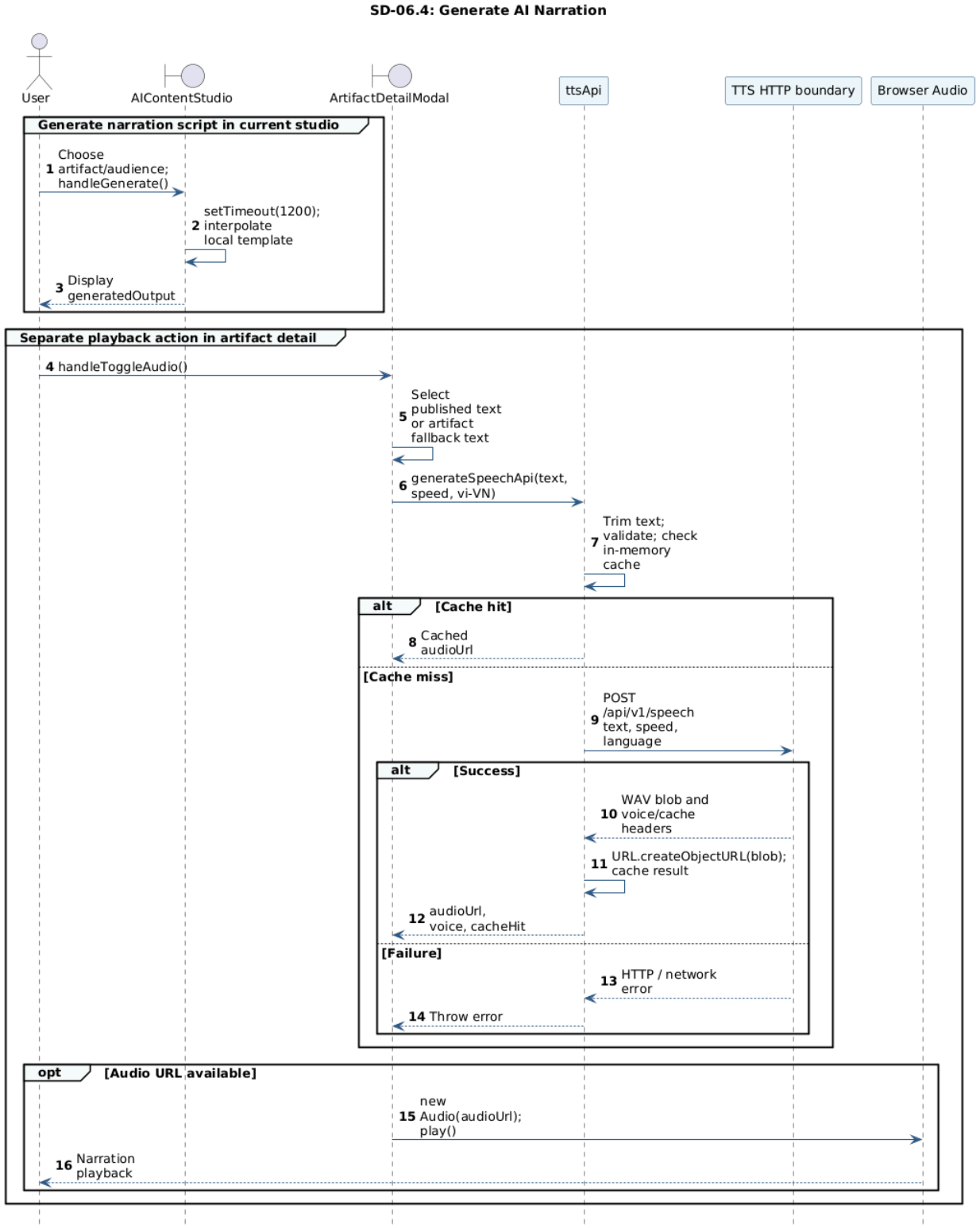
3.6.4 Sequence Diagrams-Manage Multilingual Content



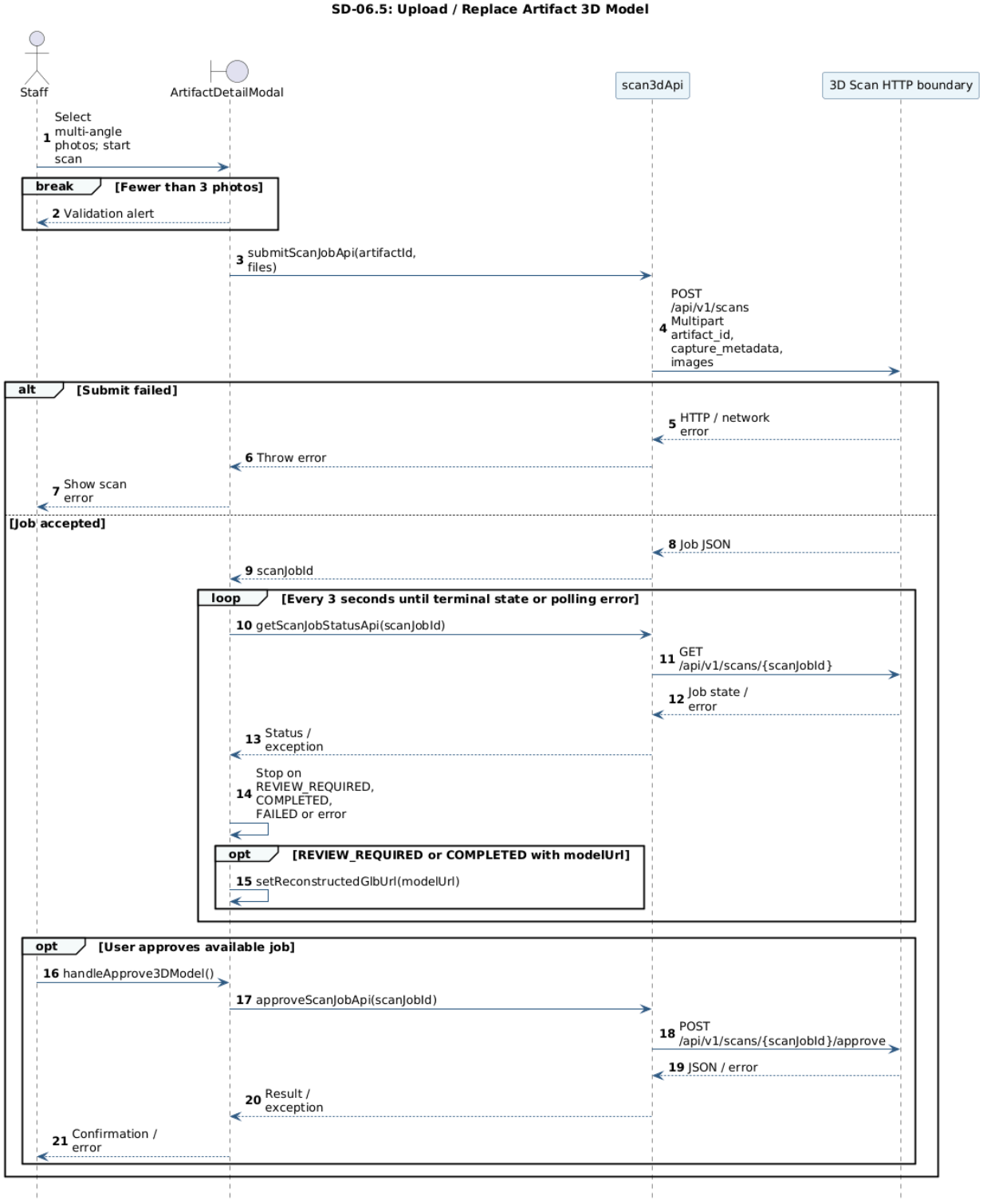
3.6.5 Sequence Diagrams-Upload / Manage Artifact Media



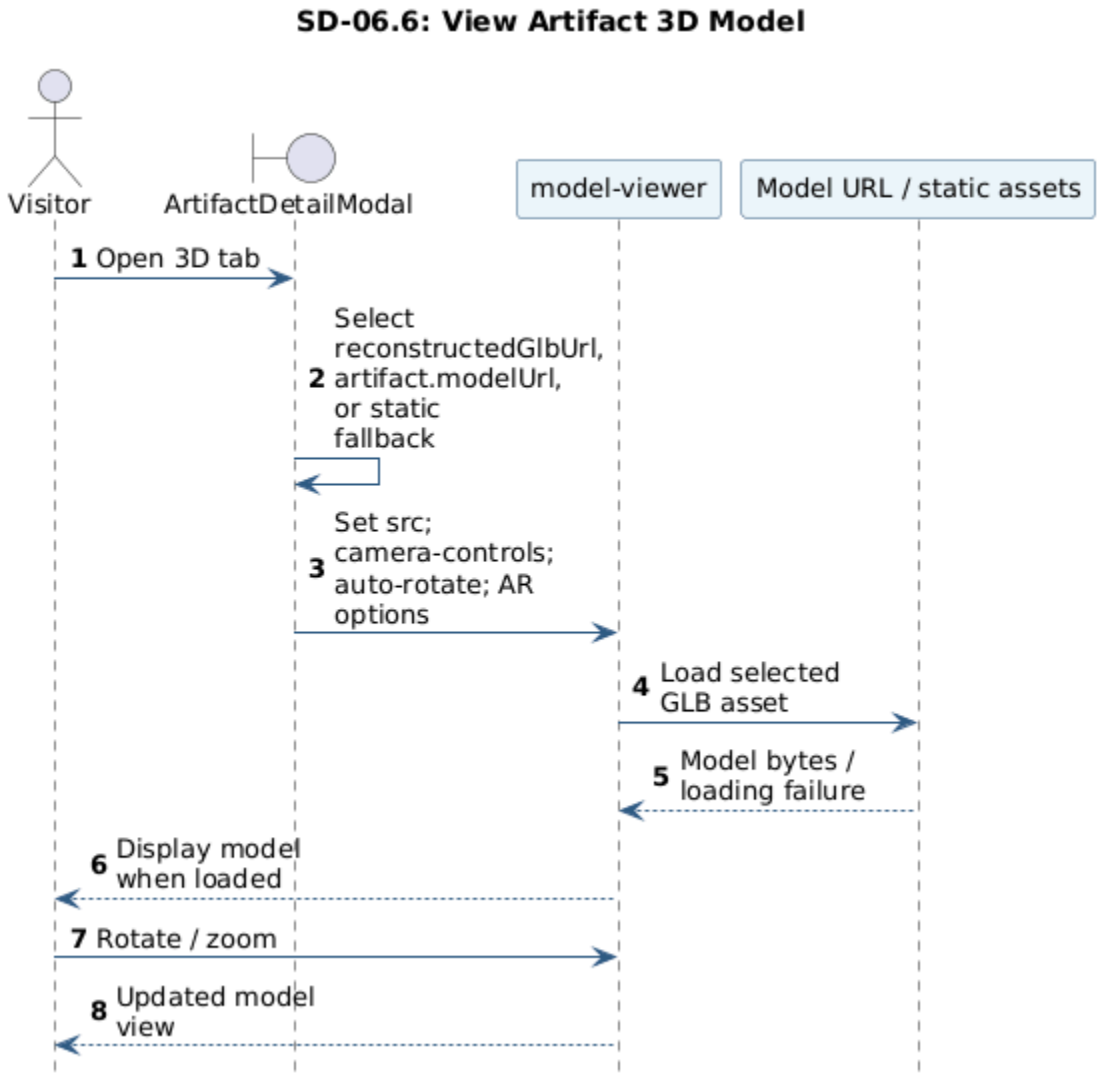
3.6.6 Sequence Diagrams-Generate AI Narration



3.6.7 Sequence Diagrams-Upload / Replace Artifact 3D Model



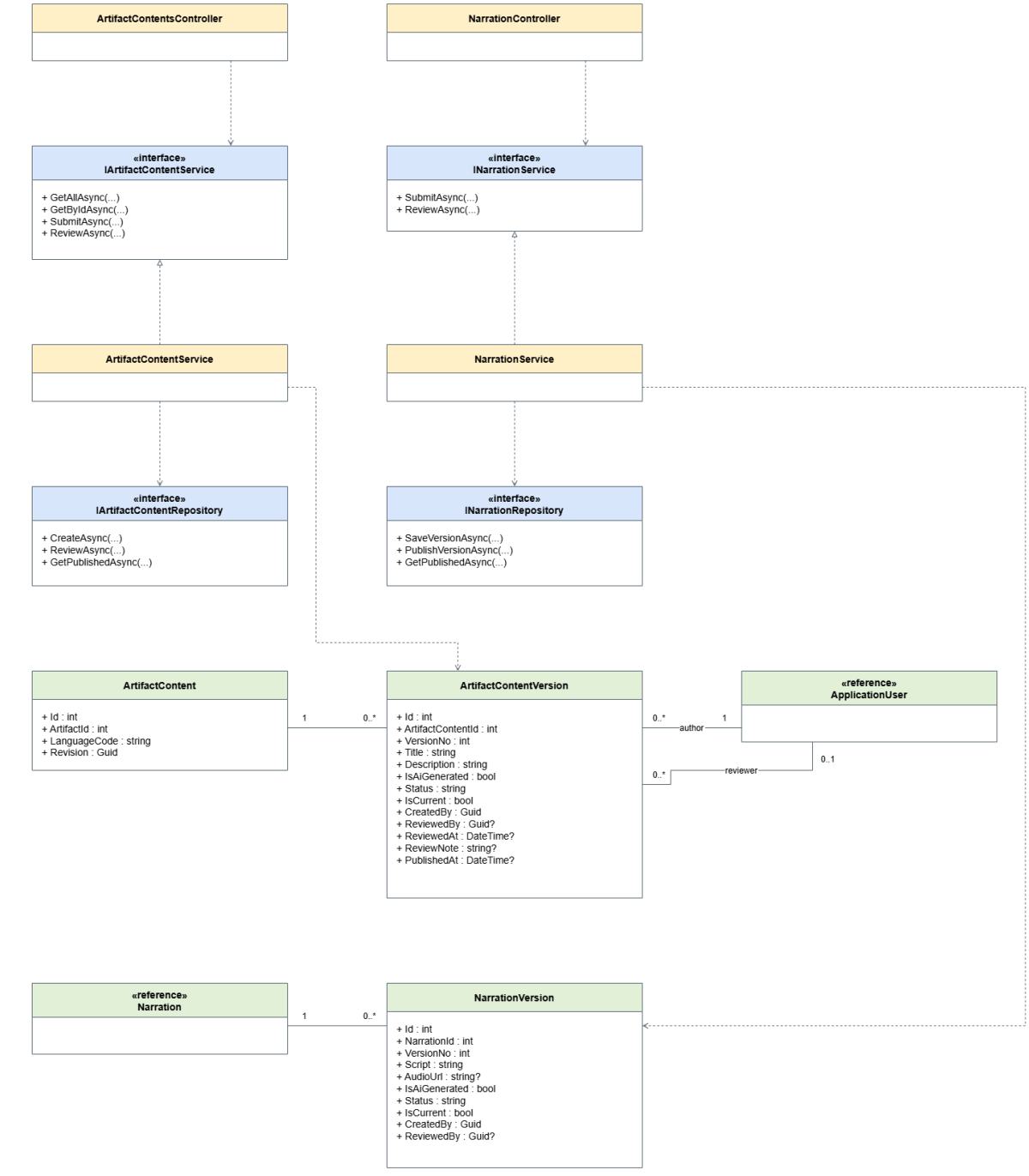
3.6.8 Sequence Diagrams-View Artifact 3D Model



3.7 Content Review & Publication

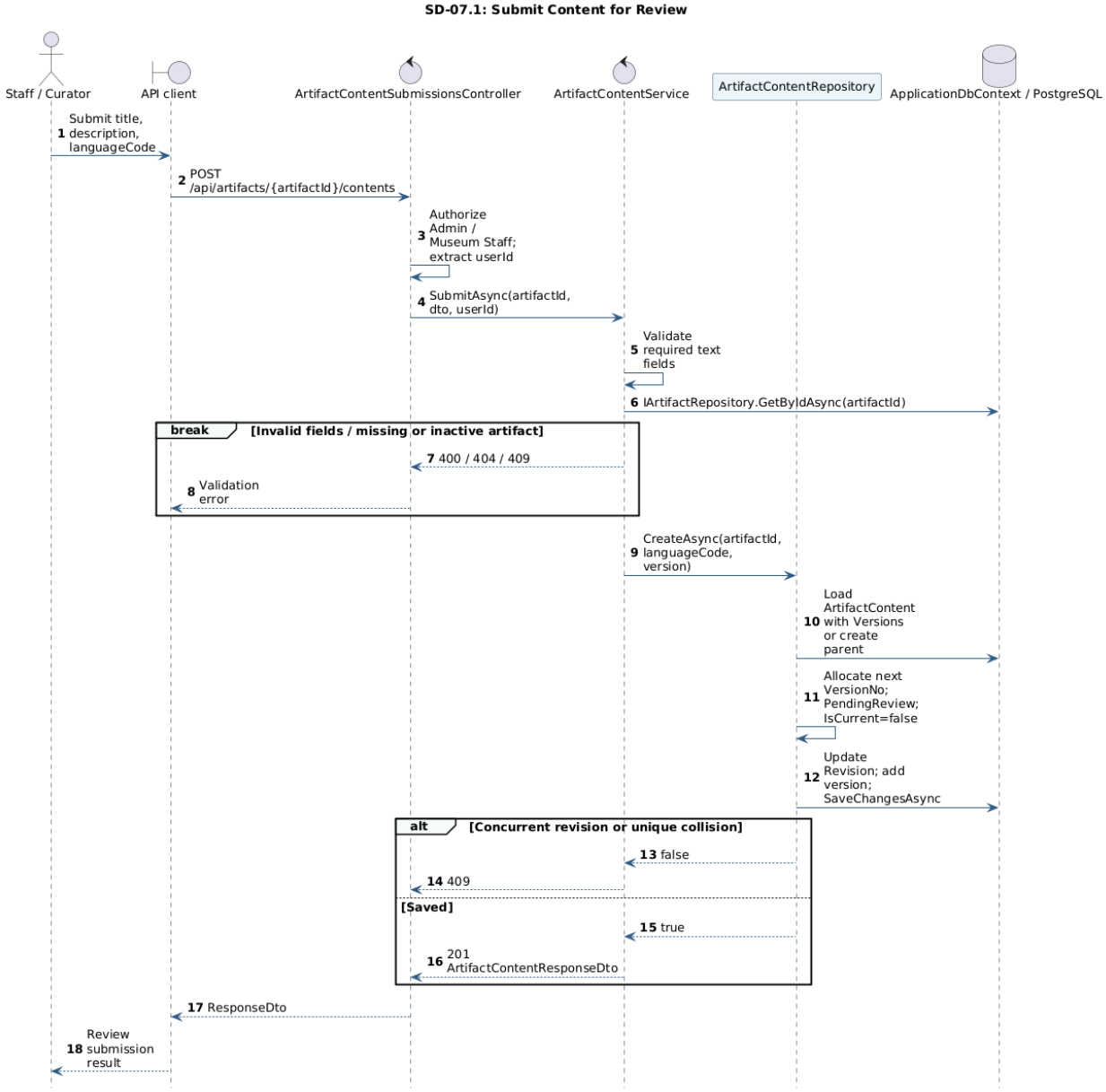
3.7.1 Class Diagram

CD-07 Class Diagram — Content Review, Publication & Versioning



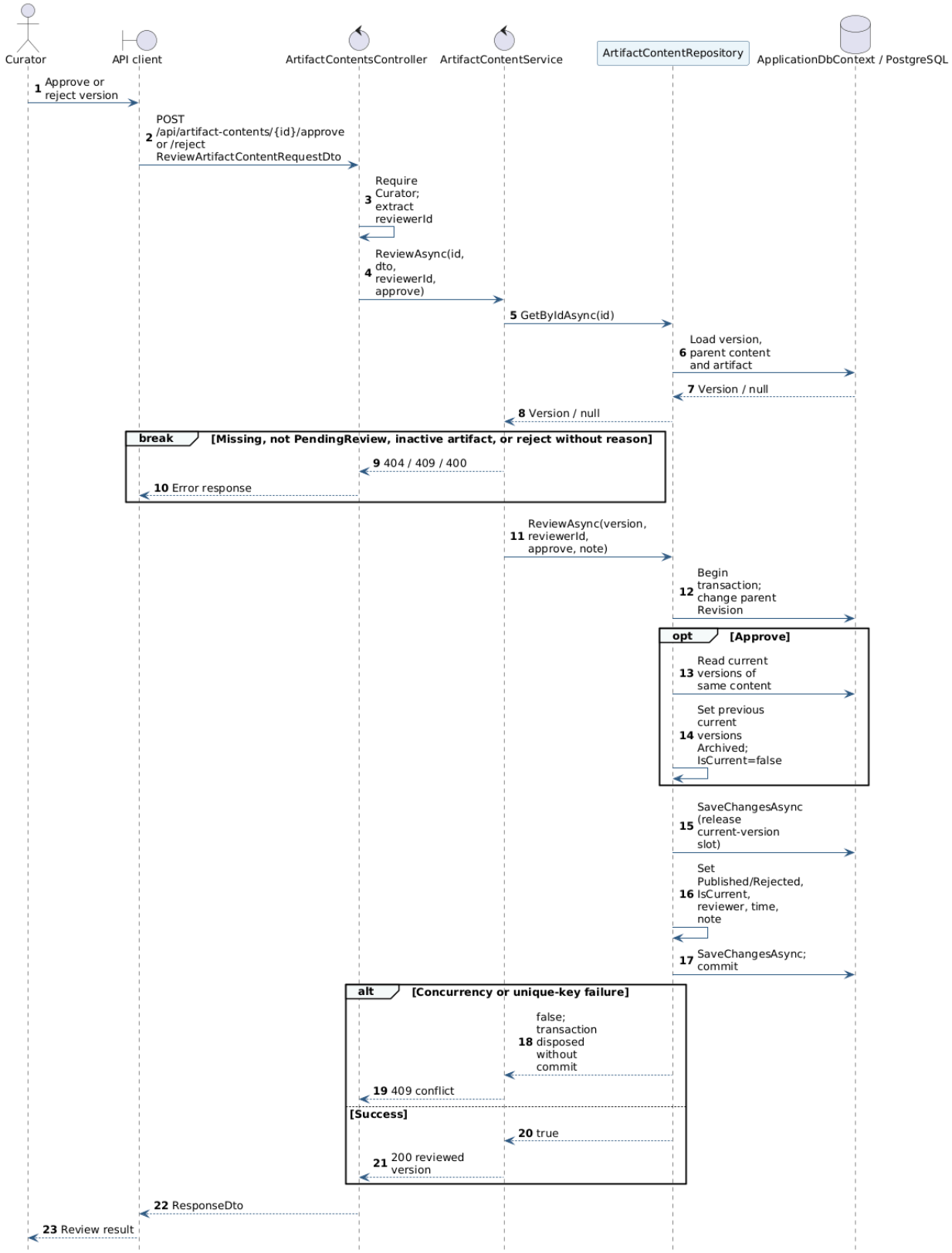
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.7.2 Sequence Diagrams-Submit Content for Review

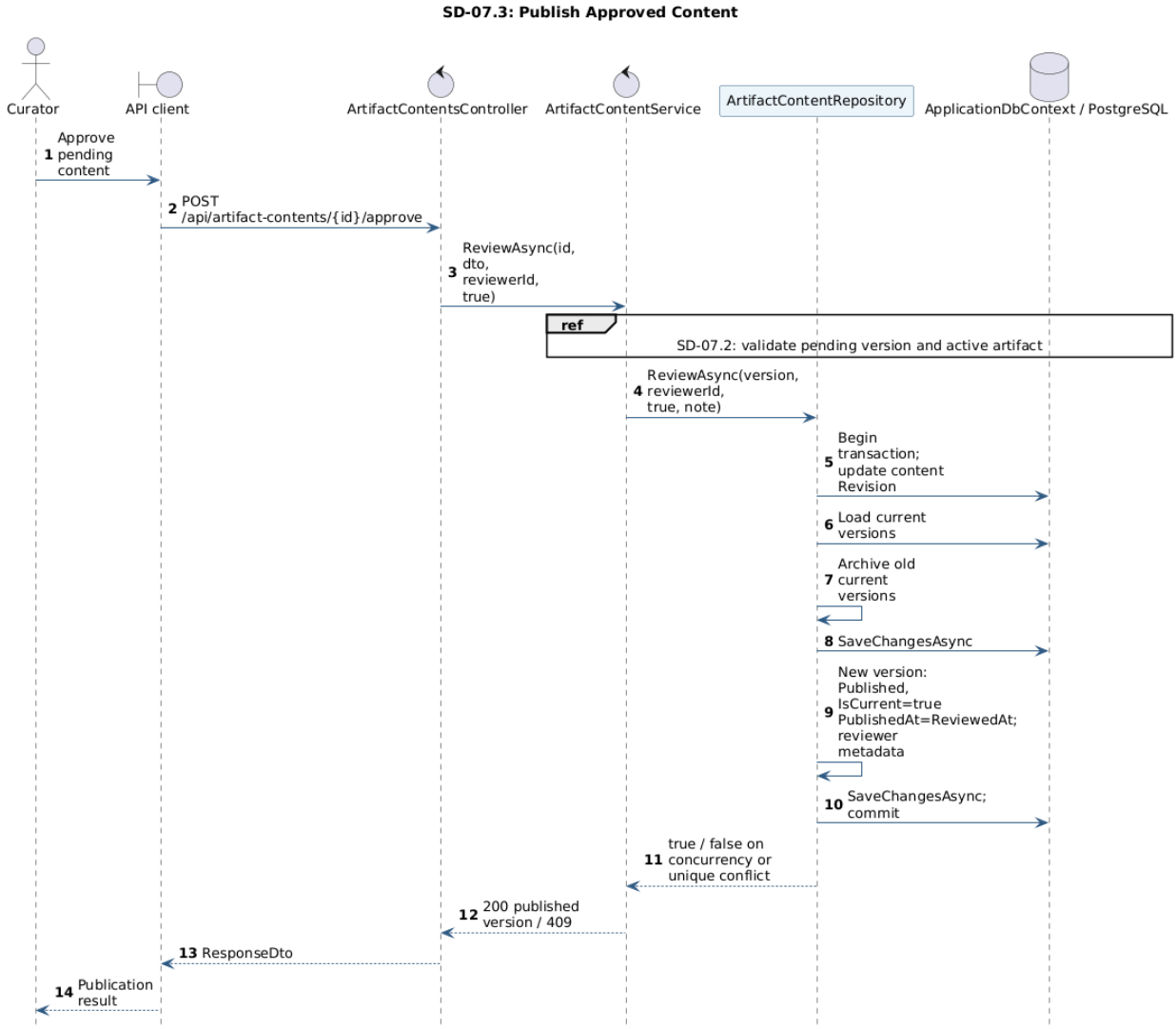


3.7.3 Sequence Diagrams-Approve / Reject Content

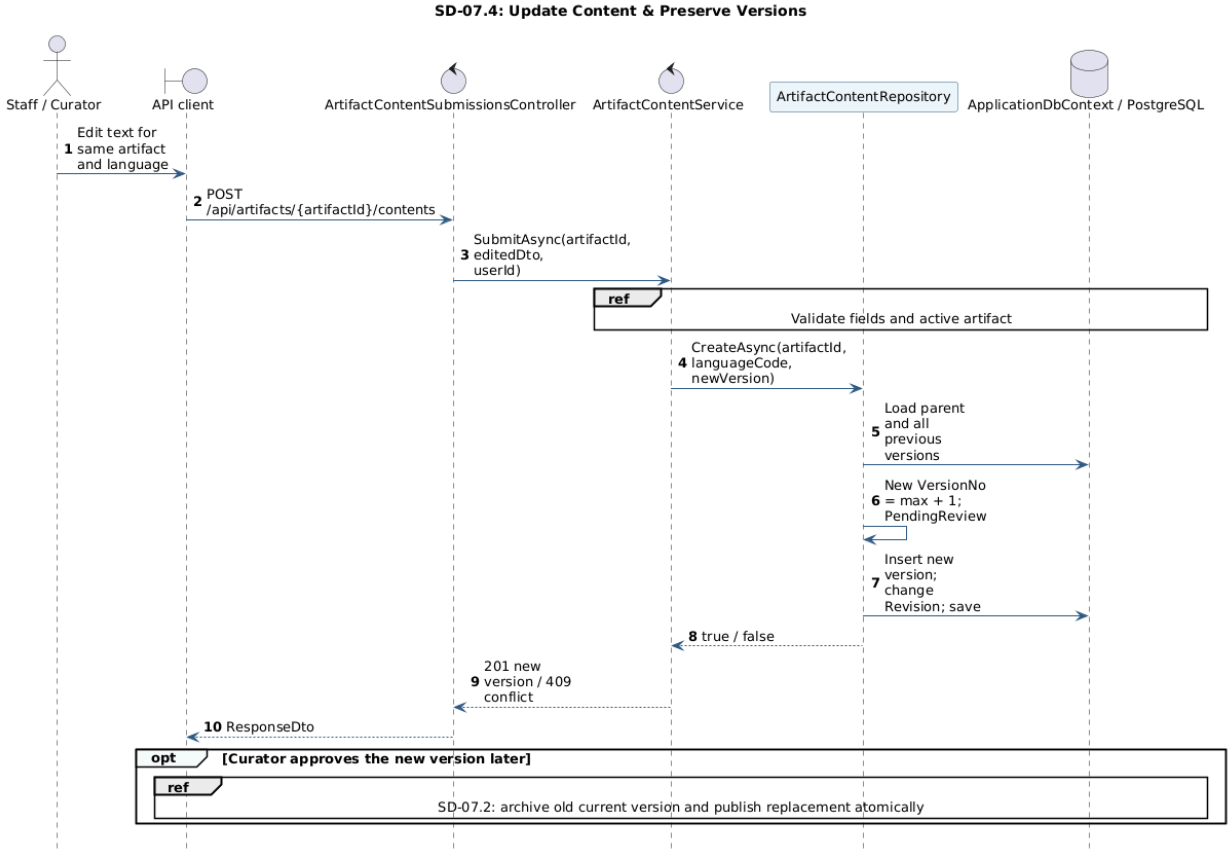
SD-07.2: Approve / Reject Content



3.7.4 Sequence Diagrams-Publish Approved Content



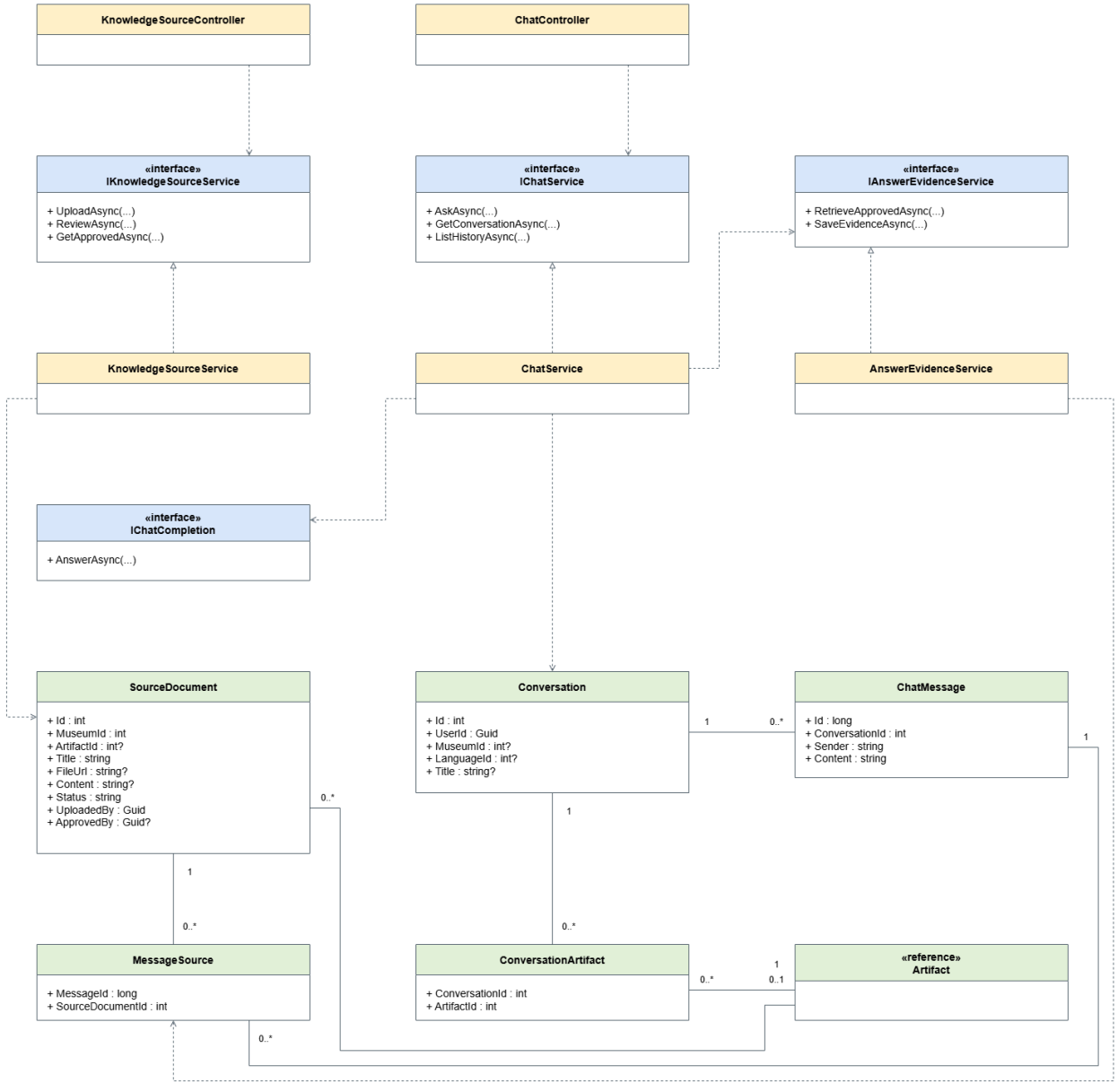
3.7.5 Sequence Diagrams-Update Content & Preserve Versions



3.8 AI Chatbot & Knowledge Management

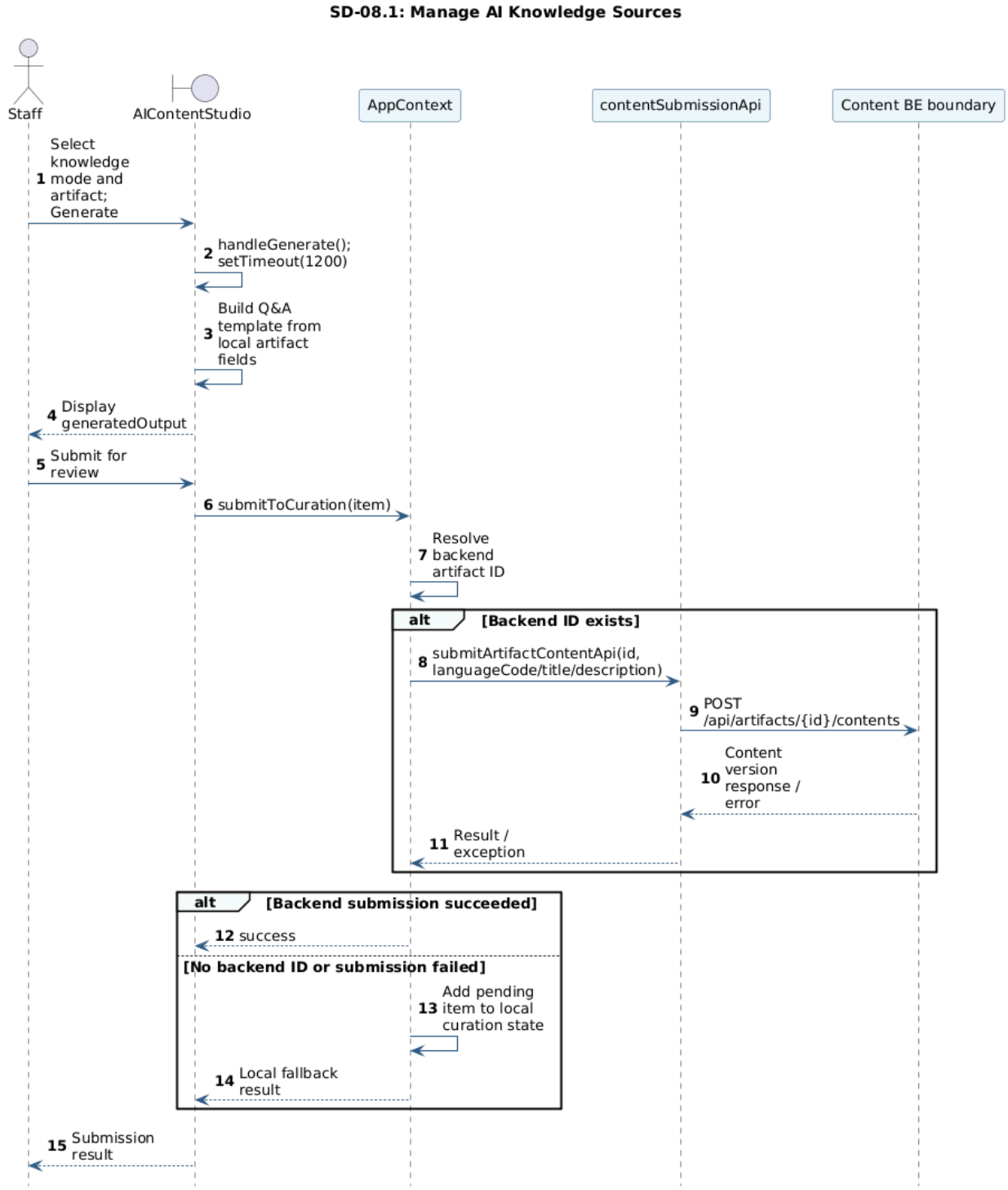
3.8.1 Class Diagram

CD-08 Class Diagram — AI Chatbot & Knowledge Management

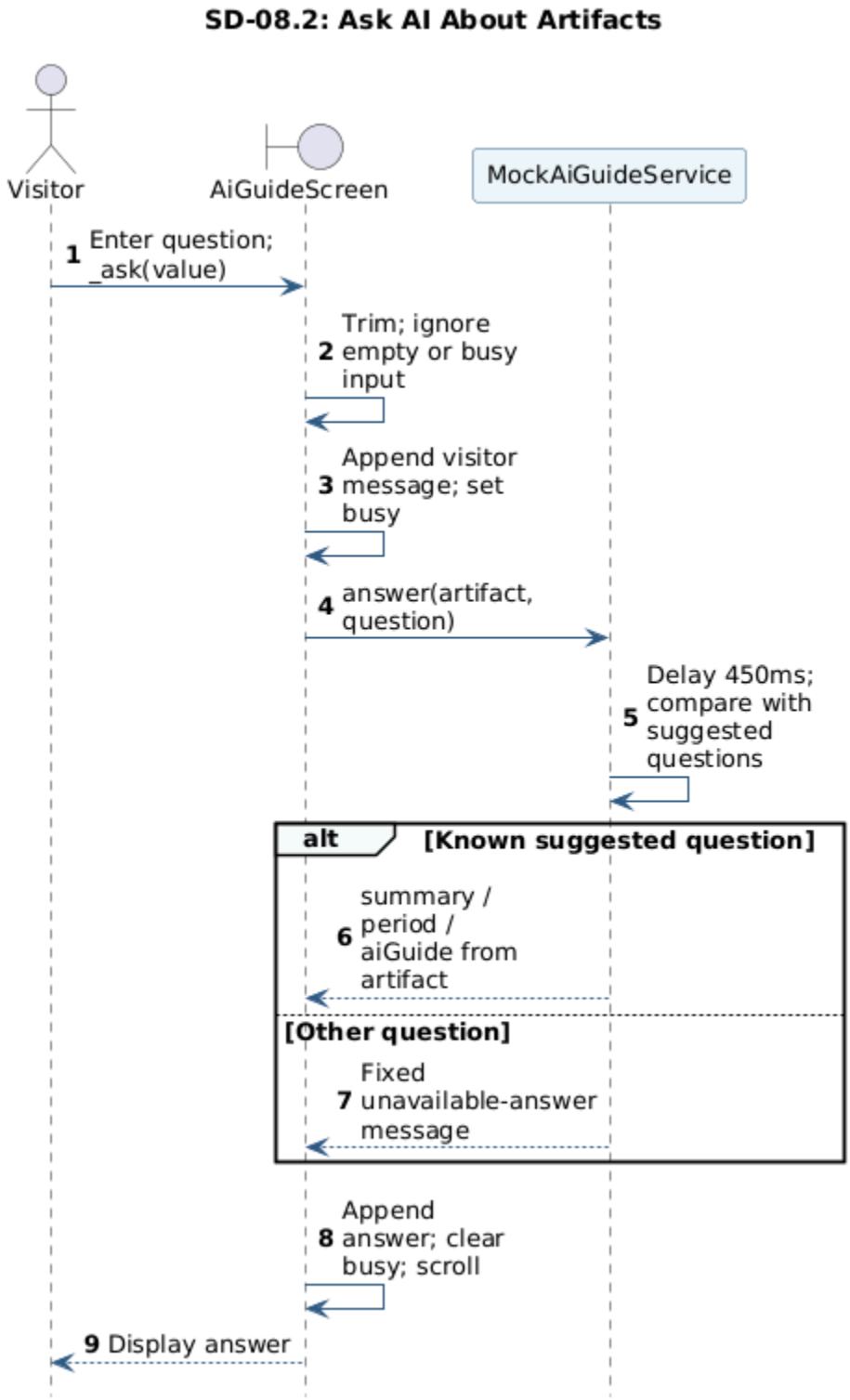


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

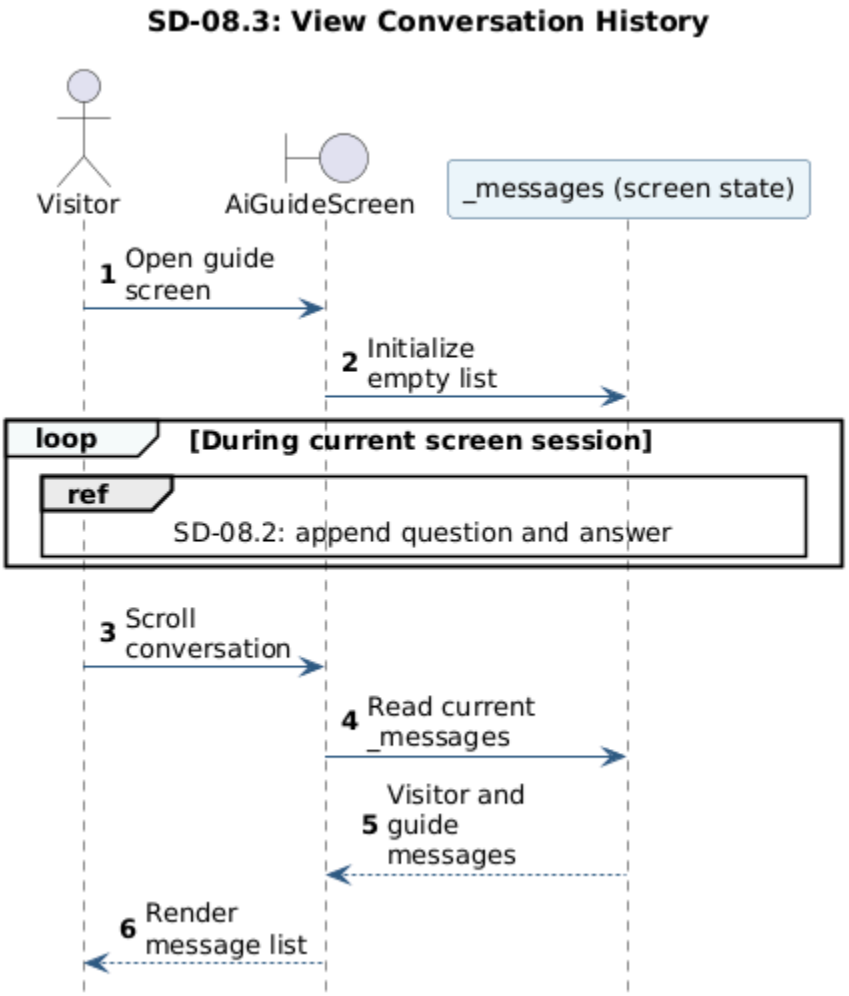
3.8.2 Sequence Diagrams-Manage AI Knowledge Sources



3.8.3 Sequence Diagrams-Ask AI About Artifacts



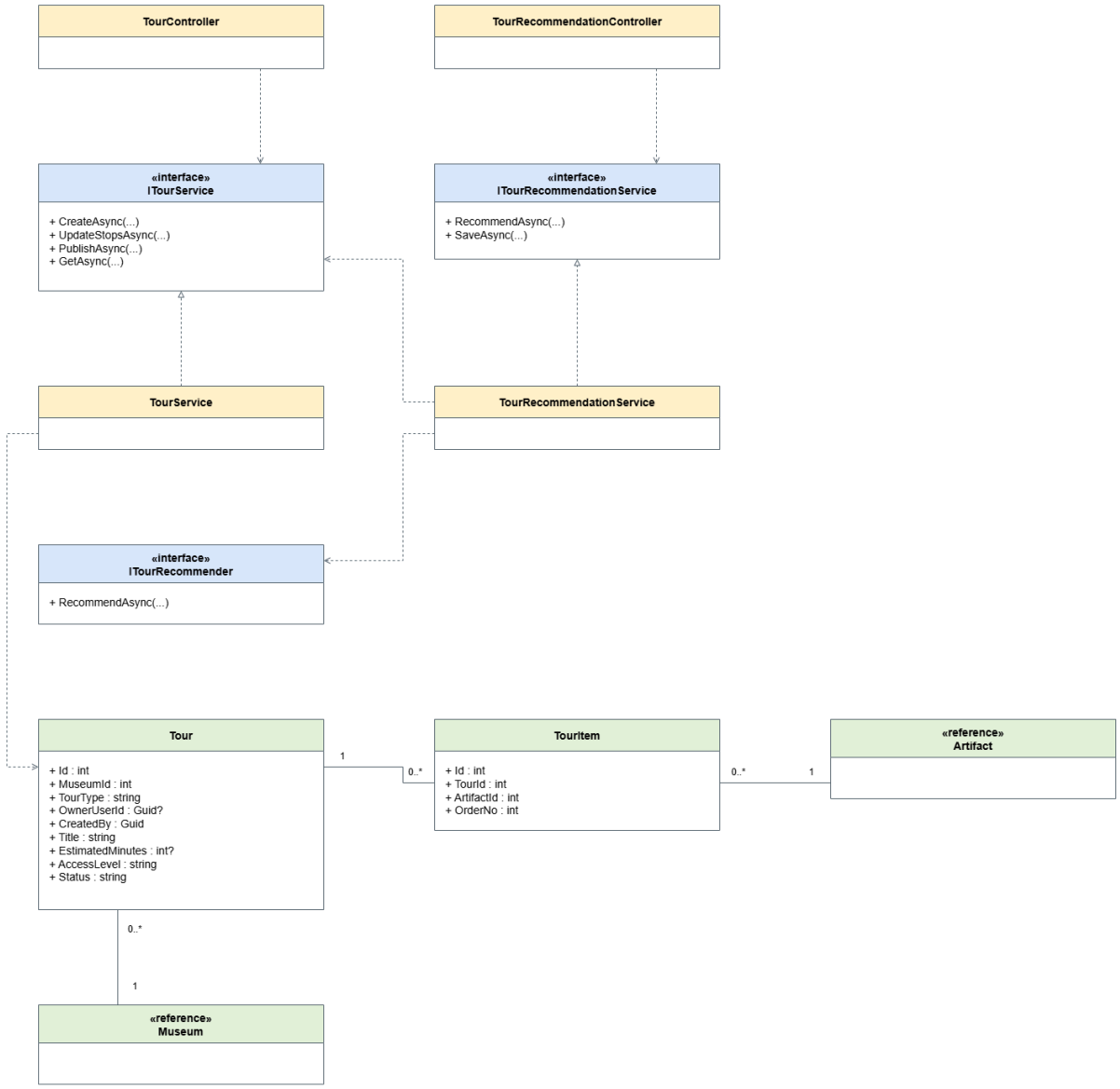
3.8.4 Sequence Diagrams-View Conversation History



3.9 Tour Management & Recommendation

3.9.1 Class Diagram

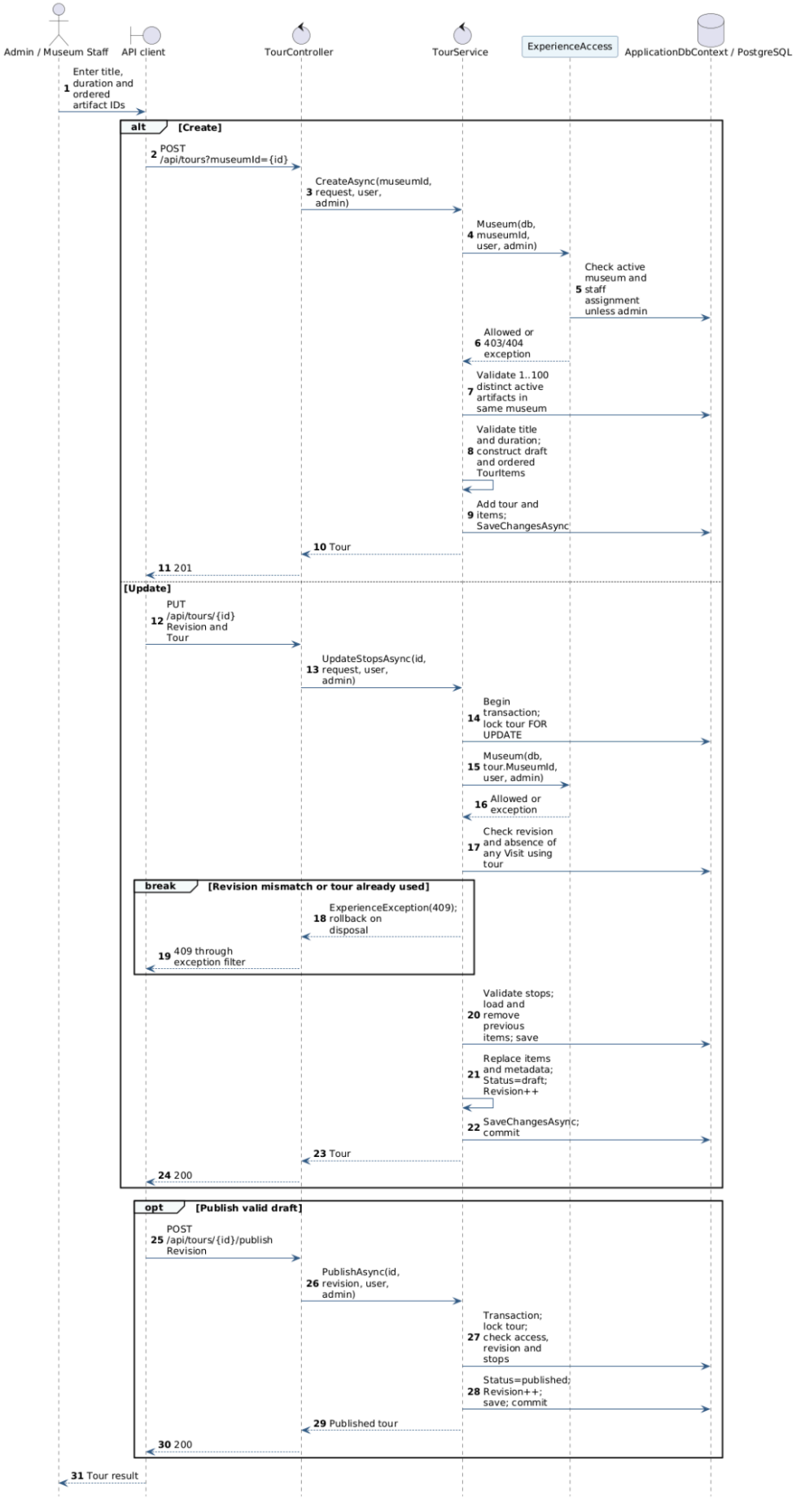
CD-09 Class Diagram — Tour Management & Recommendation



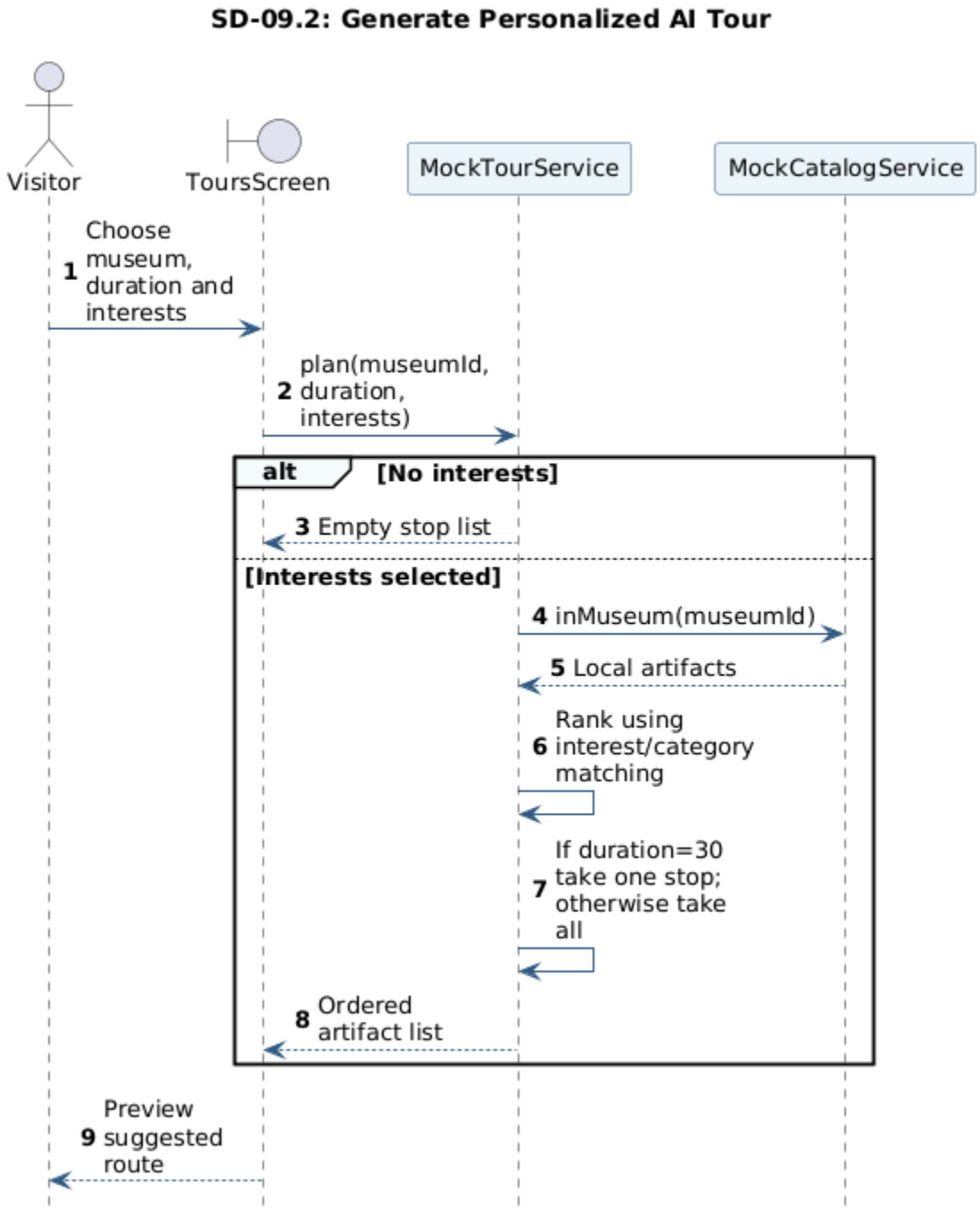
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.9.2 Sequence Diagrams-Create / Update Predefined Tour

SD-09.1: Create / Update Predefined Tour



3.9.3 Sequence Diagrams-Generate Personalized AI Tour

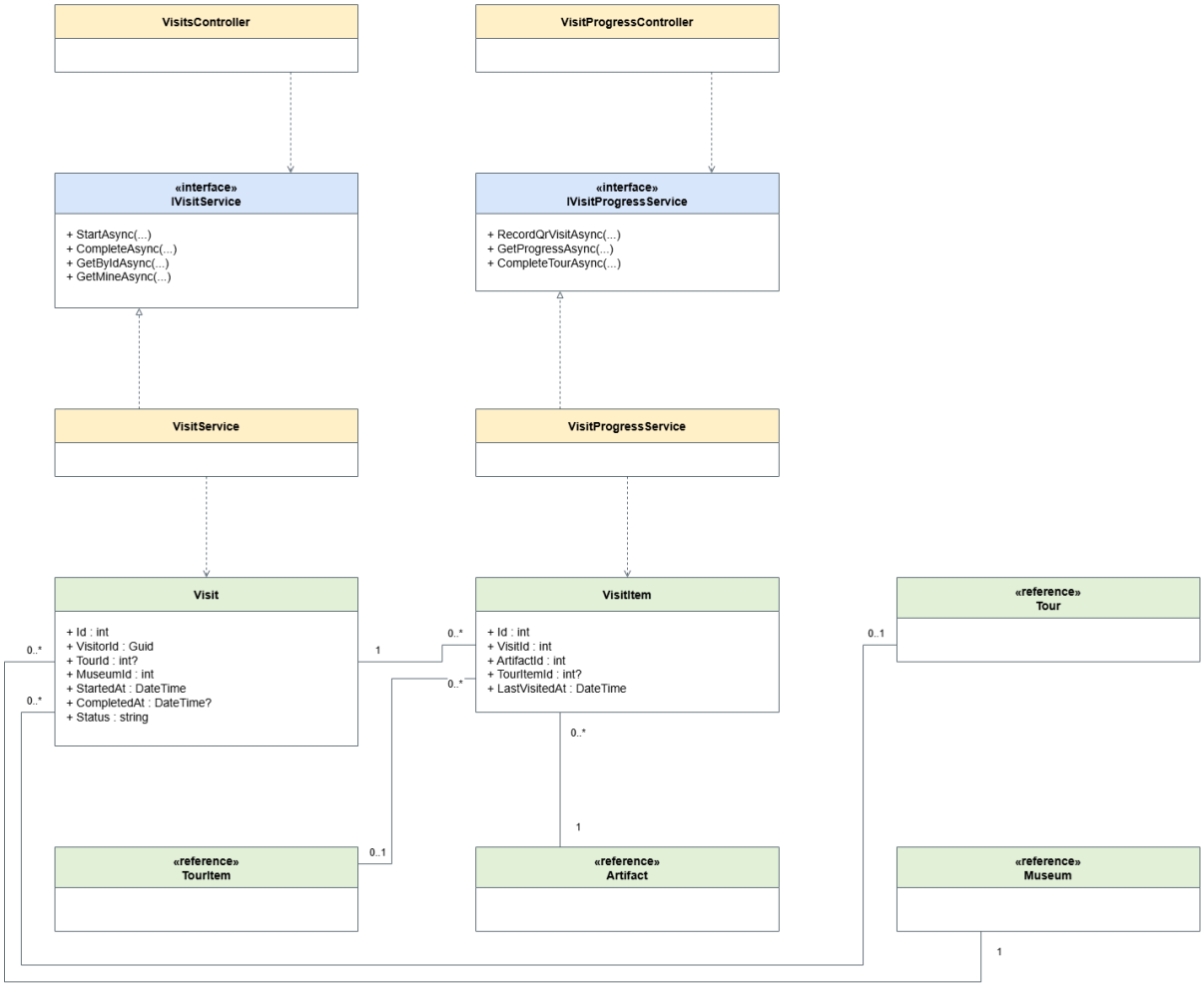


3.9.4 Sequence Diagrams-Save / Edit AI-Recommended Tour

3.10 Visit & Tour Progress Management

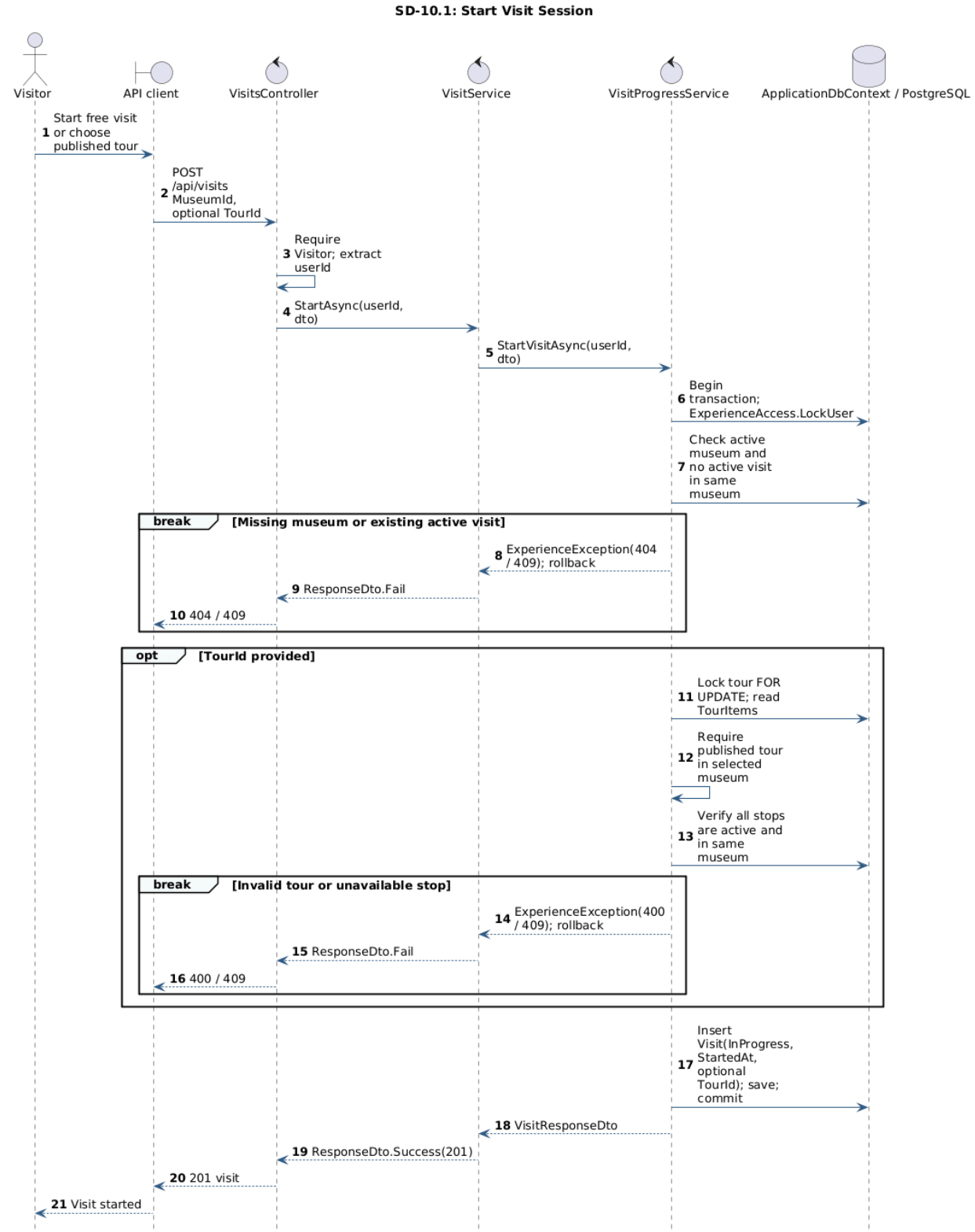
3.10.1 Class Diagram

CD-10 Class Diagram — Visit & Tour Progress Management



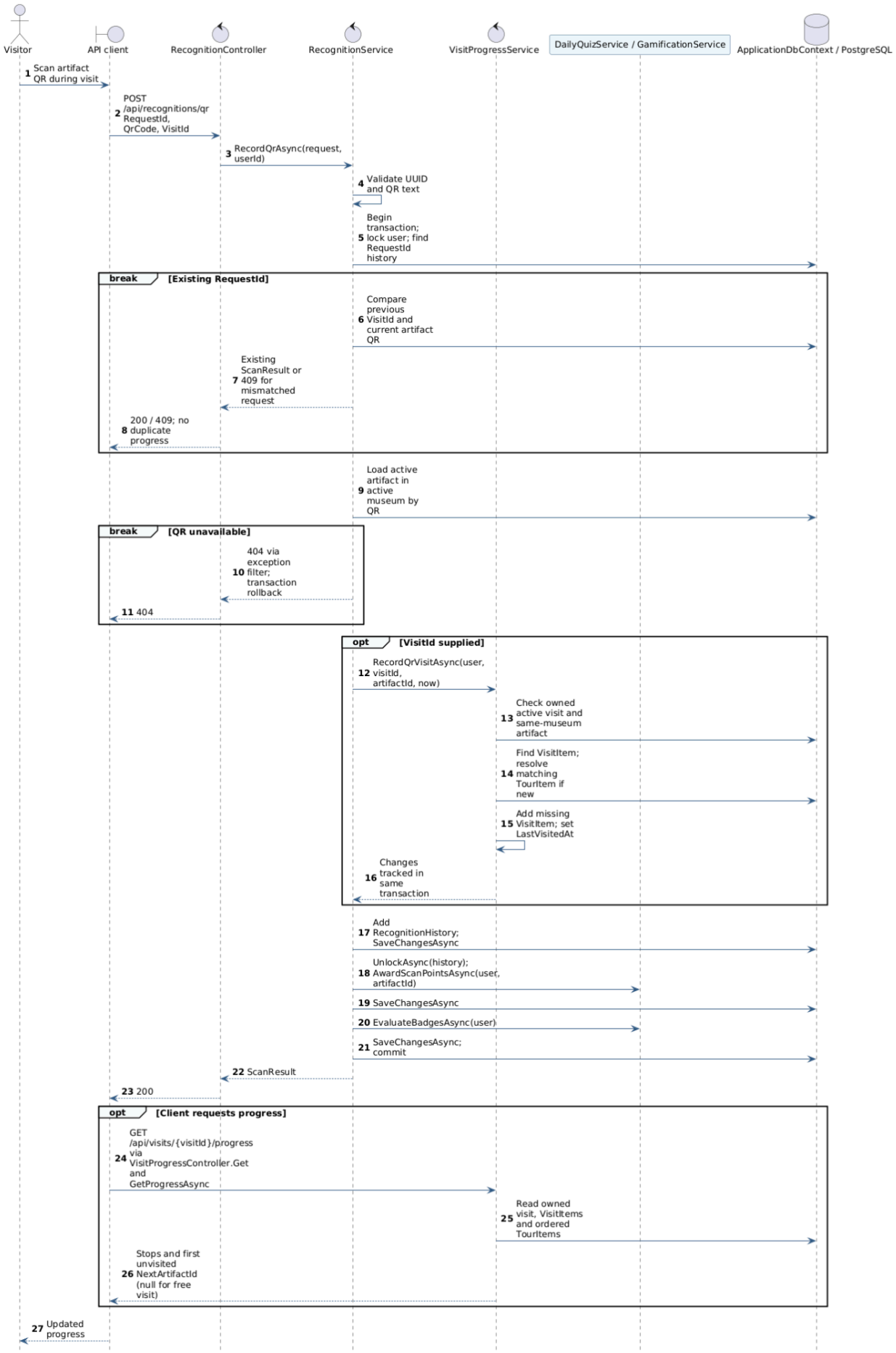
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.10.2 Sequence Diagrams-Start Visit Session

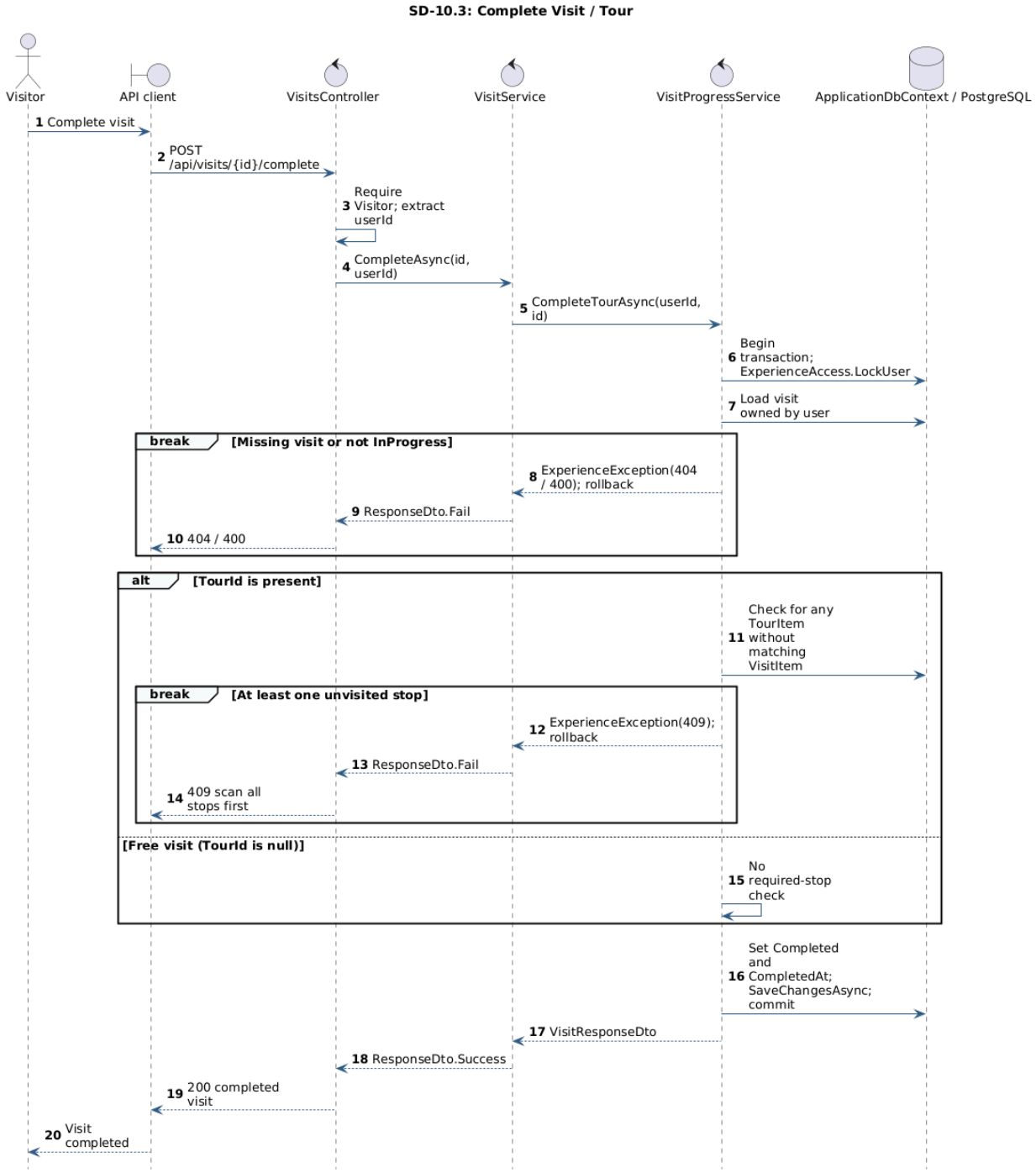


3.10.3 Sequence Diagrams-Update Tour Progress After QR Scan

SD-10.2: Update Tour Progress After QR Scan



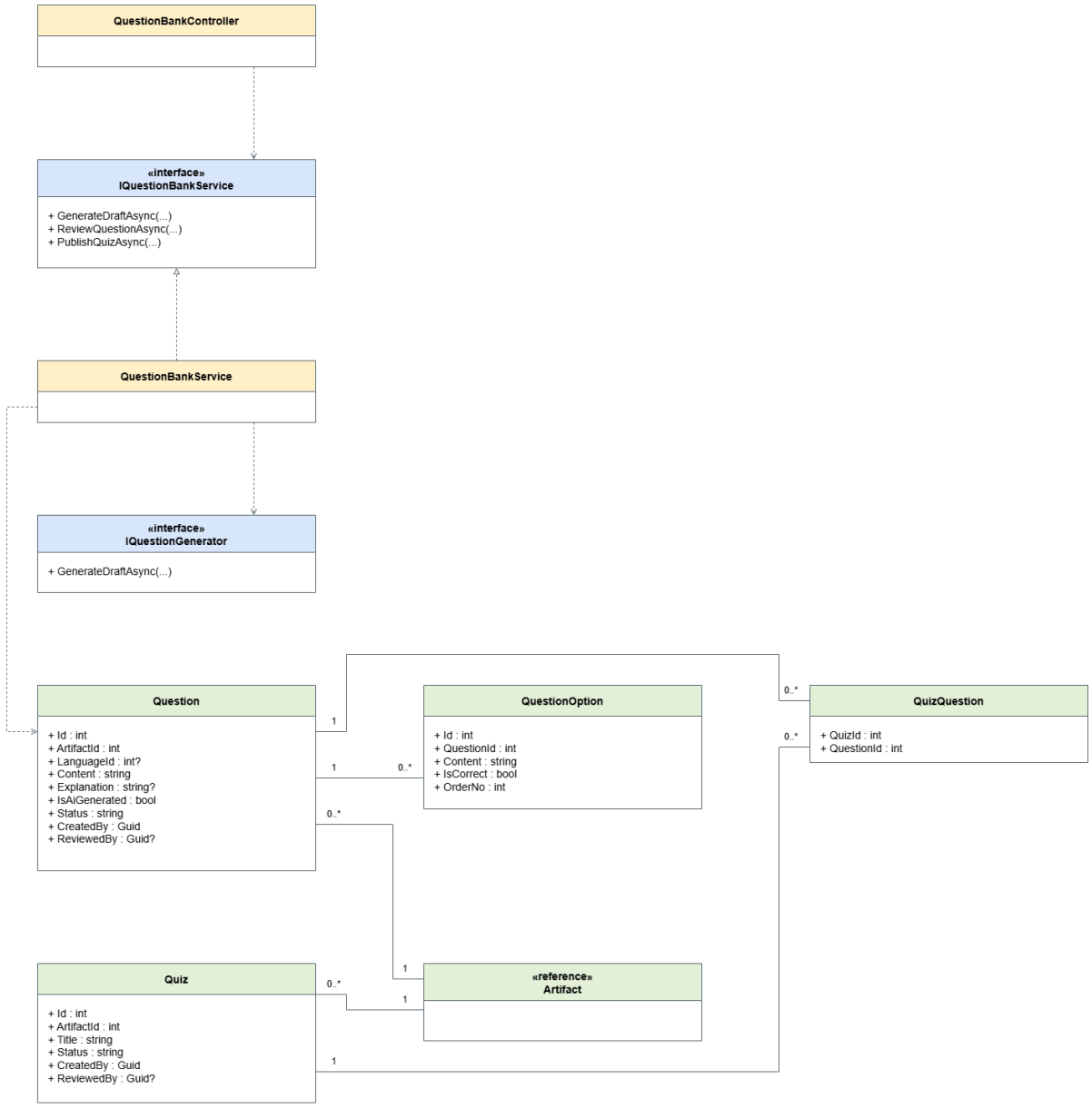
3.10.4 Sequence Diagrams-Complete Visit / Tour



3.11 Quiz Management & Participation

3.11.1 Class Diagram

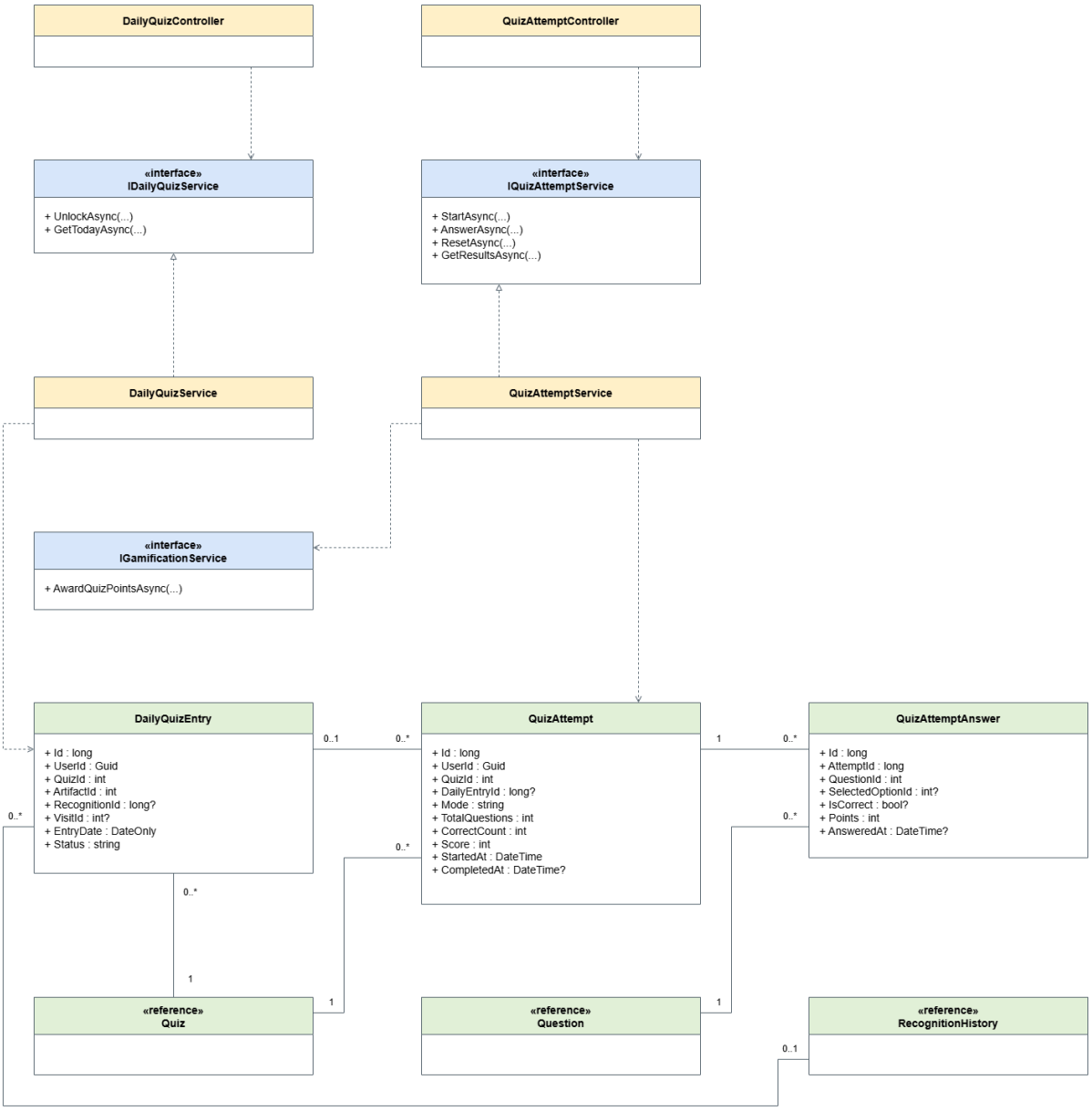
CD-11A Class Diagram — Quiz Question Bank & Approval



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

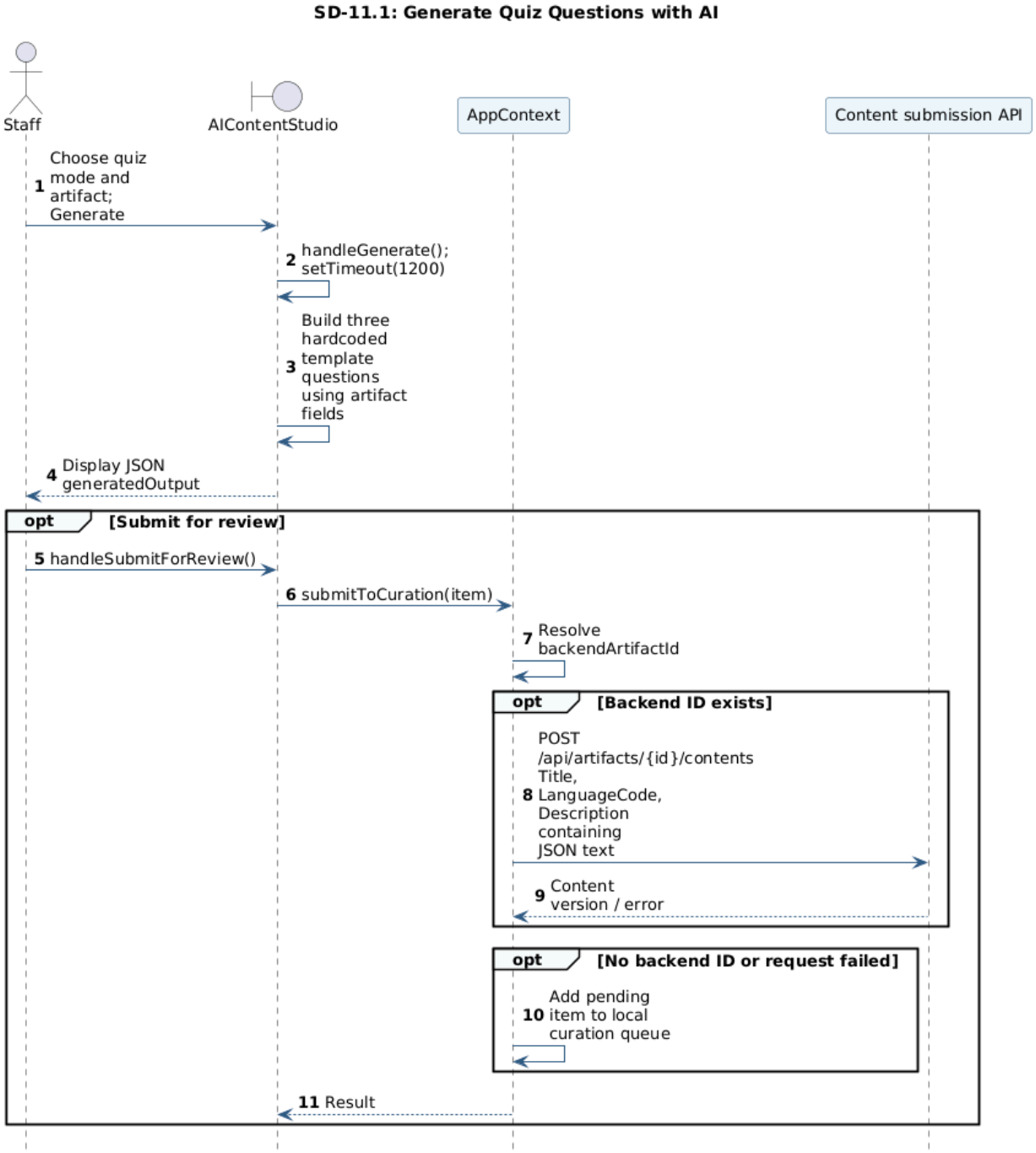
3.11.2 Class Diagram

CD-11B Class Diagram — Daily Quiz & Quiz Attempts



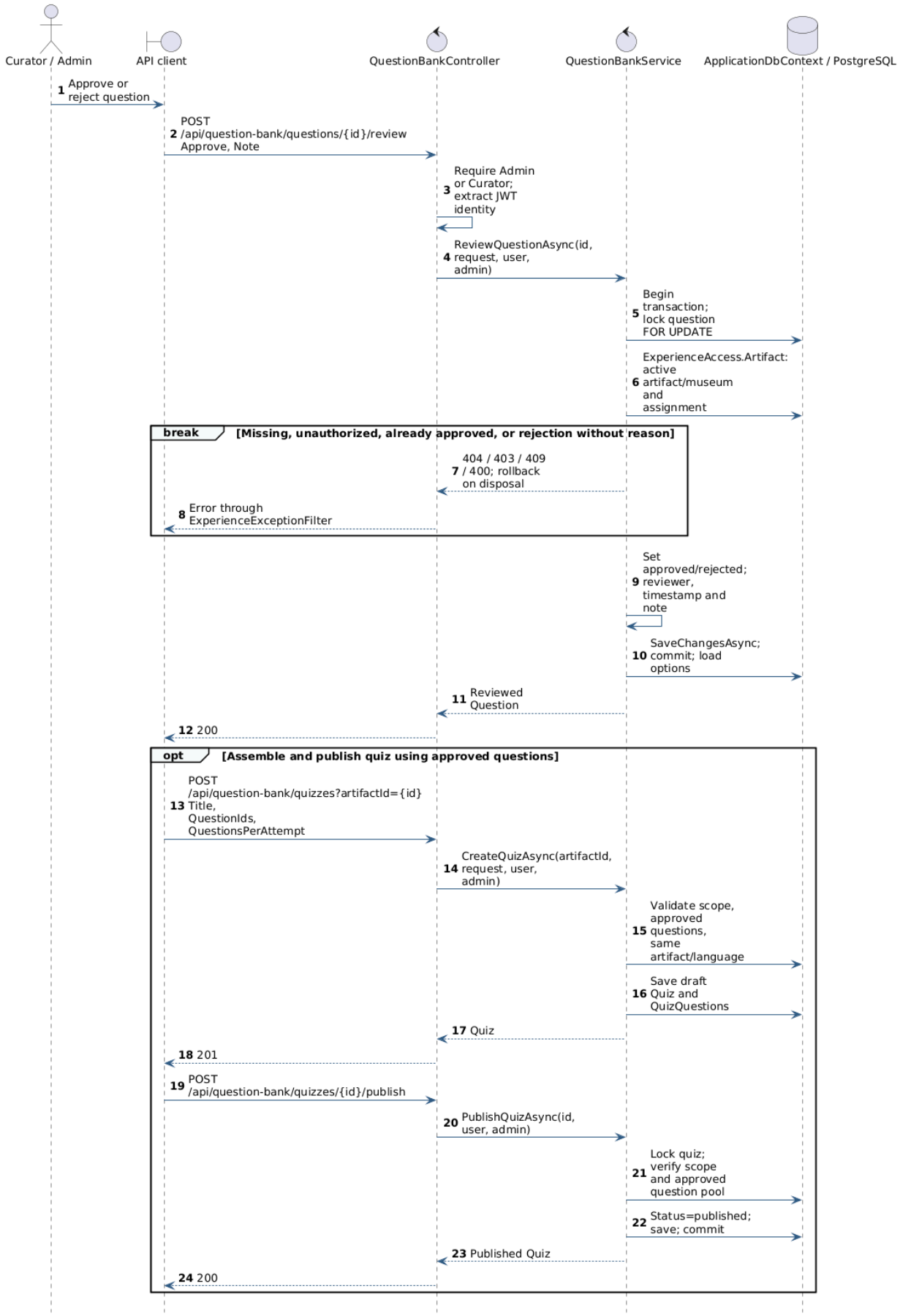
Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.11.3 Sequence Diagrams-Generate Quiz Questions with AI

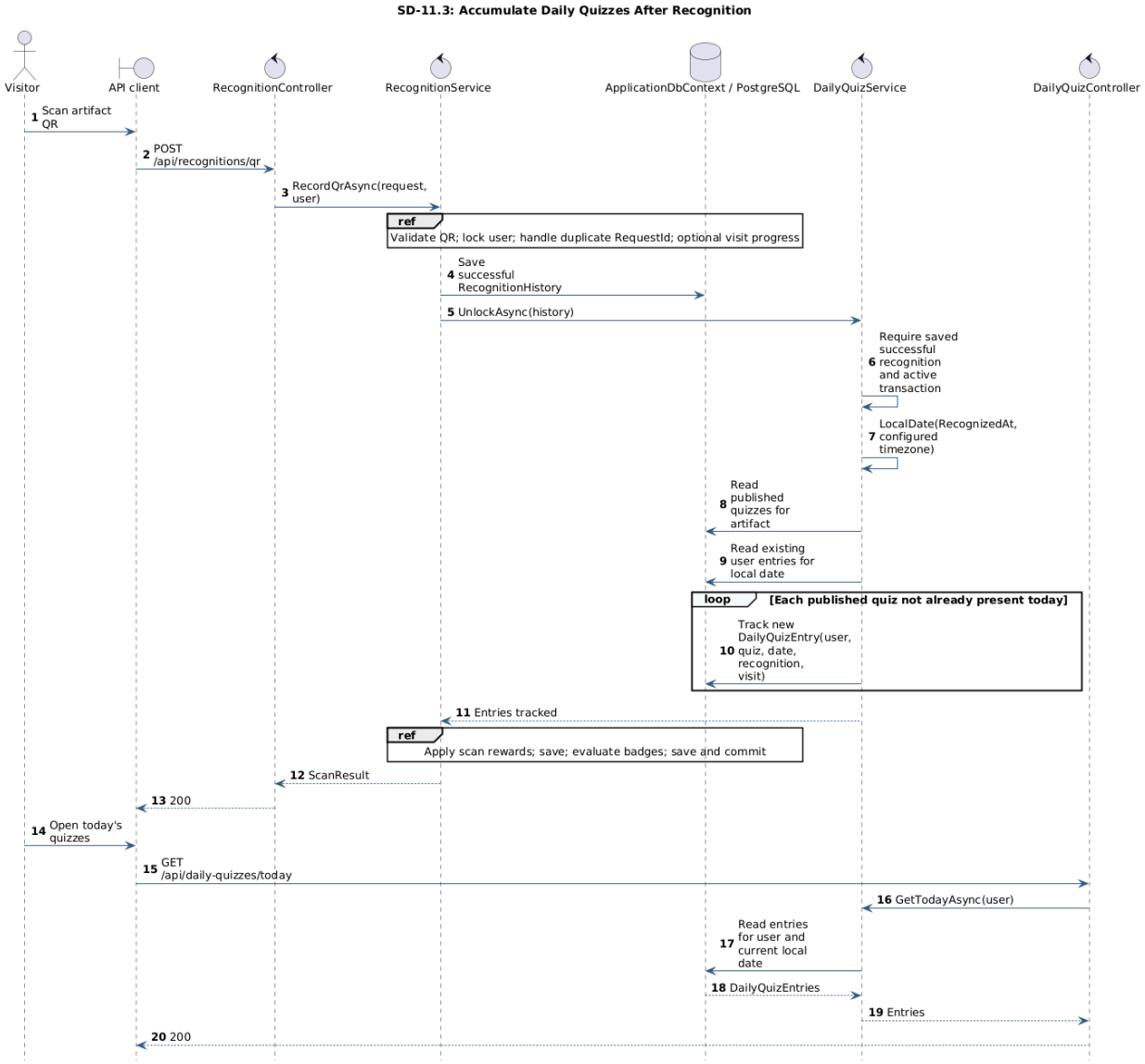


3.11.4 Sequence Diagrams-Review / Approve Quiz Questions

SD-11.2: Review / Approve Quiz Questions

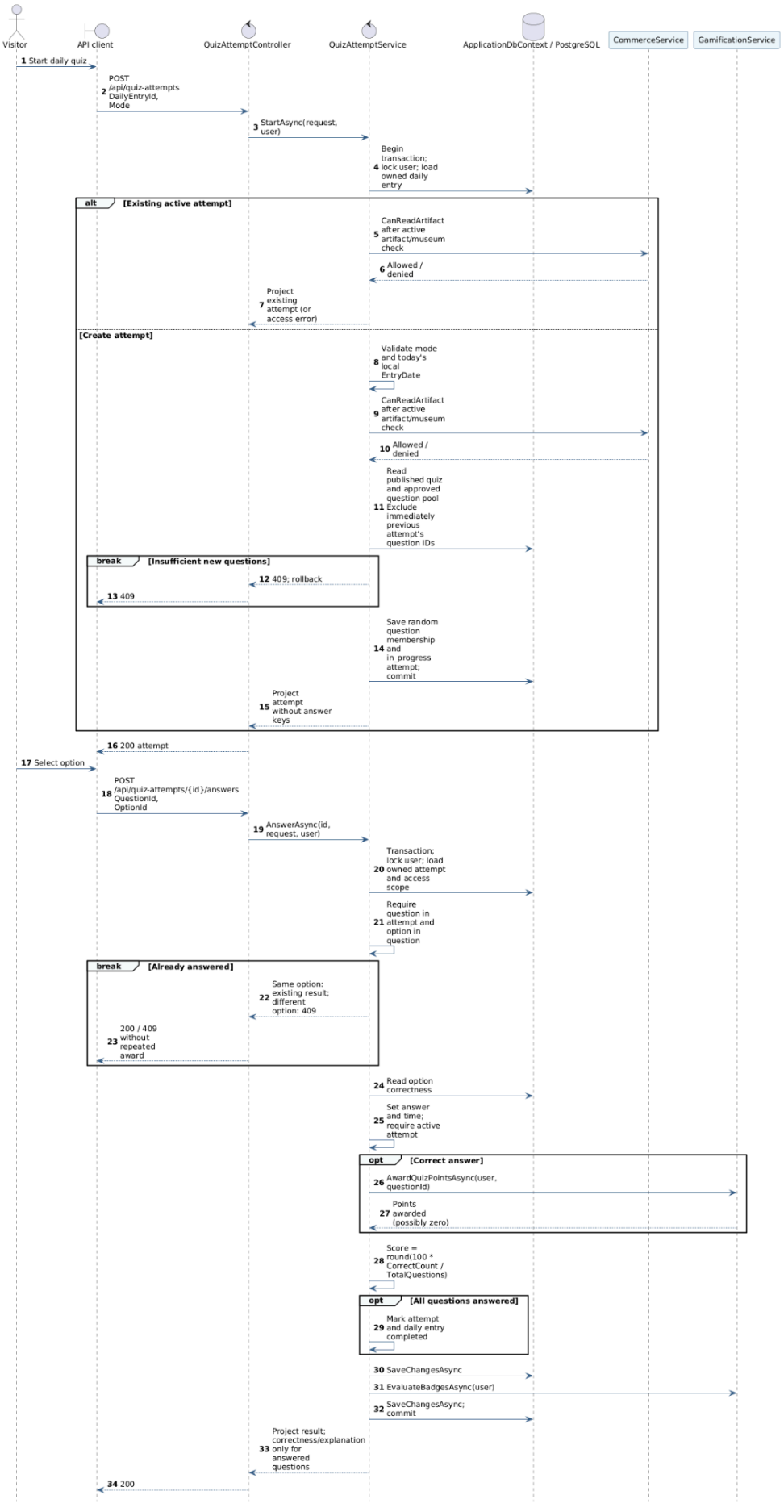


3.11.5 Sequence Diagrams-Accumulate Daily Quizzes After Recognition

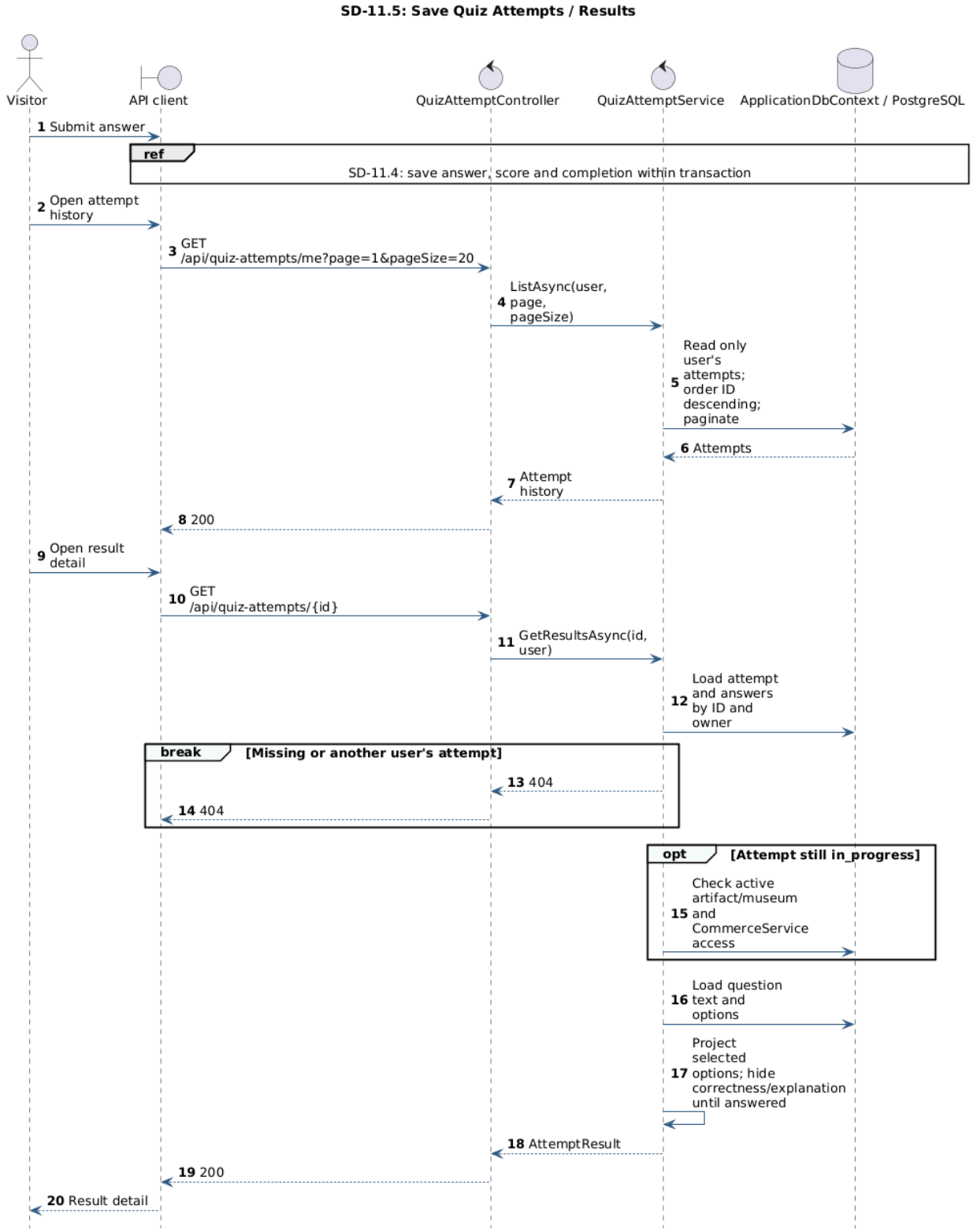


3.11.6 Sequence Diagrams-Take Quiz & Evaluate Answers

SD-11.4: Take Quiz & Evaluate Answers

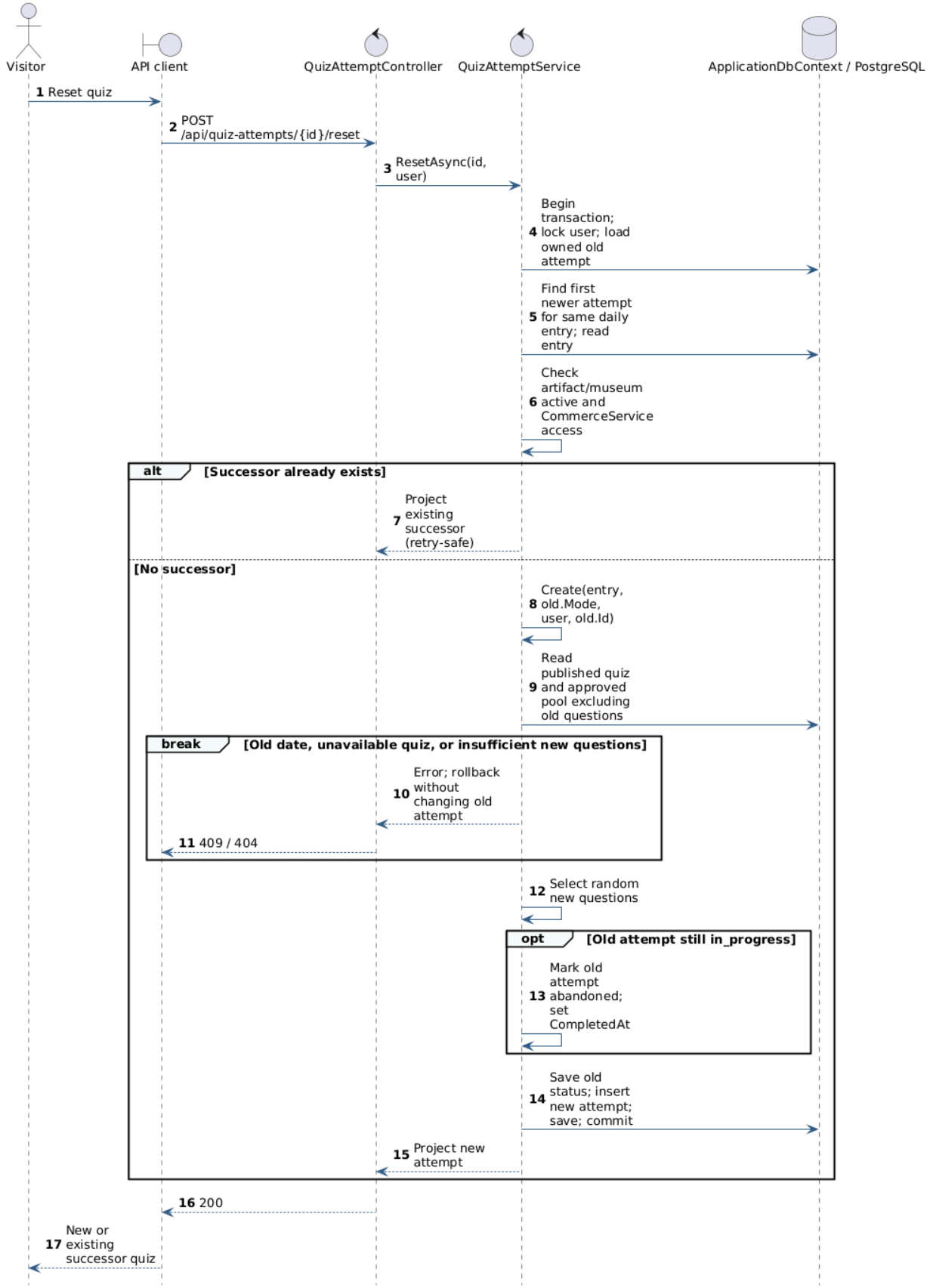


3.11.7 Sequence Diagrams-Save Quiz Attempts / Results



3.11.8 Sequence Diagrams-Reset Quiz for New Questions

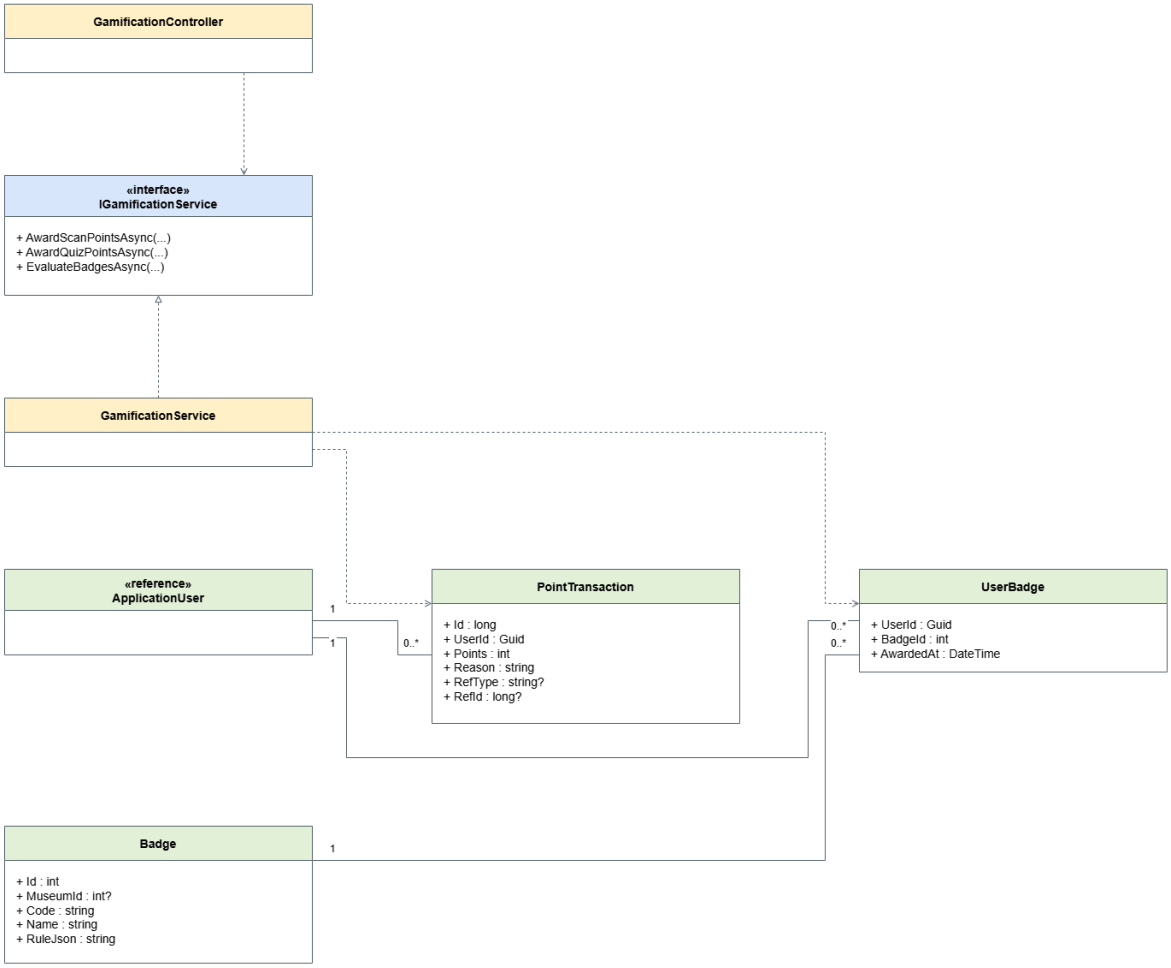
SD-11.6: Reset Quiz for New Questions



3.12 Gamification & Rewards

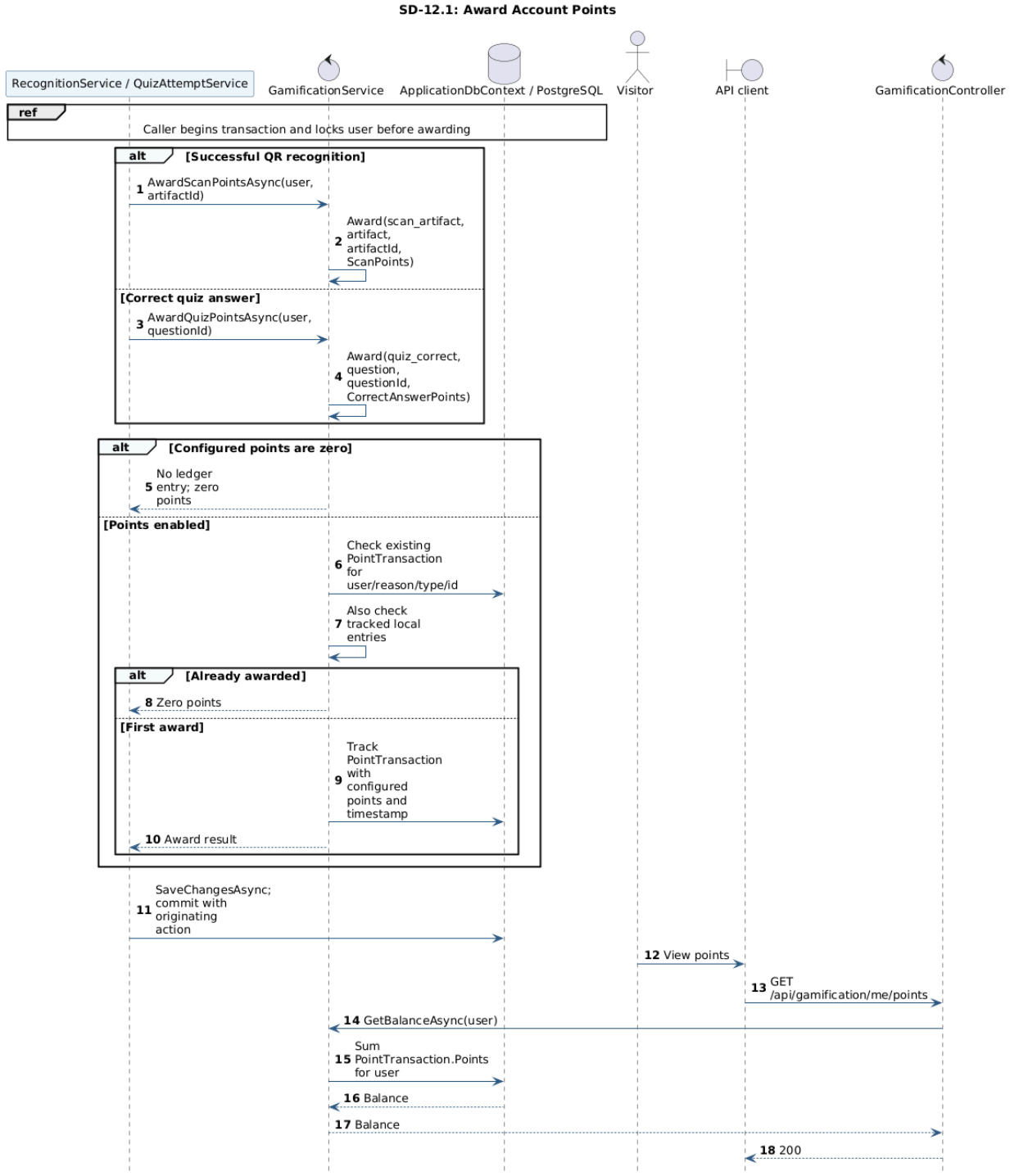
3.12.1 Class Diagram

CD-12 Class Diagram — Gamification & Rewards

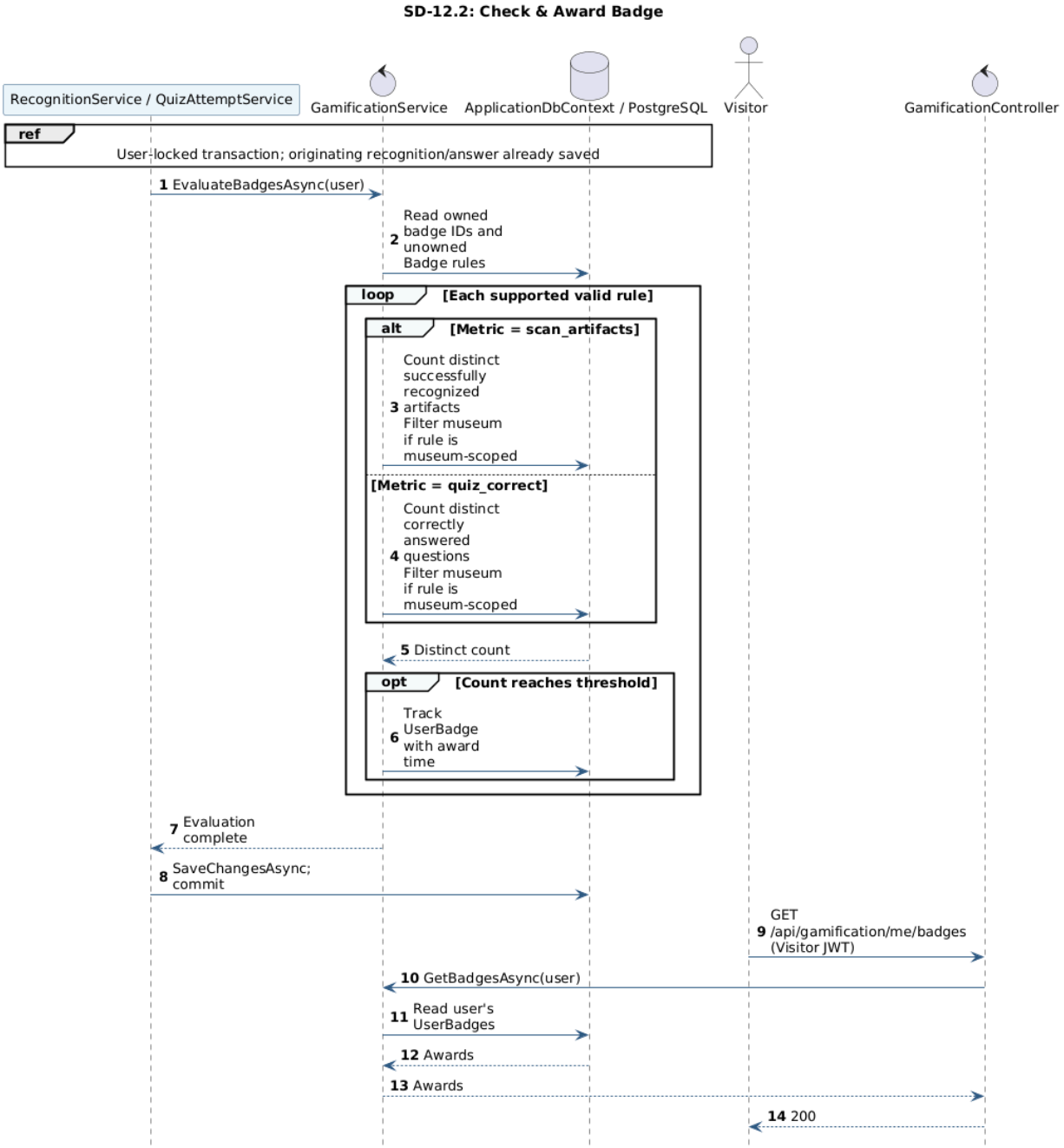


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.12.2 Sequence Diagrams-Award Account Points



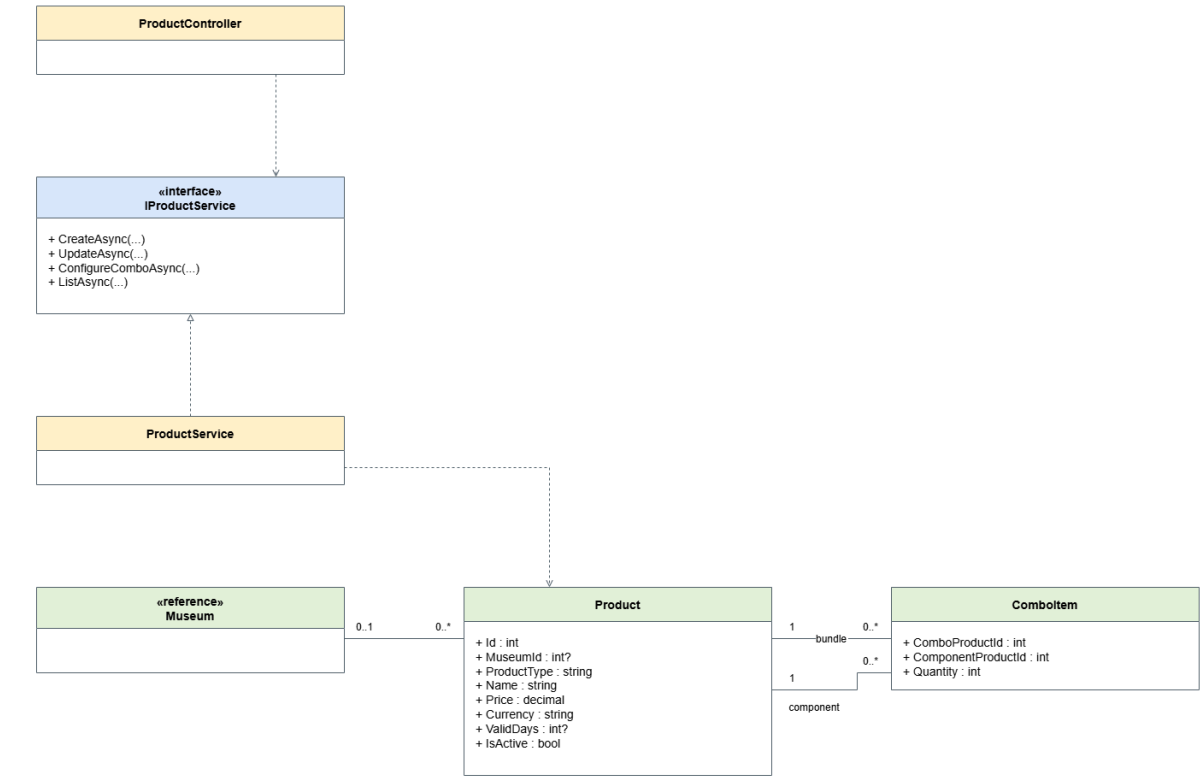
3.12.3 Sequence Diagrams-Check & Award Badge



3.13 Ticketing, Digital Services & Payment

3.13.1 Class Diagram

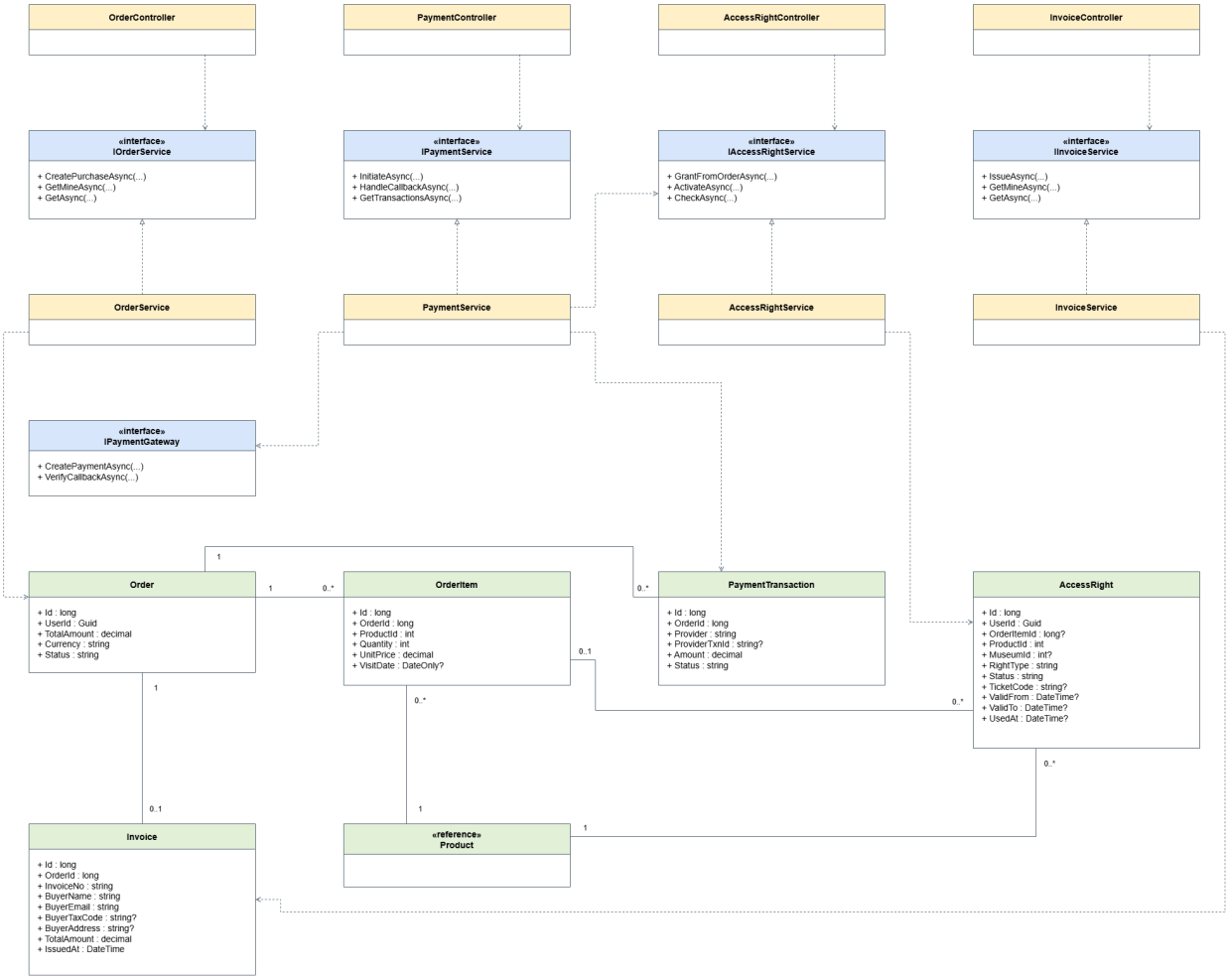
CD-13A Class Diagram — Ticket & Digital Service Catalog



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

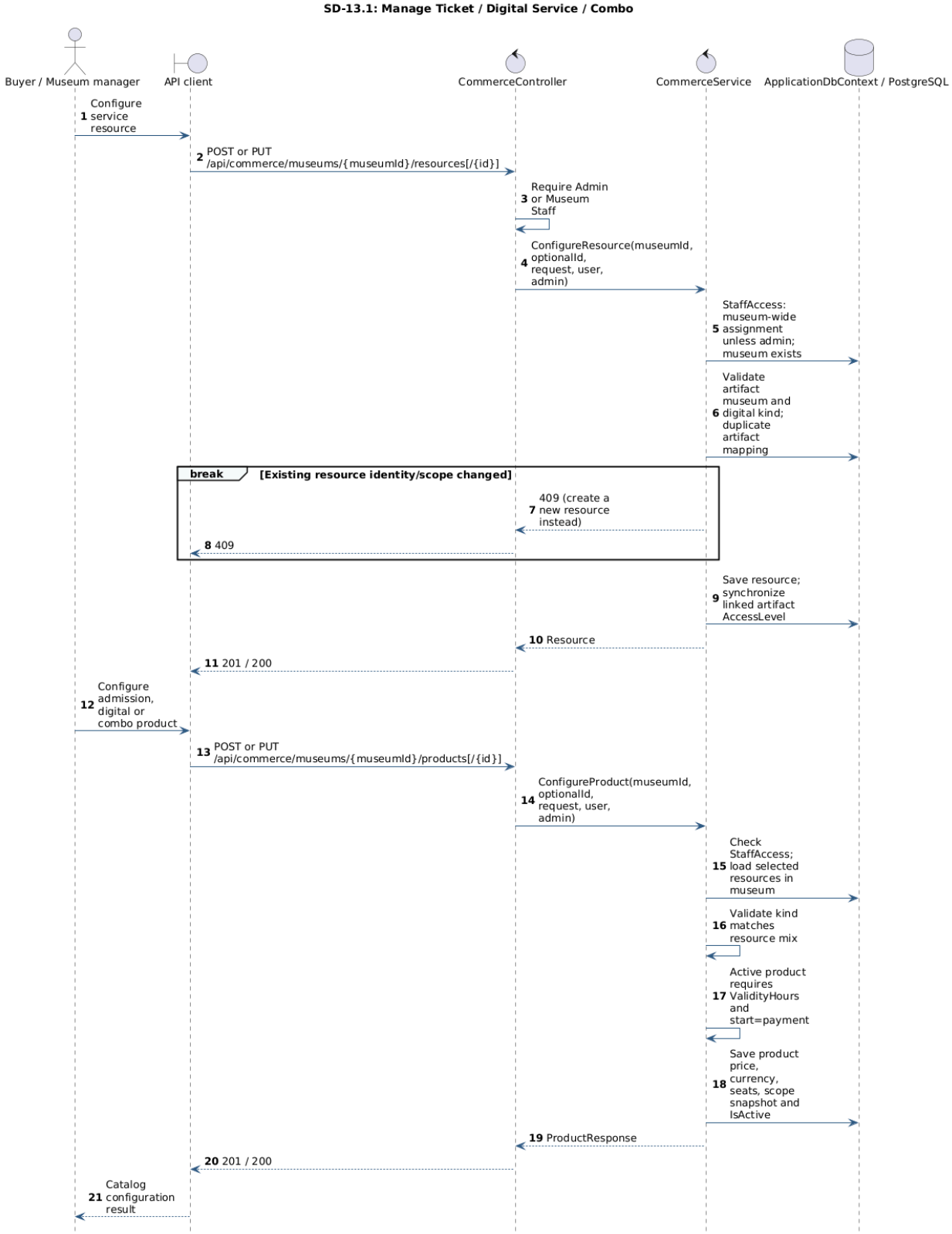
3.13.2 Class Diagram

CD-13B Class Diagram — Order, Payment & Entitlement

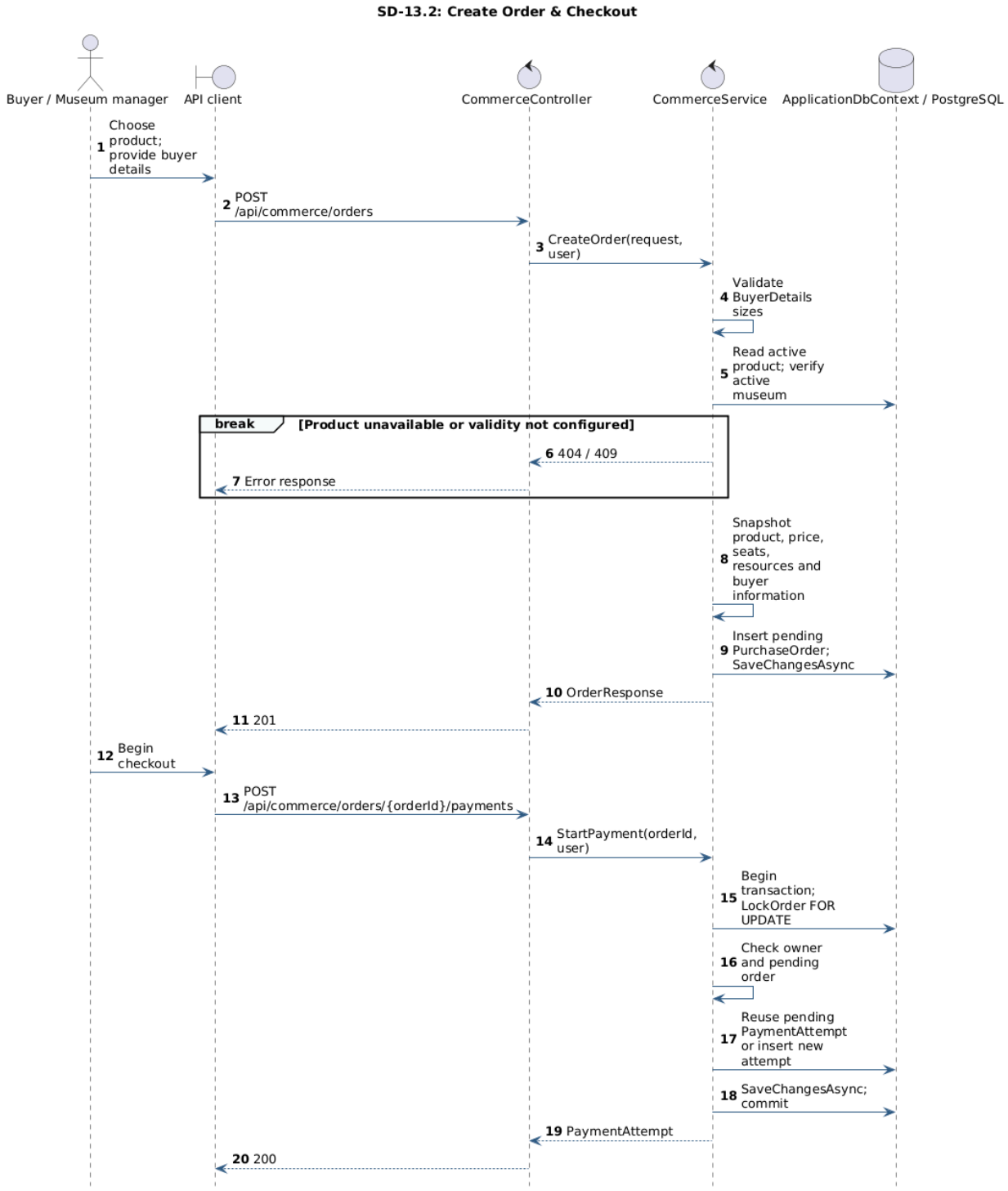


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.13.3 Sequence Diagrams-Manage Ticket / Digital Service / Combo

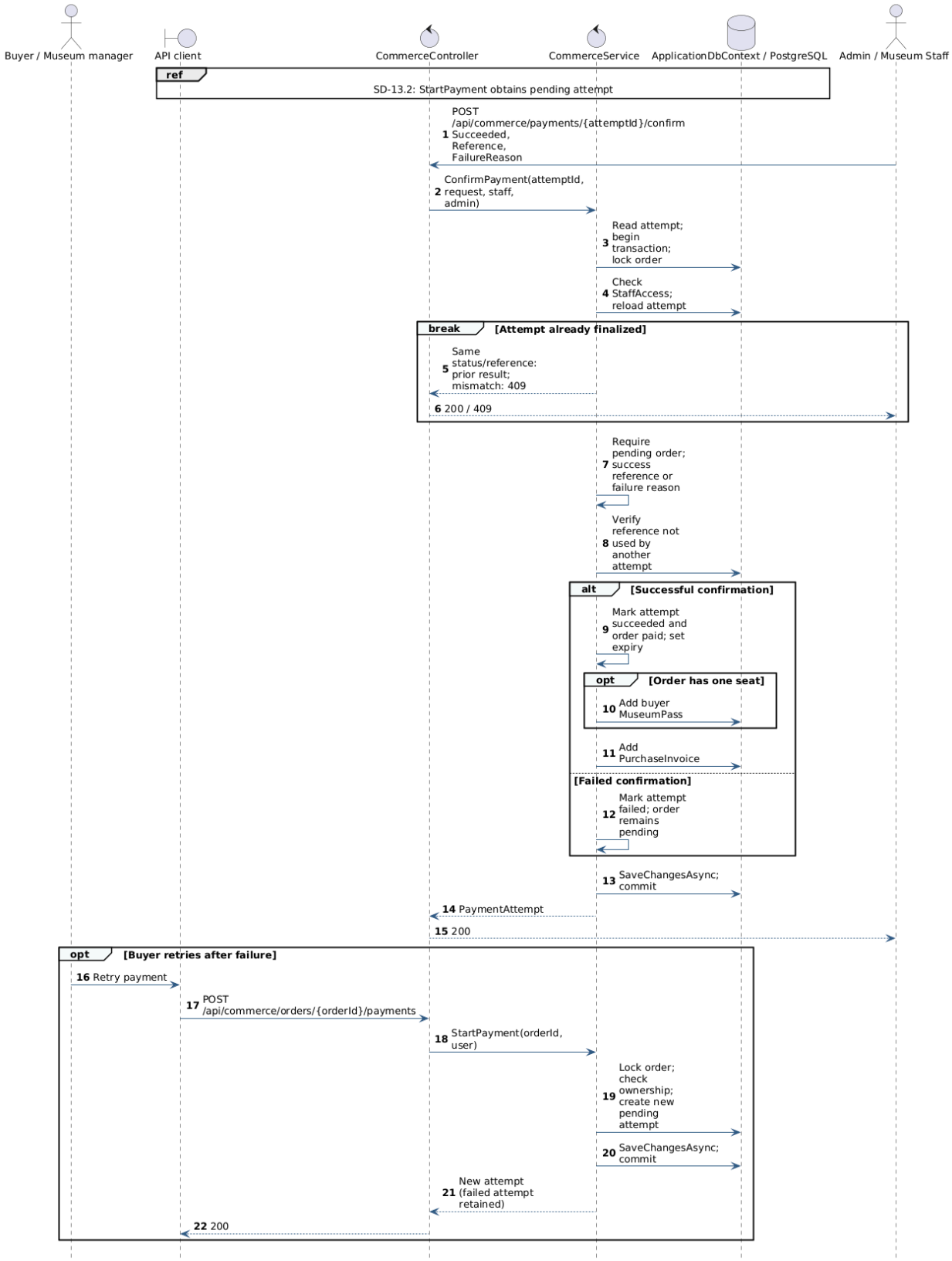


3.13.4 Sequence Diagrams-Create Order & Checkout



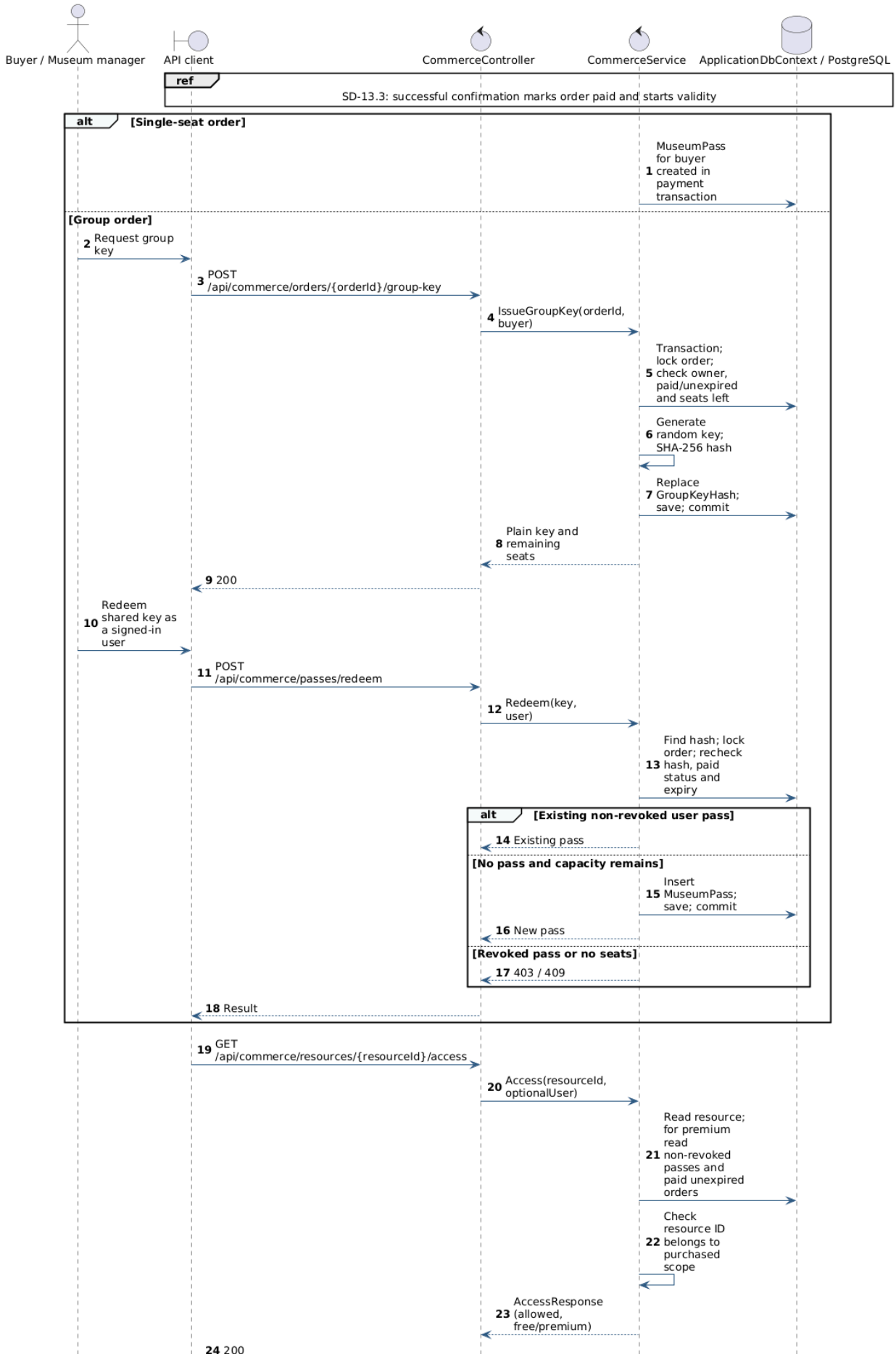
3.13.5 Sequence Diagrams-Process Payment & Retry

SD-13.3: Process Payment & Retry

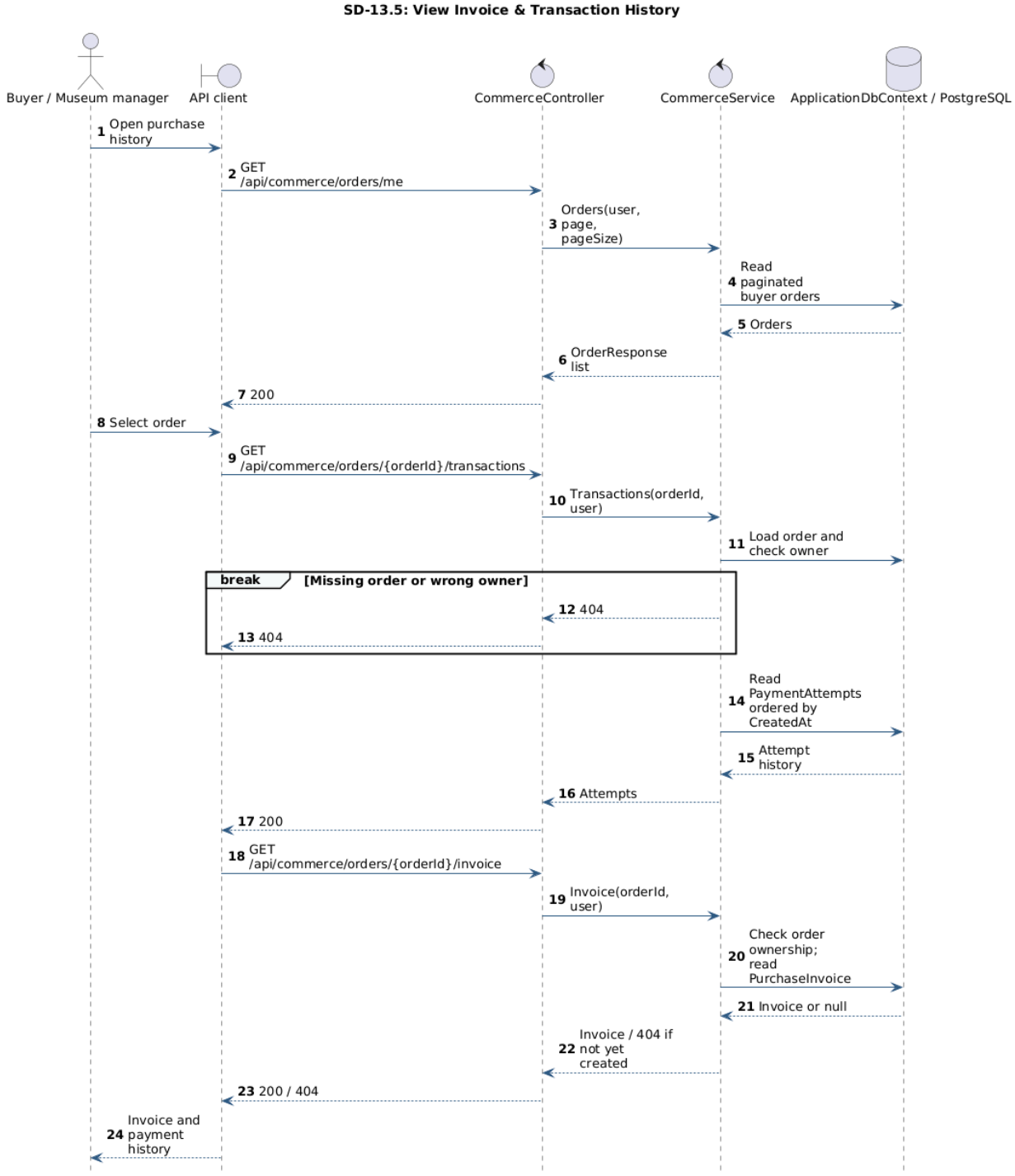


3.13.6 Sequence Diagrams-Activate Ticket / Digital Service Entitlement

SD-13.4: Activate Ticket / Digital Service Entitlement



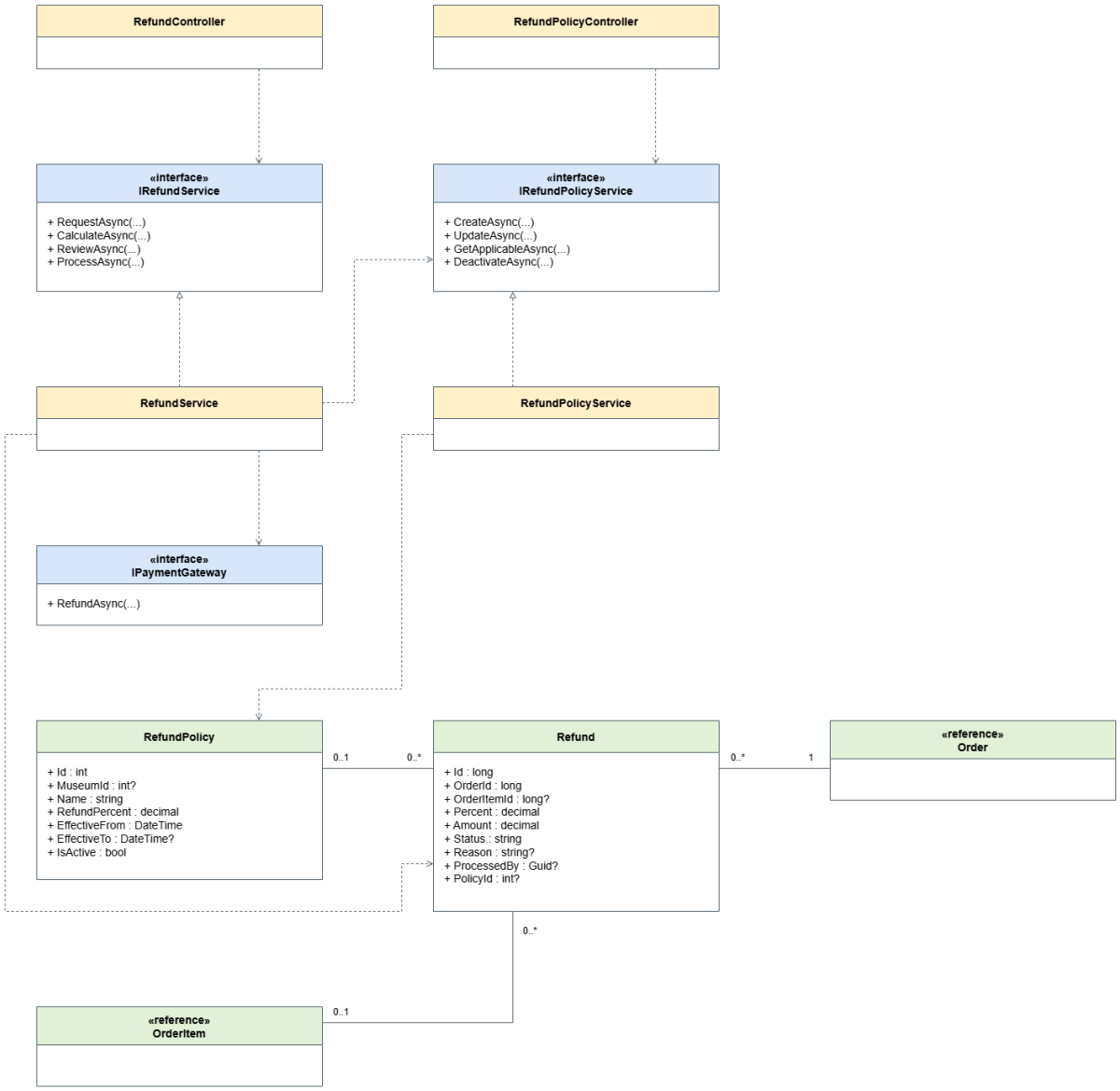
3.13.7 Sequence Diagrams-View Invoice & Transaction History



3.14 Refund Management

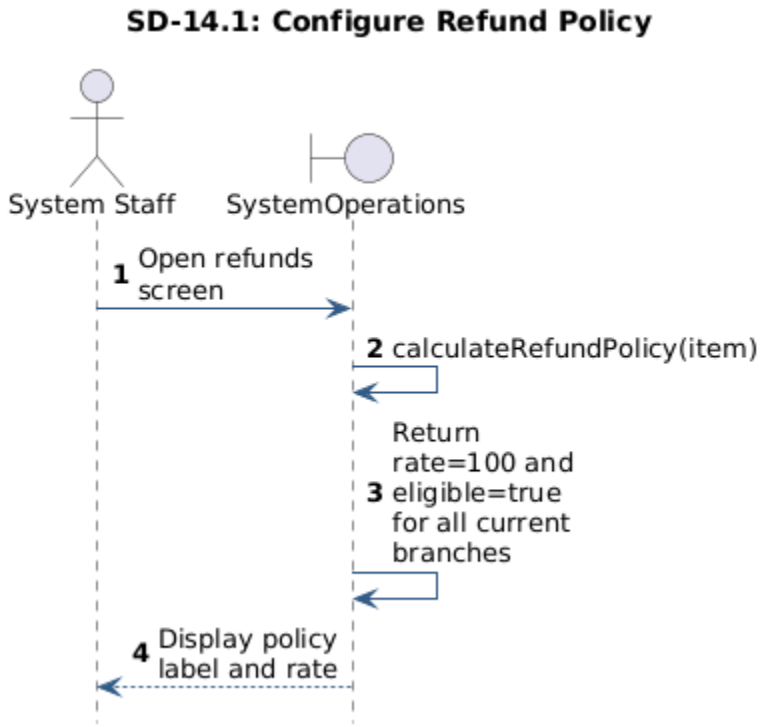
3.14.1 Class Diagram

CD-14 Class Diagram — Refund Management

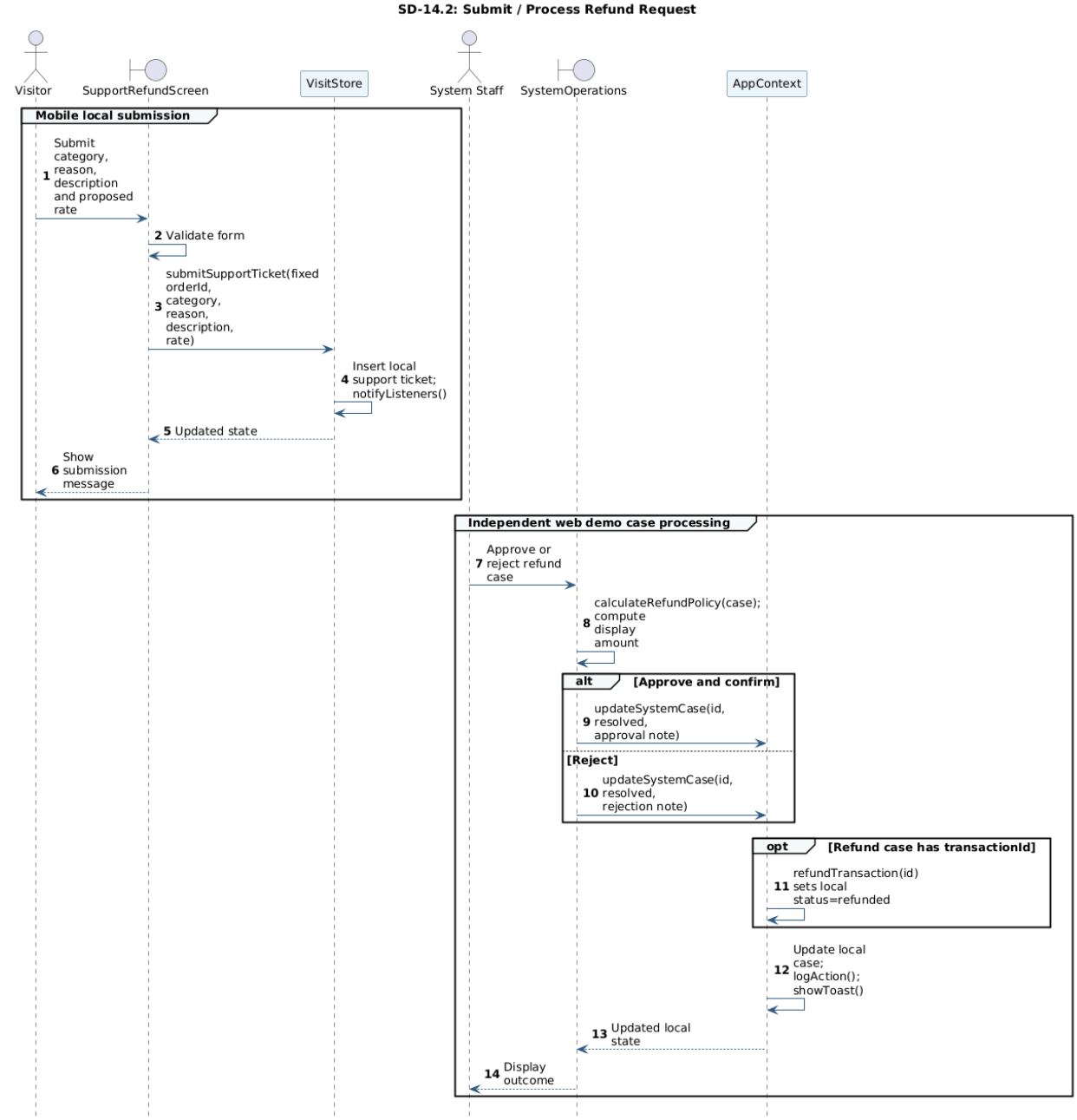


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.14.2 Sequence Diagrams-Configure Refund Policy



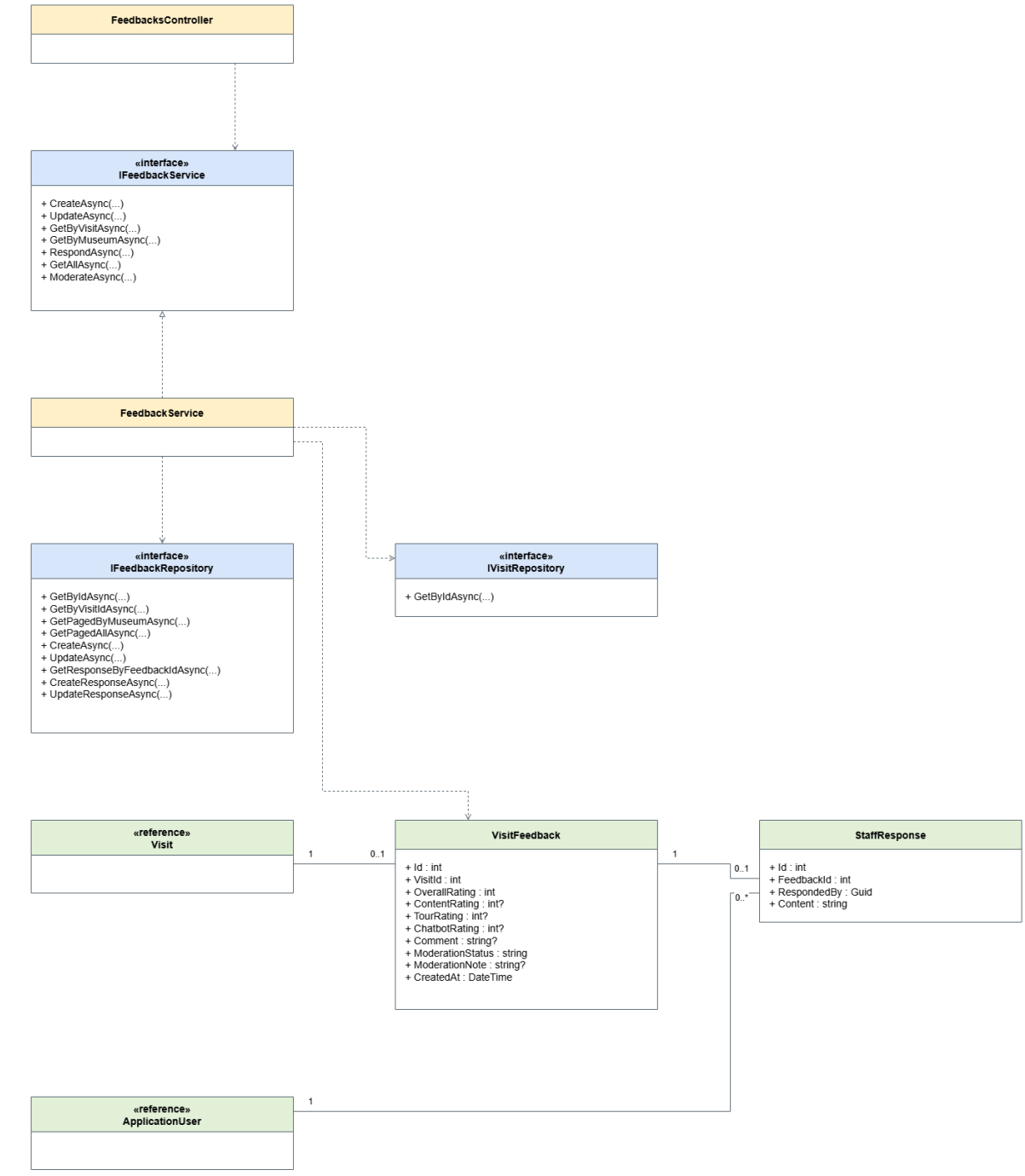
3.14.3 Sequence Diagrams-Submit / Process Refund Request



3.15 Feedback, Complaints & Support

3.15.1 Class Diagram

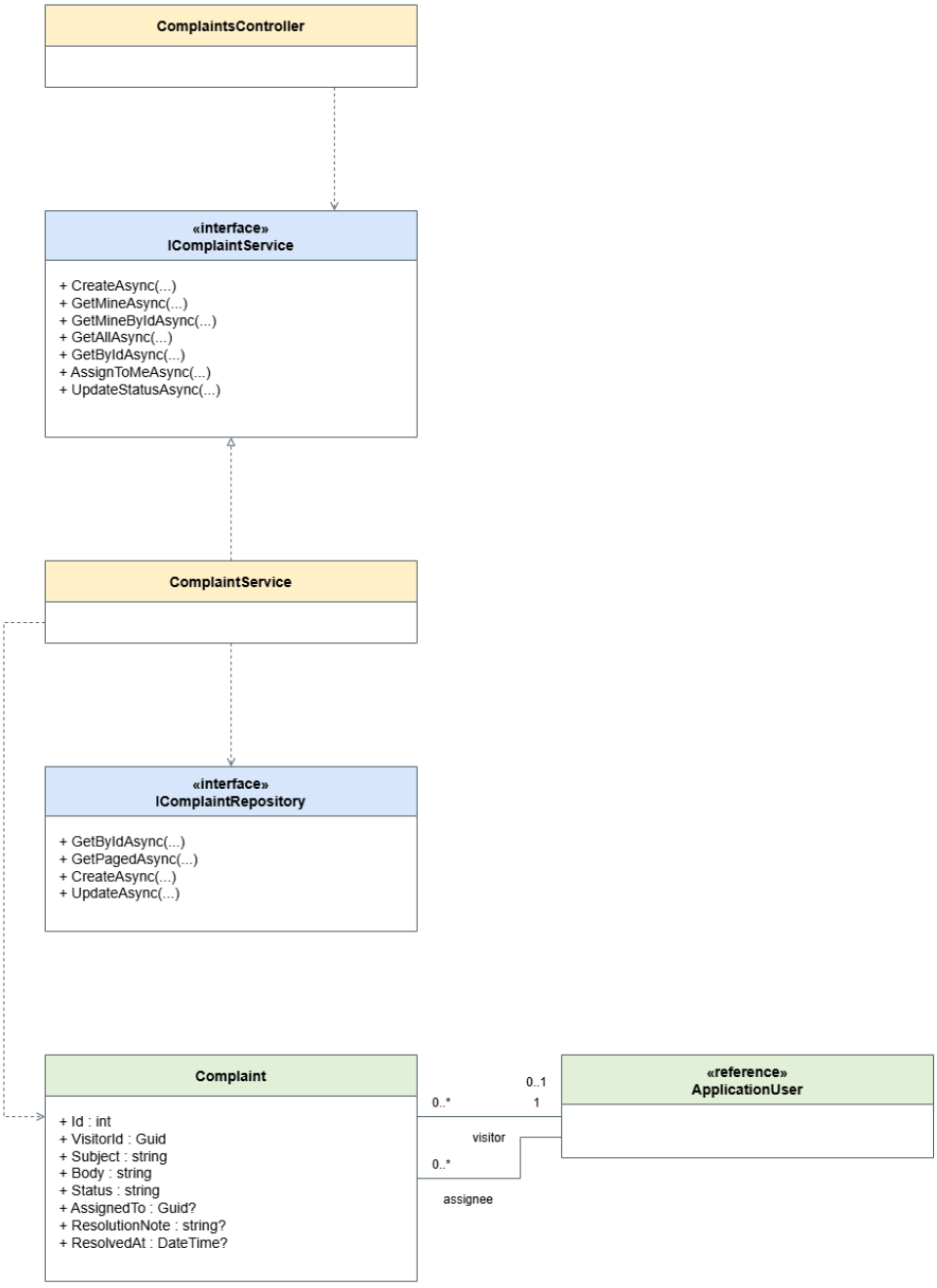
CD-15A Class Diagram — Feedback & Staff Response



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

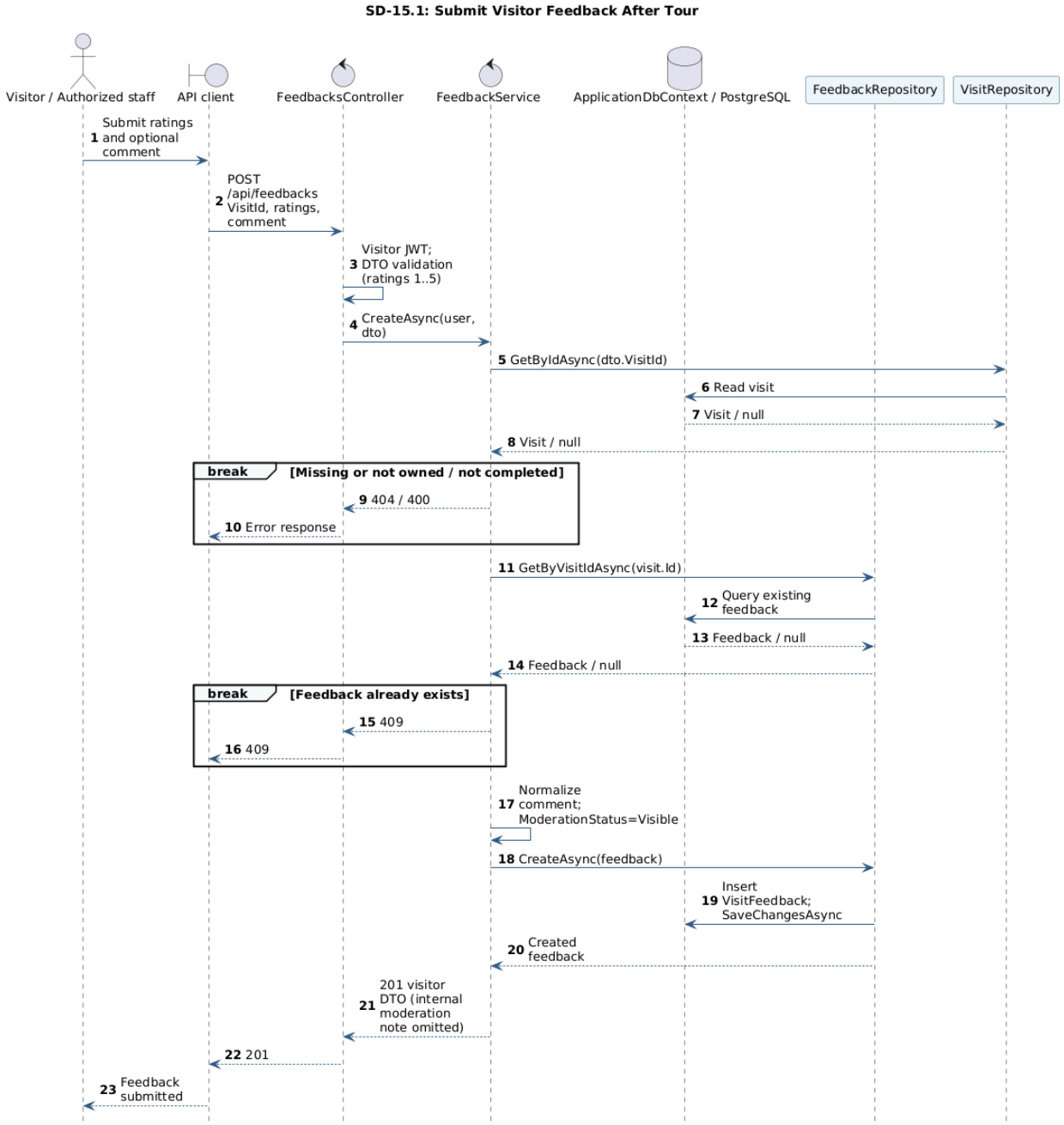
3.15.2 Class Diagram

CD-15B Class Diagram — Complaint & Support Management

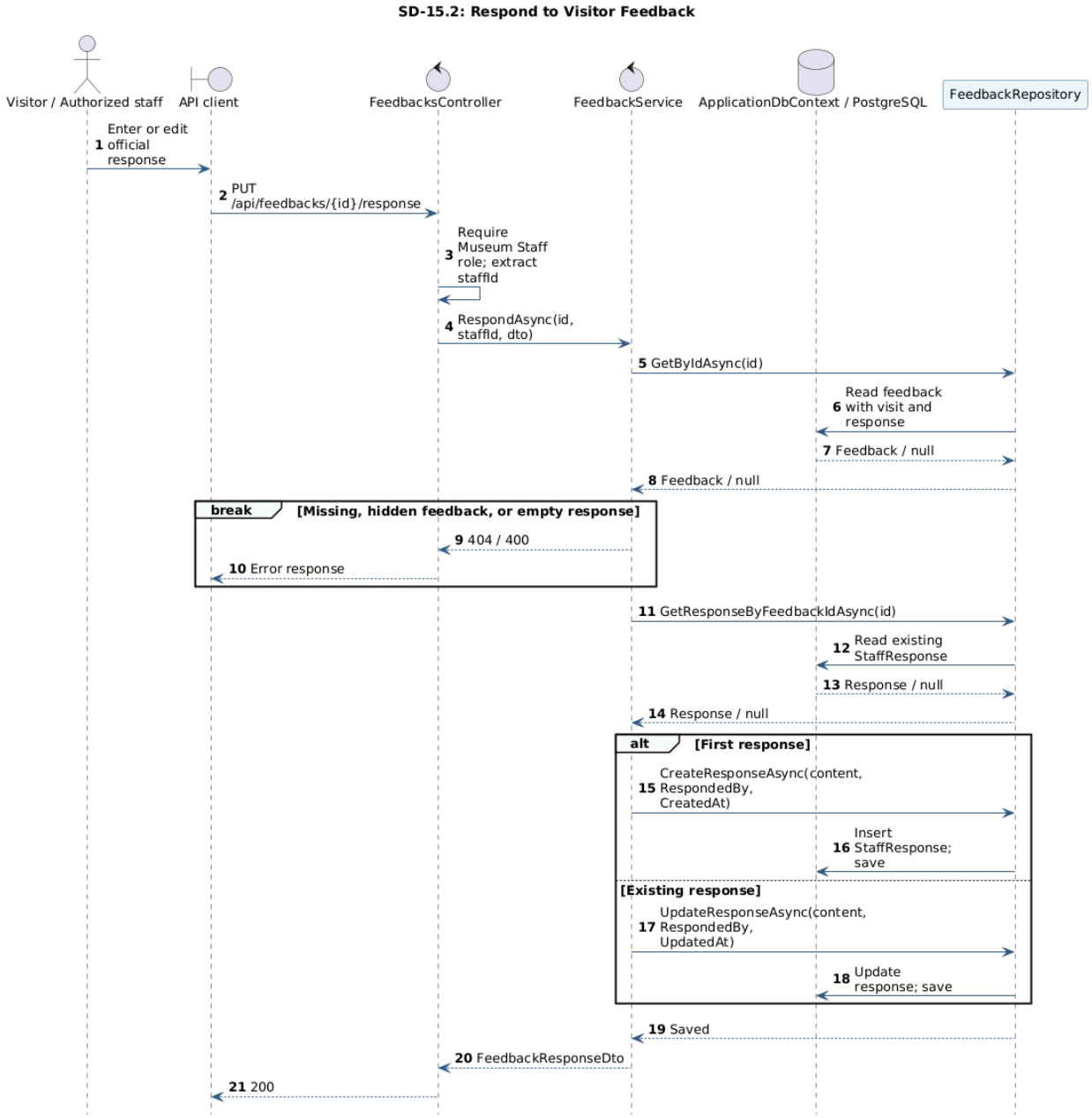


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

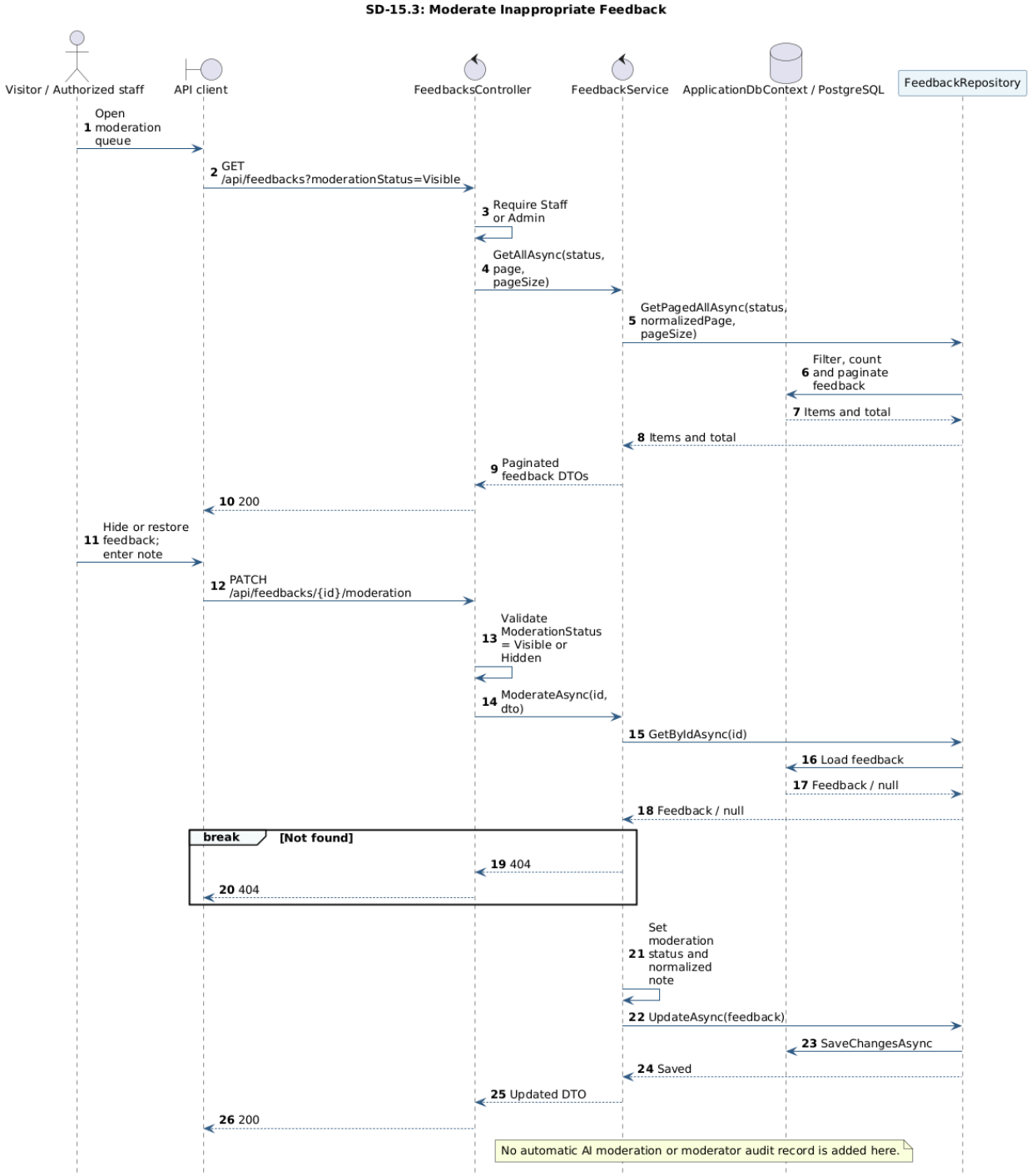
3.15.3 Sequence Diagrams-Submit Visitor Feedback After Tour



3.15.4 Sequence Diagrams-Respond to Visitor Feedback

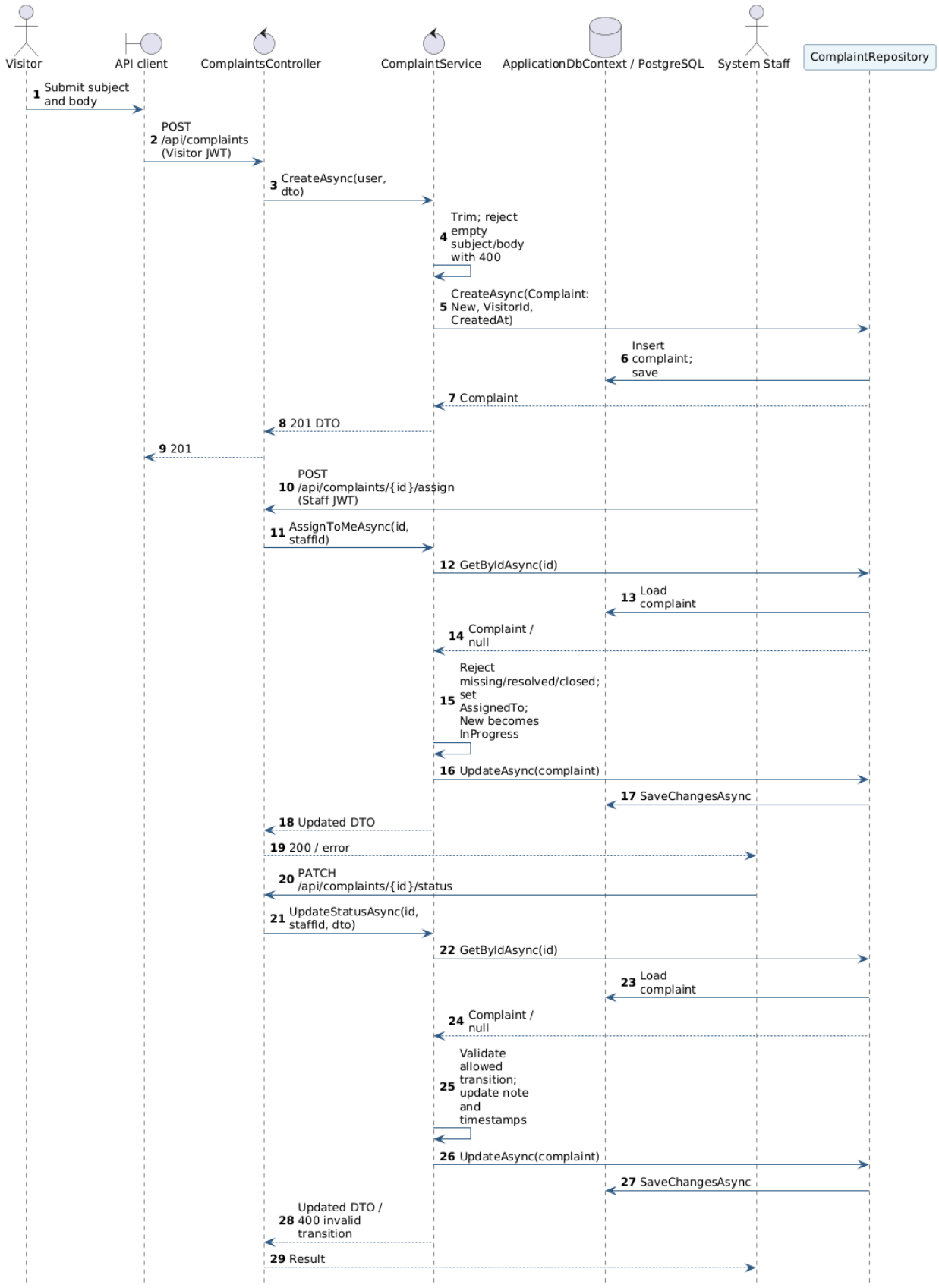


3.15.5 Sequence Diagrams-Moderate Inappropriate Feedback

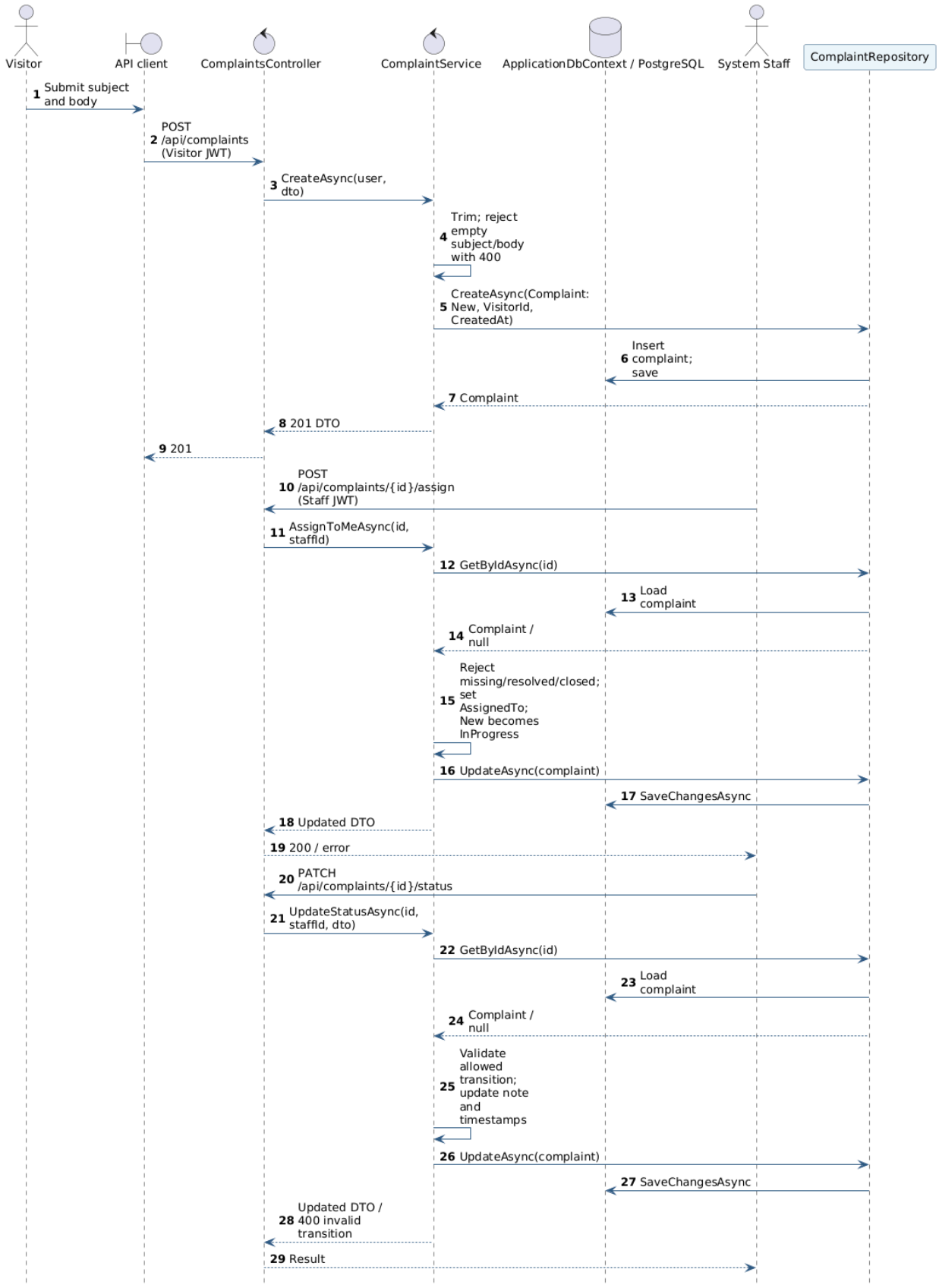


3.15.6 Sequence Diagrams-Submit / Process Visitor Complaint

SD-15.4: Submit / Process Visitor Complaint



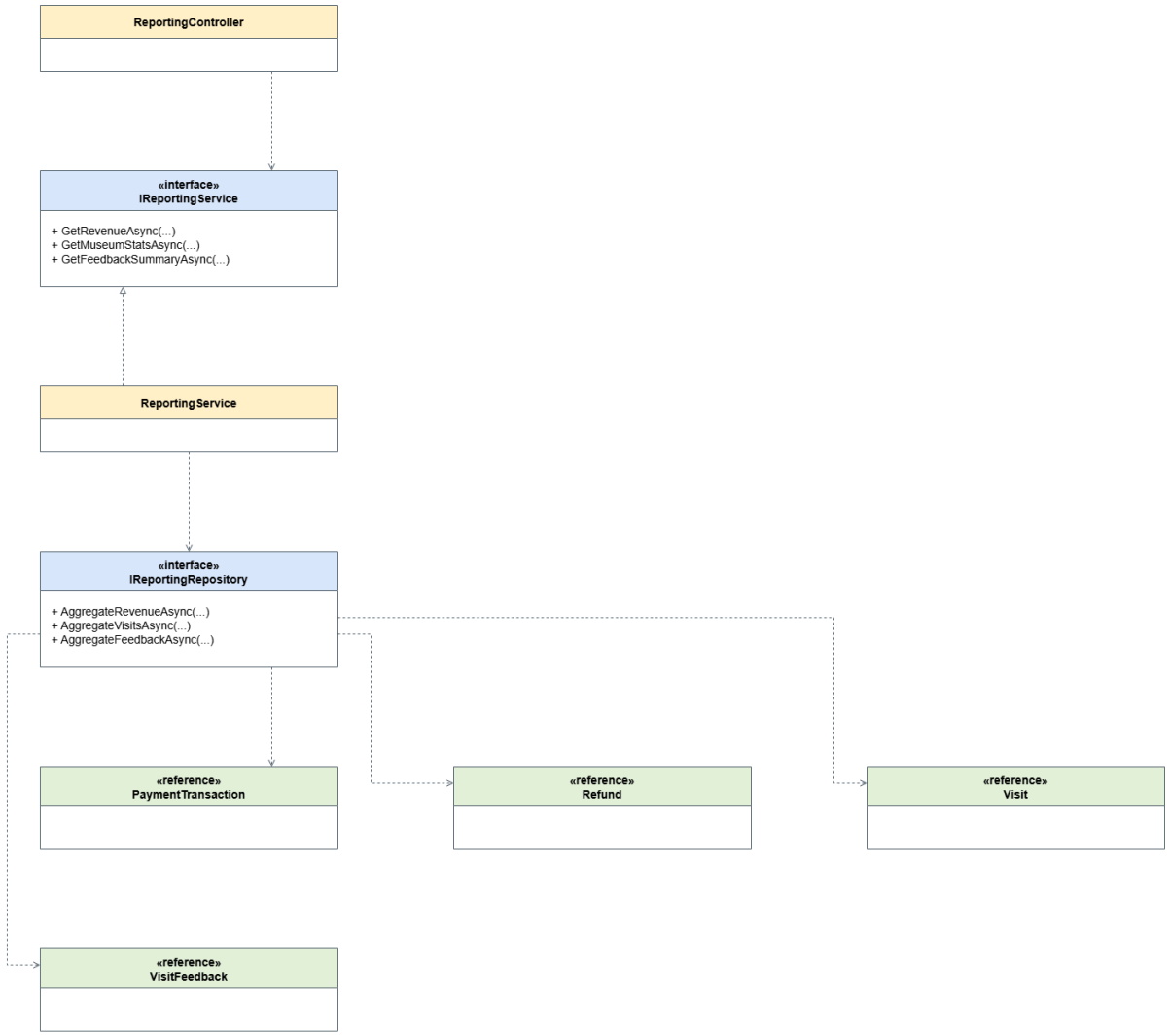
SD-15.4: Submit / Process Visitor Complaint



3.16 Reporting, Monitoring & Audit

3.16.1 Class Diagram

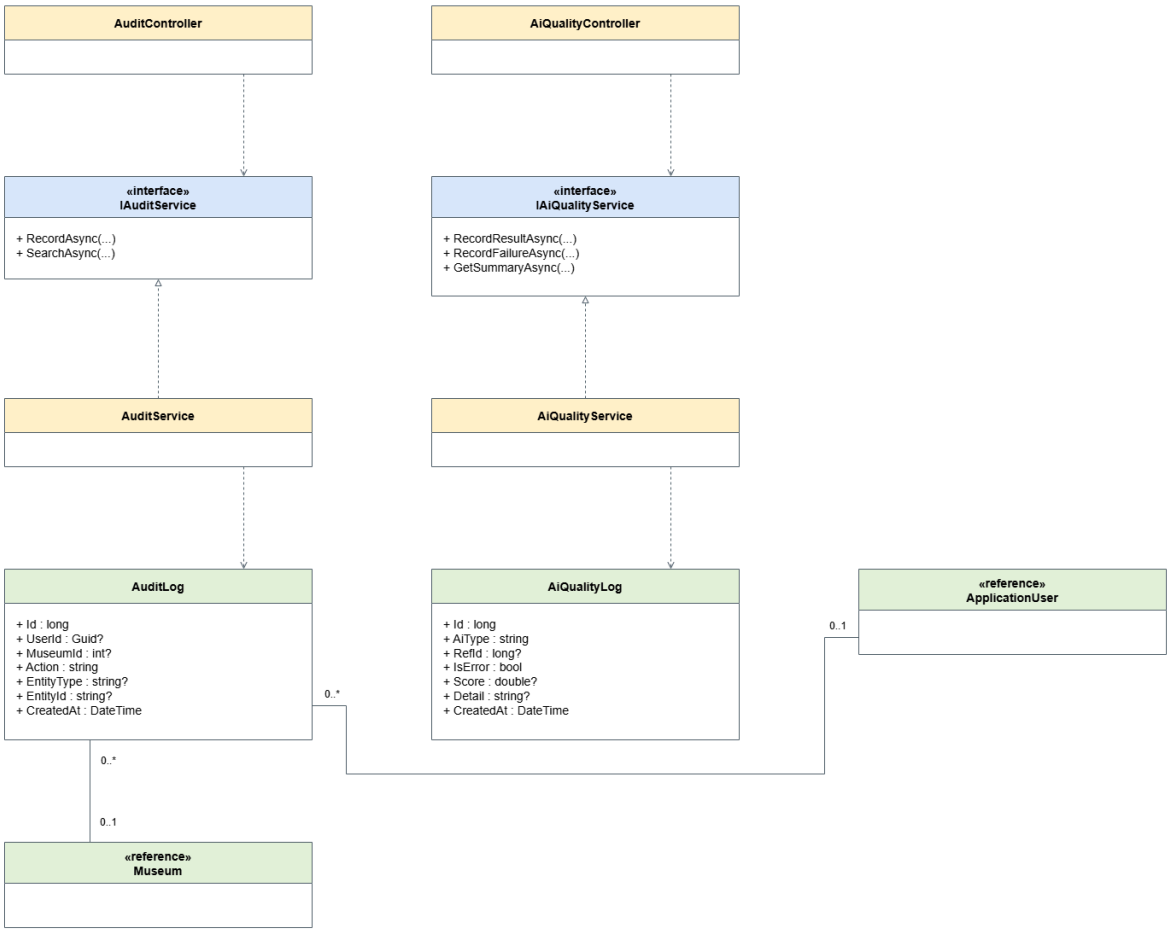
CD-16A Class Diagram — Reporting & Analytics



Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

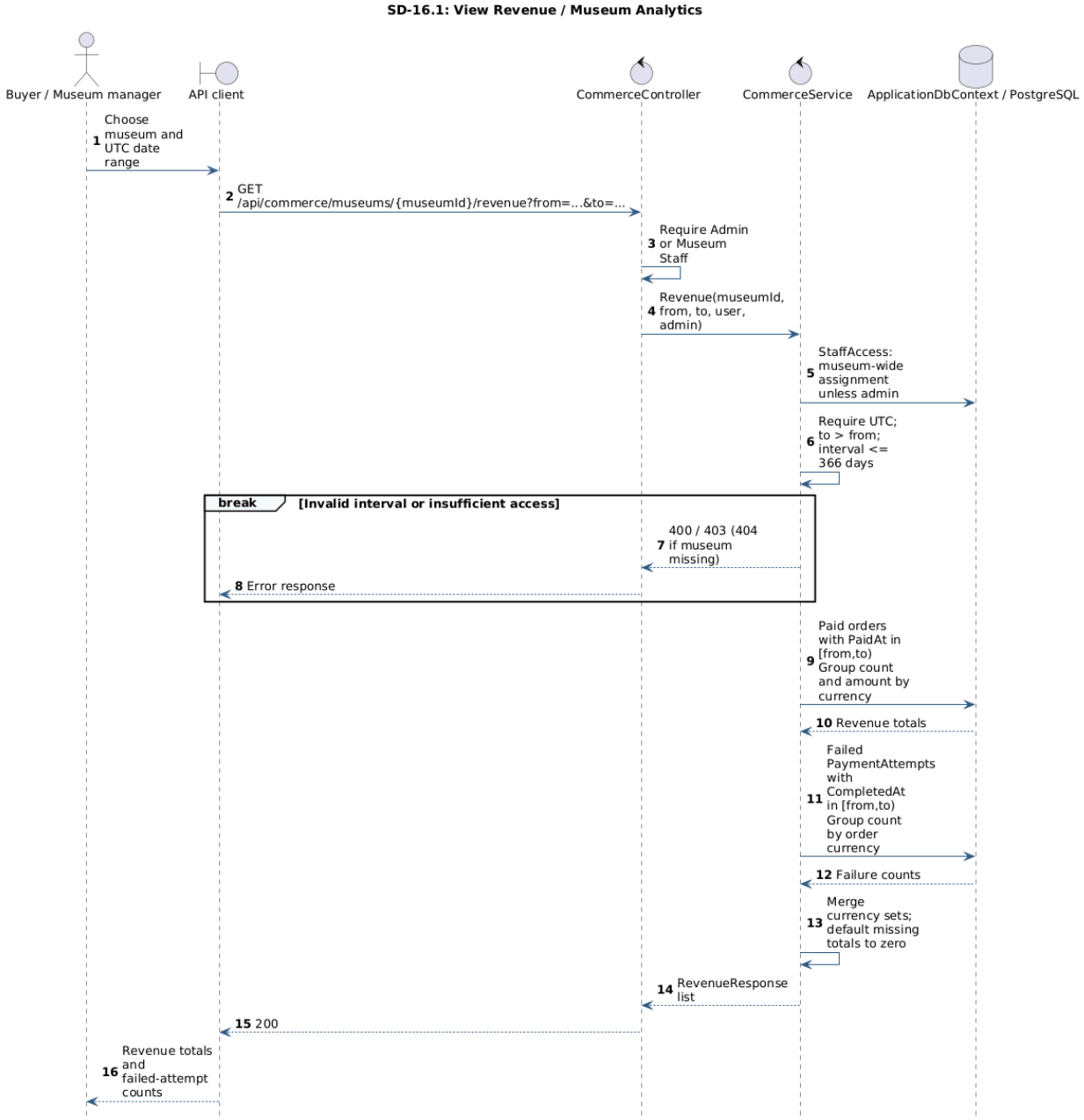
3.16.2 Class Diagram

CD-16B Class Diagram — Monitoring & Audit

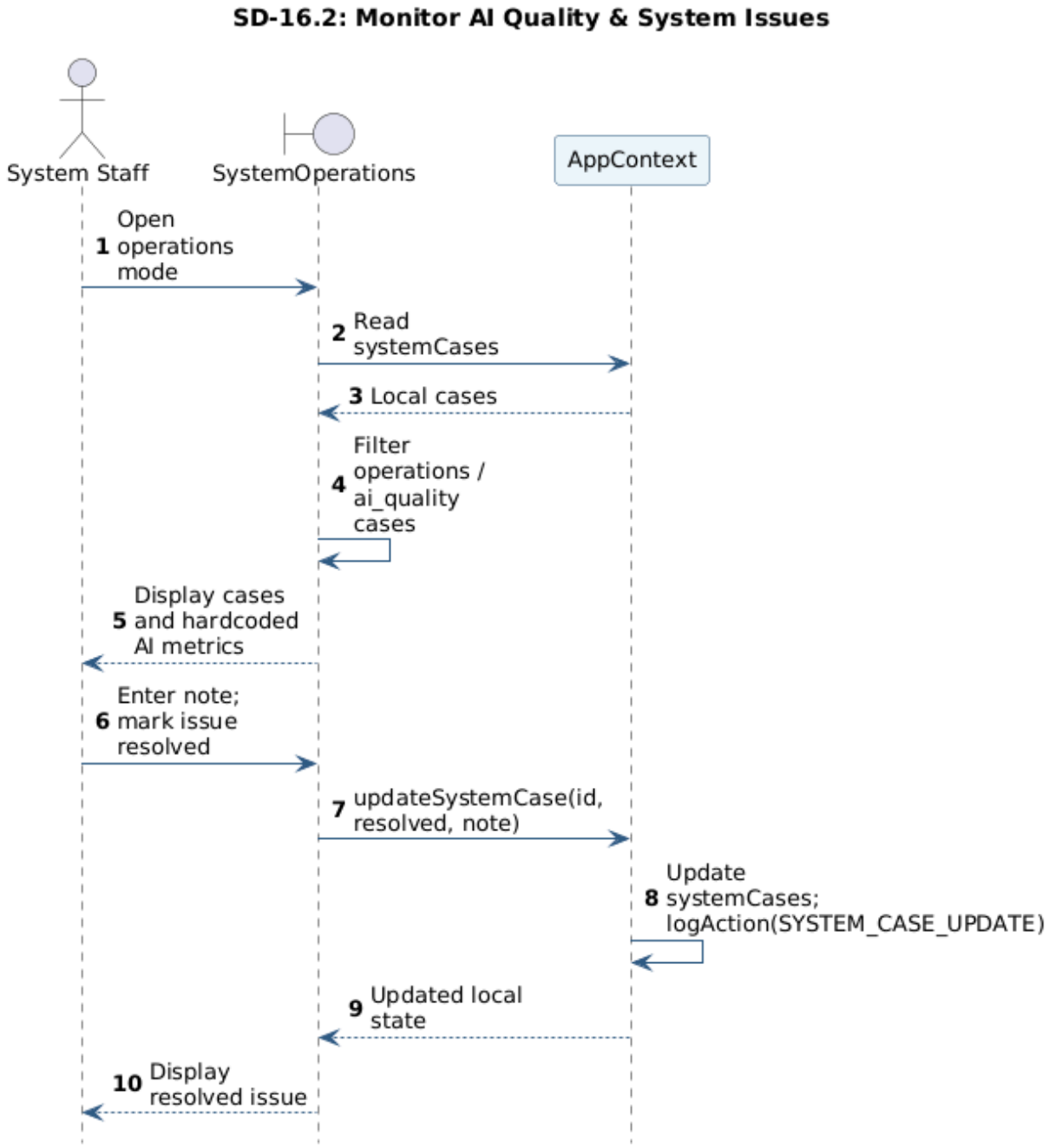


Green: entity · Blue: interface · Yellow: implementation · Gray: infrastructure / graph DTO
Solid line: association · Dashed open arrow: dependency · Dashed hollow triangle: realization · Solid hollow triangle: inheritance

3.16.3 Sequence Diagrams-View Revenue / Museum Analytics



3.16.3 Sequence Diagrams-Monitor AI Quality & System Issues



3.16.3 Sequence Diagrams-Record / View Audit Logs

